

# MicroserviceBase

**v. 2.0.0**

Nguyen Huynh Tri Cuong

09.02.2026

# Contents

|          |                                        |           |
|----------|----------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                    | <b>1</b>  |
| 1.1      | Introduction . . . . .                 | 1         |
| 1.1.1    | Target Audience . . . . .              | 1         |
| 1.1.2    | Prerequisites . . . . .                | 1         |
| <b>2</b> | <b>Description</b>                     | <b>2</b>  |
| 2.1      | Description . . . . .                  | 2         |
| 2.1.1    | Overview . . . . .                     | 2         |
| 2.1.2    | Architecture . . . . .                 | 3         |
| 2.1.3    | Components . . . . .                   | 4         |
| 2.1.4    | Communication Patterns . . . . .       | 7         |
| 2.1.5    | GUI Architecture . . . . .             | 8         |
| 2.1.6    | GUI User Guide . . . . .               | 8         |
| 2.1.7    | Process Configuration . . . . .        | 20        |
| 2.1.8    | Windows Process Lifecycle . . . . .    | 21        |
| 2.1.9    | Installation . . . . .                 | 21        |
| 2.1.10   | Quick Start . . . . .                  | 22        |
| 2.1.11   | Diagrams Reference . . . . .           | 22        |
| <b>3</b> | <b>01_basic_service.py</b>             | <b>26</b> |
| 3.1      | Function: main . . . . .               | 26        |
| 3.2      | Class: CalculatorService . . . . .     | 26        |
| 3.2.1    | Method: svc_api_add . . . . .          | 26        |
| 3.2.2    | Method: svc_api_subtract . . . . .     | 27        |
| 3.2.3    | Method: svc_api_multiply . . . . .     | 27        |
| <b>4</b> | <b>02_service_client.py</b>            | <b>28</b> |
| 4.1      | Function: main . . . . .               | 28        |
| <b>5</b> | <b>03_service_registry.py</b>          | <b>29</b> |
| 5.1      | Function: main . . . . .               | 29        |
| <b>6</b> | <b>04_eventbus_transport.py</b>        | <b>30</b> |
| 6.1      | Function: create_config_file . . . . . | 30        |
| 6.2      | Function: main . . . . .               | 30        |
| 6.3      | Class: GreeterService . . . . .        | 30        |
| 6.3.1    | Method: svc_api_hello . . . . .        | 31        |
| 6.3.2    | Method: svc_api_goodbye . . . . .      | 31        |

|           |                                |           |
|-----------|--------------------------------|-----------|
| <b>7</b>  | <b>05_alias_routing.py</b>     | <b>32</b> |
| 7.1       | Function: main                 | 32        |
| <b>8</b>  | <b>06_fastapi_bridge.py</b>    | <b>33</b> |
| 8.1       | Function: main                 | 33        |
| <b>9</b>  | <b>07_mock_fleet_api.py</b>    | <b>34</b> |
| 9.1       | Function: dashboard            | 34        |
| 9.2       | Function: fleet_status         | 34        |
| 9.3       | Function: fleet_hubs           | 34        |
| 9.4       | Function: fleet_hub_detail     | 34        |
| 9.5       | Function: fleet_hub_start      | 34        |
| 9.6       | Function: fleet_hub_stop       | 34        |
| 9.7       | Function: fleet_hub_reset      | 34        |
| 9.8       | Function: fleet_command        | 34        |
| 9.9       | Function: main                 | 34        |
| 9.10      | Class: ProcessActionRequest    | 34        |
| 9.11      | Class: CommandRequest          | 35        |
| <b>10</b> | <b>08_fleet_demo.py</b>        | <b>36</b> |
| 10.1      | Function: start_mock_fleet     | 36        |
| <b>11</b> | <b>run_hub_agent.py</b>        | <b>37</b> |
| 11.1      | Function: build_process_config | 37        |
| <b>12</b> | <b>run_orchestrator.py</b>     | <b>38</b> |
| 12.1      | Function: main                 | 38        |
| <b>13</b> | <b>serve_dev.py</b>            | <b>39</b> |
| 13.1      | Function: load_broker_config   | 39        |
| 13.2      | Class: DevHandler              | 39        |
| 13.2.1    | Method: do_POST                | 39        |
| 13.2.2    | Method: guess_type             | 39        |
| 13.2.3    | Method: log_message            | 39        |
| <b>14</b> | <b>__main__.py</b>             | <b>40</b> |
| <b>15</b> | <b>main.py</b>                 | <b>41</b> |
| 15.1      | Function: main                 | 41        |
| 15.1.1    | Method: svc_api_echo           | 41        |
| <b>16</b> | <b>launcher.py</b>             | <b>43</b> |
| 16.1      | Function: parse_args           | 43        |
| 16.2      | Function: main                 | 43        |
| <b>17</b> | <b>ClewareAccessHelper.py</b>  | <b>44</b> |
| 17.1      | Class: ClewareAccessHelper     | 44        |
| 17.1.1    | Method: load_config            | 44        |
| 17.1.2    | Method: init_cleware           | 45        |
| 17.1.3    | Method: open_cleware           | 45        |

|                                                     |           |
|-----------------------------------------------------|-----------|
| 17.1.4 Method: <code>close_cleware</code>           | 45        |
| 17.1.5 Method: <code>get_handle</code>              | 45        |
| 17.1.6 Method: <code>set_value</code>               | 45        |
| 17.1.7 Method: <code>get_value</code>               | 45        |
| 17.1.8 Method: <code>set_switch</code>              | 45        |
| 17.1.9 Method: <code>set_switch_by_port_name</code> | 45        |
| 17.1.10 Method: <code>get_switch</code>             | 45        |
| 17.1.11 Method: <code>get_version</code>            | 45        |
| 17.1.12 Method: <code>get_usb_type</code>           | 45        |
| 17.1.13 Method: <code>get_serial_number</code>      | 45        |
| 17.1.14 Method: <code>valid_ser_number</code>       | 45        |
| 17.1.15 Method: <code>get_hw_version</code>         | 45        |
| 17.1.16 Method: <code>is_ampel</code>               | 45        |
| 17.1.17 Method: <code>iox</code>                    | 45        |
| 17.1.18 Method: <code>get_all_devices_state</code>  | 45        |
| <b>18 ClewareAccessHelperAbs.py</b>                 | <b>46</b> |
| 18.1 Class: <code>ClewareAccessHelperAbs</code>     | 46        |
| 18.1.1 Method: <code>open_cleware</code>            | 46        |
| 18.1.2 Method: <code>close_cleware</code>           | 46        |
| 18.1.3 Method: <code>set_value</code>               | 46        |
| 18.1.4 Method: <code>get_value</code>               | 46        |
| 18.1.5 Method: <code>set_switch</code>              | 46        |
| 18.1.6 Method: <code>get_switch</code>              | 46        |
| 18.1.7 Method: <code>get_handle</code>              | 46        |
| 18.1.8 Method: <code>get_version</code>             | 46        |
| 18.1.9 Method: <code>get_usb_type</code>            | 46        |
| 18.1.10 Method: <code>get_usb_type</code>           | 46        |
| 18.1.11 Method: <code>get_serial_number</code>      | 46        |
| 18.1.12 Method: <code>valid_ser_number</code>       | 46        |
| 18.1.13 Method: <code>get_hw_version</code>         | 46        |
| 18.1.14 Method: <code>is_ampel</code>               | 46        |
| 18.1.15 Method: <code>iox</code>                    | 46        |
| <b>19 ClewareAccessHelperLinux.py</b>               | <b>47</b> |
| 19.1 Class: <code>ClewareAccessHelperLinux</code>   | 47        |
| 19.1.1 Method: <code>init_cleware</code>            | 48        |
| 19.1.2 Method: <code>open_cleware</code>            | 48        |
| 19.1.3 Method: <code>close_cleware</code>           | 48        |
| 19.1.4 Method: <code>get_handle</code>              | 48        |
| 19.1.5 Method: <code>set_value</code>               | 48        |
| 19.1.6 Method: <code>get_value</code>               | 48        |
| 19.1.7 Method: <code>set_switch</code>              | 48        |
| 19.1.8 Method: <code>get_switch</code>              | 48        |
| 19.1.9 Method: <code>get_version</code>             | 48        |
| 19.1.10 Method: <code>get_usb_type</code>           | 48        |
| 19.1.11 Method: <code>get_serial_number</code>      | 48        |

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| 19.1.12 Method: <code>valid_ser_number</code> . . . . .             | 48        |
| 19.1.13 Method: <code>get_hw_version</code> . . . . .               | 48        |
| 19.1.14 Method: <code>is_ampel</code> . . . . .                     | 48        |
| 19.1.15 Method: <code>iox</code> . . . . .                          | 48        |
| 19.1.16 Method: <code>get_all_sw_state</code> . . . . .             | 48        |
| <b>20 ClewareAccessHelperWindows.py</b>                             | <b>49</b> |
| 20.1 Class: <code>ClewareAccessHelperWindows</code> . . . . .       | 49        |
| 20.1.1 Method: <code>init_cleware</code> . . . . .                  | 49        |
| 20.1.2 Method: <code>open_cleware</code> . . . . .                  | 49        |
| 20.1.3 Method: <code>close_cleware</code> . . . . .                 | 49        |
| 20.1.4 Method: <code>get_handle</code> . . . . .                    | 49        |
| 20.1.5 Method: <code>set_value</code> . . . . .                     | 49        |
| 20.1.6 Method: <code>get_value</code> . . . . .                     | 49        |
| 20.1.7 Method: <code>set_switch</code> . . . . .                    | 49        |
| 20.1.8 Method: <code>get_switch</code> . . . . .                    | 49        |
| 20.1.9 Method: <code>get_version</code> . . . . .                   | 49        |
| 20.1.10 Method: <code>get_usb_type</code> . . . . .                 | 49        |
| 20.1.11 Method: <code>get_serial_number</code> . . . . .            | 49        |
| 20.1.12 Method: <code>valid_ser_number</code> . . . . .             | 49        |
| 20.1.13 Method: <code>get_hw_version</code> . . . . .               | 49        |
| 20.1.14 Method: <code>is_ampel</code> . . . . .                     | 49        |
| 20.1.15 Method: <code>iox</code> . . . . .                          | 49        |
| <b>21 ServiceCleware.py</b>                                         | <b>50</b> |
| 21.1 Class: <code>ServiceCleware</code> . . . . .                   | 50        |
| 21.1.1 Method: <code>svc_api_get_all_devices_state</code> . . . . . | 50        |
| 21.1.2 Method: <code>svc_api_set_switch</code> . . . . .            | 50        |
| 21.1.3 Method: <code>notify_updates</code> . . . . .                | 51        |
| <b>22 ServiceLogger.py</b>                                          | <b>52</b> |
| 22.1 Class: <code>ServiceLogger</code> . . . . .                    | 52        |
| 22.1.1 Method: <code>get_config_file</code> . . . . .               | 52        |
| 22.1.2 Method: <code>log</code> . . . . .                           | 52        |
| 22.1.3 Method: <code>log_info</code> . . . . .                      | 52        |
| 22.1.4 Method: <code>log_debug</code> . . . . .                     | 52        |
| 22.1.5 Method: <code>log_warning</code> . . . . .                   | 52        |
| 22.1.6 Method: <code>log_error</code> . . . . .                     | 52        |
| 22.1.7 Method: <code>log_critical</code> . . . . .                  | 52        |
| <b>23 Utils.py</b>                                                  | <b>53</b> |
| 23.1 Class: <code>Singleton</code> . . . . .                        | 53        |
| 23.2 Class: <code>Utils</code> . . . . .                            | 53        |
| 23.2.1 Method: <code>get_all_sub_classes</code> . . . . .           | 53        |
| 23.2.2 Method: <code>caller_name</code> . . . . .                   | 53        |
| 23.2.3 Method: <code>load_library</code> . . . . .                  | 54        |
| 23.2.4 Method: <code>is_ascii_or_unicode</code> . . . . .           | 54        |

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| 23.2.5 Method: <code>get_registry_parameters</code> . . . . .        | 54        |
| 23.2.6 Method: <code>create_registry_parameters</code> . . . . .     | 54        |
| 23.3 Class: <code>Job</code> . . . . .                               | 54        |
| 23.3.1 Method: <code>stop</code> . . . . .                           | 54        |
| 23.3.2 Method: <code>run</code> . . . . .                            | 54        |
| <b>24 <code>__main__.py</code></b>                                   | <b>55</b> |
| 24.1 Function: <code>main</code> . . . . .                           | 55        |
| 24.2 Function: <code>signal_handler</code> . . . . .                 | 55        |
| <b>25 <code>main.py</code></b>                                       | <b>56</b> |
| 25.1 Function: <code>main</code> . . . . .                           | 56        |
| 25.2 Function: <code>signal_handler</code> . . . . .                 | 56        |
| <b>26 <code>start_bridge.py</code></b>                               | <b>57</b> |
| 26.1 Function: <code>parse_args</code> . . . . .                     | 57        |
| 26.2 Function: <code>main</code> . . . . .                           | 57        |
| <b>27 <code>start_registry.py</code></b>                             | <b>58</b> |
| 27.1 Function: <code>parse_args</code> . . . . .                     | 58        |
| 27.2 Function: <code>main</code> . . . . .                           | 58        |
| <b>28 <code>__init__.py</code></b>                                   | <b>59</b> |
| 28.1 Function: <code>readPlist</code> . . . . .                      | 59        |
| 28.2 Function: <code>writePlist</code> . . . . .                     | 59        |
| 28.3 Function: <code>readPlistFromString</code> . . . . .            | 59        |
| 28.4 Function: <code>writePlistToString</code> . . . . .             | 59        |
| 28.5 Function: <code>is_stream_binary_plist</code> . . . . .         | 59        |
| 28.6 Class: <code>Uid</code> . . . . .                               | 59        |
| 28.6.1 Method: <code>parse</code> . . . . .                          | 61        |
| 28.6.2 Method: <code>reset</code> . . . . .                          | 61        |
| 28.6.3 Method: <code>readRoot</code> . . . . .                       | 61        |
| 28.6.4 Method: <code>setCurrentOffsetToObjectNumber</code> . . . . . | 61        |
| 28.6.5 Method: <code>beginOffsetProtection</code> . . . . .          | 61        |
| 28.6.6 Method: <code>endOffsetProtection</code> . . . . .            | 61        |
| 28.6.7 Method: <code>readObject</code> . . . . .                     | 61        |
| 28.6.8 Method: <code>readContents</code> . . . . .                   | 61        |
| 28.6.9 Method: <code>readInteger</code> . . . . .                    | 61        |
| 28.6.10 Method: <code>readReal</code> . . . . .                      | 61        |
| 28.6.11 Method: <code>readRefs</code> . . . . .                      | 61        |
| 28.6.12 Method: <code>readArray</code> . . . . .                     | 61        |
| 28.6.13 Method: <code>readDict</code> . . . . .                      | 61        |
| 28.6.14 Method: <code>readAsciiString</code> . . . . .               | 61        |
| 28.6.15 Method: <code>readUnicode</code> . . . . .                   | 61        |
| 28.6.16 Method: <code>readDate</code> . . . . .                      | 61        |
| 28.6.17 Method: <code>readData</code> . . . . .                      | 61        |
| 28.6.18 Method: <code>readUid</code> . . . . .                       | 61        |
| 28.6.19 Method: <code>getSizedInteger</code> . . . . .               | 61        |

|                                                     |           |
|-----------------------------------------------------|-----------|
| 28.7 Class: BoolWrapper . . . . .                   | 61        |
| 28.8 Class: FloatWrapper . . . . .                  | 61        |
| 28.9 Class: StringWrapper . . . . .                 | 62        |
| 28.9.1 Method: encodingMarker . . . . .             | 62        |
| 28.10 Class: PlistWriter . . . . .                  | 62        |
| 28.10.1 Method: reset . . . . .                     | 62        |
| 28.10.2 Method: positionOfObjectReference . . . . . | 62        |
| 28.10.3 Method: endRecursionProtection . . . . .    | 62        |
| 28.10.4 Method: wrapRoot . . . . .                  | 62        |
| 28.10.5 Method: incrementByteCount . . . . .        | 62        |
| 28.10.6 Method: computeOffsets . . . . .            | 62        |
| 28.10.7 Method: writeObjectReference . . . . .      | 62        |
| 28.10.8 Method: binaryInt . . . . .                 | 63        |
| 28.10.9 Method: intSize . . . . .                   | 63        |
| <b>29 badge.py . . . . .</b>                        | <b>64</b> |
| 29.1 Function: badge_disk_icon . . . . .            | 64        |
| <b>30 colors.py . . . . .</b>                       | <b>65</b> |
| 30.1 Function: isAColor . . . . .                   | 65        |
| 30.2 Function: parseColor . . . . .                 | 65        |
| 30.3 Class: Color . . . . .                         | 65        |
| 30.3.1 Method: to_rgb . . . . .                     | 65        |
| 30.4 Class: RGB . . . . .                           | 65        |
| 30.4.1 Method: to_rgb . . . . .                     | 65        |
| 30.5 Class: HSL . . . . .                           | 65        |
| 30.5.1 Method: to_rgb . . . . .                     | 65        |
| 30.6 Class: HWB . . . . .                           | 65        |
| 30.6.1 Method: to_rgb . . . . .                     | 66        |
| 30.7 Class: CMYK . . . . .                          | 66        |
| 30.7.1 Method: to_rgb . . . . .                     | 66        |
| 30.8 Class: Gray . . . . .                          | 66        |
| 30.8.1 Method: to_rgb . . . . .                     | 66        |
| 30.9 Class: ColorParser . . . . .                   | 66        |
| 30.9.1 Method: skipws . . . . .                     | 67        |
| 30.9.2 Method: expect . . . . .                     | 67        |
| 30.9.3 Method: expectEnd . . . . .                  | 67        |
| 30.9.4 Method: getToken . . . . .                   | 67        |
| 30.9.5 Method: parseNumber . . . . .                | 67        |
| 30.9.6 Method: parseColor . . . . .                 | 67        |
| 30.9.7 Method: parseRGB . . . . .                   | 67        |
| 30.9.8 Method: parseHSL . . . . .                   | 67        |
| 30.9.9 Method: parseHWB . . . . .                   | 67        |
| 30.9.10 Method: parseCMYK . . . . .                 | 67        |
| 30.9.11 Method: parseGray . . . . .                 | 67        |
| 30.9.12 Method: parseValue . . . . .                | 67        |
| 30.9.13 Method: parseAngle . . . . .                | 67        |

|                                          |           |
|------------------------------------------|-----------|
| <b>31 core.py</b>                        | <b>68</b> |
| 31.1 Function: build_dmg . . . . .       | 68        |
| 31.2 Class: DMGError . . . . .           | 68        |
| <b>32 buddy.py</b>                       | <b>69</b> |
| 32.1 Class: BuddyError . . . . .         | 69        |
| 32.2 Class: Block . . . . .              | 69        |
| 32.2.1 Method: close . . . . .           | 69        |
| 32.2.2 Method: flush . . . . .           | 69        |
| 32.2.3 Method: invalidate . . . . .      | 69        |
| 32.2.4 Method: zero_fill . . . . .       | 69        |
| 32.2.5 Method: tell . . . . .            | 69        |
| 32.2.6 Method: seek . . . . .            | 69        |
| 32.2.7 Method: read . . . . .            | 69        |
| 32.2.8 Method: write . . . . .           | 69        |
| 32.2.9 Method: insert . . . . .          | 69        |
| 32.2.10 Method: delete . . . . .         | 69        |
| 32.3 Class: Allocator . . . . .          | 69        |
| 32.3.1 Method: open . . . . .            | 70        |
| 32.3.2 Method: close . . . . .           | 70        |
| 32.3.3 Method: flush . . . . .           | 70        |
| 32.3.4 Method: read . . . . .            | 70        |
| 32.3.5 Method: allocate . . . . .        | 70        |
| 32.3.6 Method: iterkeys . . . . .        | 70        |
| 32.3.7 Method: keys . . . . .            | 70        |
| <b>33 store.py</b>                       | <b>71</b> |
| 33.1 Class: ILocCodec . . . . .          | 71        |
| 33.1.1 Method: encode . . . . .          | 71        |
| 33.1.2 Method: decode . . . . .          | 71        |
| 33.2 Class: PlistCodec . . . . .         | 71        |
| 33.2.1 Method: encode . . . . .          | 71        |
| 33.2.2 Method: decode . . . . .          | 71        |
| 33.3 Class: BookmarkCodec . . . . .      | 71        |
| 33.3.1 Method: encode . . . . .          | 71        |
| 33.3.2 Method: decode . . . . .          | 71        |
| 33.4 Class: DSStoreEntry . . . . .       | 71        |
| <b>34 alias.py</b>                       | <b>74</b> |
| 34.1 Function: encode_utf8 . . . . .     | 74        |
| 34.2 Function: decode_utf8 . . . . .     | 74        |
| 34.3 Class: AppleShareInfo . . . . .     | 74        |
| 34.4 Class: VolumeInfo . . . . .         | 74        |
| 34.4.1 Method: filesystem_type . . . . . | 74        |
| 34.5 Class: TargetInfo . . . . .         | 74        |
| 34.6 Class: Alias . . . . .              | 74        |
| 34.6.1 Method: from_bytes . . . . .      | 75        |



|                                      |           |
|--------------------------------------|-----------|
| <b>35 bookmark.py</b>                | <b>76</b> |
| 35.1 Function: iteritems             | 76        |
| 35.2 Class: Data                     | 76        |
| 35.3 Class: URL                      | 76        |
| 35.3.1 Method: absolute              | 76        |
| 35.3.2 Method: from_bytes            | 76        |
| <b>36 osx.py</b>                     | <b>77</b> |
| 36.1 Function: getattrlist           | 77        |
| 36.2 Function: fgetattrlist          | 77        |
| 36.3 Function: statfs                | 77        |
| 36.4 Function: fstatfs               | 77        |
| 36.5 Class: attrlist                 | 77        |
| 36.6 Class: attribute_set_t          | 77        |
| 36.7 Class: fsobj_id_t               | 77        |
| 36.8 Class: timespec                 | 77        |
| 36.9 Class: attrreference_t          | 78        |
| 36.10 Class: fsid_t                  | 78        |
| 36.11 Class: guid_t                  | 78        |
| 36.12 Class: kauth_ace               | 78        |
| 36.13 Class: kauth_acl               | 78        |
| 36.14 Class: kauth_filesec           | 78        |
| 36.15 Class: diskextent              | 79        |
| 36.16 Class: Point                   | 79        |
| 36.17 Class: Rect                    | 79        |
| 36.18 Class: FileInfo                | 79        |
| 36.19 Class: FolderInfo              | 79        |
| 36.20 Class: FinderInfo              | 79        |
| 36.21 Class: vol_capabilities_attr_t | 79        |
| 36.22 Class: vol_attributes_attr_t   | 80        |
| 36.23 Class: struct_statfs           | 80        |
| <b>37 utils.py</b>                   | <b>81</b> |
| 37.1 Class: UTC                      | 81        |
| 37.1.1 Method: utcoffset             | 81        |
| 37.1.2 Method: dst                   | 81        |
| 37.1.3 Method: tzname                | 81        |
| <b>38 launcher.py</b>                | <b>82</b> |
| 38.1 Function: parse_args            | 82        |
| 38.2 Function: main                  | 82        |
| <b>39 ClewareAccessHelper.py</b>     | <b>83</b> |
| 39.1 Class: ClewareAccessHelper      | 83        |
| 39.1.1 Method: load_config           | 83        |
| 39.1.2 Method: init_cleware          | 84        |
| 39.1.3 Method: open_cleware          | 84        |

|                                                  |           |
|--------------------------------------------------|-----------|
| 39.1.4 Method: close_cleware . . . . .           | 84        |
| 39.1.5 Method: get_handle . . . . .              | 84        |
| 39.1.6 Method: set_value . . . . .               | 84        |
| 39.1.7 Method: get_value . . . . .               | 84        |
| 39.1.8 Method: set_switch . . . . .              | 84        |
| 39.1.9 Method: set_switch_by_port_name . . . . . | 84        |
| 39.1.10 Method: get_switch . . . . .             | 84        |
| 39.1.11 Method: get_version . . . . .            | 84        |
| 39.1.12 Method: get_usb_type . . . . .           | 84        |
| 39.1.13 Method: get_serial_number . . . . .      | 84        |
| 39.1.14 Method: valid_ser_number . . . . .       | 84        |
| 39.1.15 Method: get_hw_version . . . . .         | 84        |
| 39.1.16 Method: is_ampel . . . . .               | 84        |
| 39.1.17 Method: iox . . . . .                    | 84        |
| 39.1.18 Method: get_all_devices_state . . . . .  | 84        |
| <b>40 ClewareAccessHelperAbs.py</b>              | <b>85</b> |
| 40.1 Class: ClewareAccessHelperAbs . . . . .     | 85        |
| 40.1.1 Method: open_cleware . . . . .            | 85        |
| 40.1.2 Method: close_cleware . . . . .           | 85        |
| 40.1.3 Method: set_value . . . . .               | 85        |
| 40.1.4 Method: get_value . . . . .               | 85        |
| 40.1.5 Method: set_switch . . . . .              | 85        |
| 40.1.6 Method: get_switch . . . . .              | 85        |
| 40.1.7 Method: get_handle . . . . .              | 85        |
| 40.1.8 Method: get_version . . . . .             | 85        |
| 40.1.9 Method: get_usb_type . . . . .            | 85        |
| 40.1.10 Method: get_usb_type . . . . .           | 85        |
| 40.1.11 Method: get_serial_number . . . . .      | 85        |
| 40.1.12 Method: valid_ser_number . . . . .       | 85        |
| 40.1.13 Method: get_hw_version . . . . .         | 85        |
| 40.1.14 Method: is_ampel . . . . .               | 85        |
| 40.1.15 Method: iox . . . . .                    | 85        |
| <b>41 ClewareAccessHelperLinux.py</b>            | <b>86</b> |
| 41.1 Class: ClewareAccessHelperLinux . . . . .   | 86        |
| 41.1.1 Method: init_cleware . . . . .            | 87        |
| 41.1.2 Method: open_cleware . . . . .            | 87        |
| 41.1.3 Method: close_cleware . . . . .           | 87        |
| 41.1.4 Method: get_handle . . . . .              | 87        |
| 41.1.5 Method: set_value . . . . .               | 87        |
| 41.1.6 Method: get_value . . . . .               | 87        |
| 41.1.7 Method: set_switch . . . . .              | 87        |
| 41.1.8 Method: get_switch . . . . .              | 87        |
| 41.1.9 Method: get_version . . . . .             | 87        |
| 41.1.10 Method: get_usb_type . . . . .           | 87        |
| 41.1.11 Method: get_serial_number . . . . .      | 87        |

|                                                        |           |
|--------------------------------------------------------|-----------|
| 41.1.12 Method: valid_ser_number . . . . .             | 87        |
| 41.1.13 Method: get_hw_version . . . . .               | 87        |
| 41.1.14 Method: is_ampel . . . . .                     | 87        |
| 41.1.15 Method: iox . . . . .                          | 87        |
| 41.1.16 Method: get_all_sw_state . . . . .             | 87        |
| <b>42 ClewareAccessHelperWindows.py</b>                | <b>88</b> |
| 42.1 Class: ClewareAccessHelperWindows . . . . .       | 88        |
| 42.1.1 Method: init_cleware . . . . .                  | 88        |
| 42.1.2 Method: open_cleware . . . . .                  | 88        |
| 42.1.3 Method: close_cleware . . . . .                 | 88        |
| 42.1.4 Method: get_handle . . . . .                    | 88        |
| 42.1.5 Method: set_value . . . . .                     | 88        |
| 42.1.6 Method: get_value . . . . .                     | 88        |
| 42.1.7 Method: set_switch . . . . .                    | 88        |
| 42.1.8 Method: get_switch . . . . .                    | 88        |
| 42.1.9 Method: get_version . . . . .                   | 88        |
| 42.1.10 Method: get_usb_type . . . . .                 | 88        |
| 42.1.11 Method: get_serial_number . . . . .            | 88        |
| 42.1.12 Method: valid_ser_number . . . . .             | 88        |
| 42.1.13 Method: get_hw_version . . . . .               | 88        |
| 42.1.14 Method: is_ampel . . . . .                     | 88        |
| 42.1.15 Method: iox . . . . .                          | 88        |
| <b>43 ServiceCleware.py</b>                            | <b>89</b> |
| 43.1 Class: ServiceCleware . . . . .                   | 89        |
| 43.1.1 Method: svc_api_get_all_devices_state . . . . . | 89        |
| 43.1.2 Method: svc_api_set_switch . . . . .            | 89        |
| 43.1.3 Method: notify_updates . . . . .                | 90        |
| <b>44 ServiceLogger.py</b>                             | <b>91</b> |
| 44.1 Class: ServiceLogger . . . . .                    | 91        |
| 44.1.1 Method: get_config_file . . . . .               | 91        |
| 44.1.2 Method: log . . . . .                           | 91        |
| 44.1.3 Method: log_info . . . . .                      | 91        |
| 44.1.4 Method: log_debug . . . . .                     | 91        |
| 44.1.5 Method: log_warning . . . . .                   | 91        |
| 44.1.6 Method: log_error . . . . .                     | 91        |
| 44.1.7 Method: log_critical . . . . .                  | 91        |
| <b>45 Utils.py</b>                                     | <b>92</b> |
| 45.1 Class: Singleton . . . . .                        | 92        |
| 45.2 Class: Utils . . . . .                            | 92        |
| 45.2.1 Method: get_all_sub_classes . . . . .           | 92        |
| 45.2.2 Method: caller_name . . . . .                   | 92        |
| 45.2.3 Method: load_library . . . . .                  | 93        |
| 45.2.4 Method: is_ascii_or_unicode . . . . .           | 93        |

|                                                                  |            |
|------------------------------------------------------------------|------------|
| 45.2.5 Method: <code>get_registry_parameters</code> . . . . .    | 93         |
| 45.2.6 Method: <code>create_registry_parameters</code> . . . . . | 93         |
| 45.3 Class: <code>Job</code> . . . . .                           | 93         |
| 45.3.1 Method: <code>stop</code> . . . . .                       | 93         |
| 45.3.2 Method: <code>run</code> . . . . .                        | 93         |
| <b>46 <code>__main__.py</code></b>                               | <b>94</b>  |
| 46.1 Function: <code>main</code> . . . . .                       | 94         |
| 46.2 Function: <code>signal_handler</code> . . . . .             | 94         |
| <b>47 <code>main.py</code></b>                                   | <b>95</b>  |
| 47.1 Function: <code>main</code> . . . . .                       | 95         |
| 47.2 Function: <code>signal_handler</code> . . . . .             | 95         |
| <b>48 <code>start_bridge.py</code></b>                           | <b>96</b>  |
| 48.1 Function: <code>parse_args</code> . . . . .                 | 96         |
| 48.2 Function: <code>main</code> . . . . .                       | 96         |
| <b>49 <code>start_registry.py</code></b>                         | <b>97</b>  |
| 49.1 Function: <code>parse_args</code> . . . . .                 | 97         |
| 49.2 Function: <code>main</code> . . . . .                       | 97         |
| <b>50 <code>eventbus_config.py</code></b>                        | <b>98</b>  |
| 50.1 Class: <code>EventBusConfig</code> . . . . .                | 98         |
| 50.1.1 Method: <code>load_config</code> . . . . .                | 98         |
| 50.1.2 Method: <code>to_config_source</code> . . . . .           | 98         |
| <b>51 <code>rabbitmq_config.py</code></b>                        | <b>99</b>  |
| 51.1 Class: <code>RabbitMQConfig</code> . . . . .                | 99         |
| 51.1.1 Method: <code>to_connection_params</code> . . . . .       | 99         |
| 51.1.2 Method: <code>from_cmd_args</code> . . . . .              | 99         |
| <b>52 <code>__init__.py</code></b>                               | <b>100</b> |
| <b>53 <code>reflect_client.py</code></b>                         | <b>101</b> |
| 53.1 Class: <code>GrpcReflectError</code> . . . . .              | 101        |
| 53.2 Class: <code>GrpcReflectClient</code> . . . . .             | 101        |
| 53.2.1 Method: <code>list_services</code> . . . . .              | 102        |
| 53.2.2 Method: <code>call_method</code> . . . . .                | 102        |
| 53.3 Class: <code>LocalProtoClient</code> . . . . .              | 102        |
| 53.3.1 Method: <code>from_search_paths</code> . . . . .          | 102        |
| 53.3.2 Method: <code>list_services</code> . . . . .              | 102        |
| <b>54 <code>local_hub_manager.py</code></b>                      | <b>104</b> |
| 54.1 Class: <code>LocalHubManager</code> . . . . .               | 104        |
| 54.1.1 Method: <code>start_hub</code> . . . . .                  | 104        |
| 54.1.2 Method: <code>stop_hub</code> . . . . .                   | 105        |
| 54.1.3 Method: <code>get_status</code> . . . . .                 | 105        |
| 54.1.4 Method: <code>start_processes</code> . . . . .            | 105        |

|                                              |            |
|----------------------------------------------|------------|
| 54.1.5 Method: stop_processes . . . . .      | 105        |
| 54.1.6 Method: get_config . . . . .          | 105        |
| 54.1.7 Method: add_config . . . . .          | 106        |
| 54.1.8 Method: update_config . . . . .       | 106        |
| 54.1.9 Method: remove_config . . . . .       | 106        |
| 54.1.10 Method: remove_service . . . . .     | 106        |
| 54.1.11 Method: get_service_log . . . . .    | 106        |
| 54.1.12 Method: get_services_dir . . . . .   | 107        |
| 54.1.13 Method: import_service . . . . .     | 107        |
| 54.1.14 Method: reset . . . . .              | 107        |
| <b>55 service_executor.py</b>                | <b>108</b> |
| 55.1 Class: ServiceExecutor . . . . .        | 108        |
| 55.1.1 Method: start . . . . .               | 108        |
| 55.1.2 Method: get_log_path . . . . .        | 108        |
| 55.1.3 Method: read_log . . . . .            | 109        |
| 55.1.4 Method: is_running . . . . .          | 109        |
| 55.1.5 Method: get_pid . . . . .             | 109        |
| 55.1.6 Method: stop . . . . .                | 109        |
| <b>56 nomad_client.py</b>                    | <b>110</b> |
| 56.1 Class: NomadAPIError . . . . .          | 110        |
| 56.1.1 Method: agent_self . . . . .          | 110        |
| 56.1.2 Method: list_jobs_blocking . . . . .  | 111        |
| <b>57 nomad_hub_adapter.py</b>               | <b>112</b> |
| 57.1 Class: NomadHubAdapter . . . . .        | 112        |
| 57.1.1 Method: start_hub . . . . .           | 112        |
| 57.1.2 Method: stop_hub . . . . .            | 112        |
| 57.1.3 Method: get_status . . . . .          | 112        |
| 57.1.4 Method: start_processes . . . . .     | 112        |
| 57.1.5 Method: stop_processes . . . . .      | 112        |
| 57.1.6 Method: get_config . . . . .          | 112        |
| 57.1.7 Method: add_config . . . . .          | 112        |
| 57.1.8 Method: update_config . . . . .       | 112        |
| 57.1.9 Method: remove_config . . . . .       | 112        |
| 57.1.10 Method: get_service_log . . . . .    | 112        |
| 57.1.11 Method: import_service . . . . .     | 112        |
| 57.1.12 Method: remove_service . . . . .     | 112        |
| 57.1.13 Method: reset . . . . .              | 112        |
| <b>58 amqp_registry_adapter.py</b>           | <b>113</b> |
| 58.1 Class: AMQPRegistryAdapter . . . . .    | 113        |
| 58.1.1 Method: register . . . . .            | 113        |
| 58.1.2 Method: unregister . . . . .          | 113        |
| 58.1.3 Method: subscribe_to_events . . . . . | 113        |
| 58.1.4 Method: notify_update . . . . .       | 114        |

|                                                               |            |
|---------------------------------------------------------------|------------|
| 58.1.5 Method: <code>subscribe_to_updates</code> . . . . .    | 114        |
| 58.1.6 Method: <code>get_update_channel_name</code> . . . . . | 114        |
| 58.1.7 Method: <code>cleanup_update_exchange</code> . . . . . | 114        |
| 58.1.8 Method: <code>cleanup</code> . . . . .                 | 114        |
| <b>59 <code>eventbus_registry_adapter.py</code></b>           | <b>115</b> |
| 59.1 Class: <code>EventBusRegistryAdapter</code> . . . . .    | 115        |
| 59.1.1 Method: <code>register</code> . . . . .                | 115        |
| 59.1.2 Method: <code>unregister</code> . . . . .              | 115        |
| 59.1.3 Method: <code>subscribe_to_events</code> . . . . .     | 115        |
| 59.1.4 Method: <code>notify_update</code> . . . . .           | 116        |
| 59.1.5 Method: <code>get_update_channel_name</code> . . . . . | 116        |
| 59.1.6 Method: <code>cleanup</code> . . . . .                 | 116        |
| <b>60 <code>__init__.py</code></b>                            | <b>117</b> |
| <b>61 <code>cpp_tmpl.py</code></b>                            | <b>118</b> |
| 61.1 Function: <code>generate</code> . . . . .                | 118        |
| 61.2 Function: <code>generate_monorepo</code> . . . . .       | 118        |
| <b>62 <code>generator.py</code></b>                           | <b>119</b> |
| 62.1 Function: <code>generate_scaffold</code> . . . . .       | 119        |
| 62.2 Class: <code>MethodSpec</code> . . . . .                 | 119        |
| 62.3 Class: <code>ServiceBlock</code> . . . . .               | 119        |
| 62.4 Class: <code>ScaffoldSpec</code> . . . . .               | 119        |
| 62.4.1 Method: <code>proto_namespace</code> . . . . .         | 120        |
| <b>63 <code>python_tmpl.py</code></b>                         | <b>121</b> |
| 63.1 Function: <code>generate</code> . . . . .                | 121        |
| 63.2 Function: <code>generate_monorepo</code> . . . . .       | 121        |
| <b>64 <code>shared.py</code></b>                              | <b>122</b> |
| 64.1 Function: <code>gen_proto</code> . . . . .               | 122        |
| 64.2 Function: <code>gen_service_config</code> . . . . .      | 122        |
| 64.3 Function: <code>gen_nomad</code> . . . . .               | 122        |
| 64.4 Function: <code>gen_readme</code> . . . . .              | 122        |
| <b>65 <code>eventbus_adapter.py</code></b>                    | <b>123</b> |
| 65.1 Class: <code>_ChannelProxy</code> . . . . .              | 123        |
| 65.1.1 Method: <code>basic_publish</code> . . . . .           | 123        |
| 65.1.2 Method: <code>basic_ack</code> . . . . .               | 123        |
| 65.2 Class: <code>_MethodProxy</code> . . . . .               | 123        |
| 65.3 Class: <code>_PropertiesProxy</code> . . . . .           | 124        |
| 65.4 Class: <code>EventBusTransportAdapter</code> . . . . .   | 124        |
| 65.4.1 Method: <code>connect</code> . . . . .                 | 124        |
| 65.4.2 Method: <code>disconnect</code> . . . . .              | 124        |
| 65.4.3 Method: <code>consume</code> . . . . .                 | 124        |
| 65.4.4 Method: <code>rpc_call</code> . . . . .                | 124        |
| 65.4.5 Method: <code>publish</code> . . . . .                 | 125        |

|           |                                 |            |
|-----------|---------------------------------|------------|
| <b>66</b> | <b>rabbitmq_adapter.py</b>      | <b>126</b> |
| 66.1      | Class: RabbitMQTransportAdapter | 126        |
| 66.1.1    | Method: connect                 | 126        |
| 66.1.2    | Method: disconnect              | 126        |
| 66.1.3    | Method: connection              | 126        |
| 66.1.4    | Method: stop_consuming          | 126        |
| 66.1.5    | Method: consume                 | 126        |
| 66.1.6    | Method: rpc_call                | 127        |
| 66.1.7    | Method: publish                 | 127        |
| <b>67</b> | <b>fastapi_bridge.py</b>        | <b>128</b> |
| 67.1      | Class: FastAPIBridge            | 128        |
| 67.1.1    | Method: start                   | 128        |
| 67.1.2    | Method: wait                    | 128        |
| 67.1.3    | Method: stop                    | 128        |
| 67.1.4    | Method: broadcast_update        | 128        |
| <b>68</b> | <b>exceptions.py</b>            | <b>130</b> |
| 68.1      | Class: ServiceError             | 130        |
| 68.2      | Class: TransportError           | 130        |
| 68.3      | Class: ServiceNotFoundError     | 130        |
| 68.4      | Class: MethodNotFoundError      | 130        |
| <b>69</b> | <b>messages.py</b>              | <b>131</b> |
| 69.1      | Class: ResultType               | 131        |
| 69.2      | Class: ServiceRequest           | 131        |
| 69.2.1    | Method: to_dict                 | 131        |
| 69.2.2    | Method: to_json                 | 131        |
| 69.2.3    | Method: from_dict               | 131        |
| 69.2.4    | Method: from_json               | 132        |
| 69.3      | Class: ServiceResponse          | 132        |
| 69.3.1    | Method: to_dict                 | 132        |
| 69.3.2    | Method: to_json                 | 132        |
| 69.3.3    | Method: get_json                | 132        |
| 69.3.4    | Method: get_data                | 133        |
| 69.3.5    | Method: from_dict               | 133        |
| 69.3.6    | Method: from_json               | 133        |
| 69.4      | Class: ServiceEvent             | 133        |
| 69.4.1    | Method: to_dict                 | 133        |
| 69.4.2    | Method: to_json                 | 134        |
| 69.4.3    | Method: from_dict               | 134        |
| 69.4.4    | Method: from_json               | 134        |
| <b>70</b> | <b>models.py</b>                | <b>135</b> |
| 70.1      | Class: MethodArgument           | 135        |
| 70.1.1    | Method: to_dict                 | 135        |
| 70.1.2    | Method: from_dict               | 135        |

|                                                               |            |
|---------------------------------------------------------------|------------|
| 70.2 Class: ServiceMethod . . . . .                           | 135        |
| 70.2.1 Method: to_dict . . . . .                              | 136        |
| 70.2.2 Method: from_dict . . . . .                            | 136        |
| 70.3 Class: ServiceInfo . . . . .                             | 136        |
| 70.3.1 Method: to_dict . . . . .                              | 136        |
| 70.3.2 Method: from_dict . . . . .                            | 136        |
| <b>71 service_base.py</b>                                     | <b>137</b> |
| 71.1 Class: ServiceBase . . . . .                             | 137        |
| 71.1.1 Method: get_service_info . . . . .                     | 137        |
| 71.1.2 Method: get_service_info_dict . . . . .                | 137        |
| 71.1.3 Method: serve . . . . .                                | 137        |
| 71.1.4 Method: register_service . . . . .                     | 137        |
| 71.1.5 Method: unregister_service . . . . .                   | 137        |
| 71.1.6 Method: request_service . . . . .                      | 137        |
| 71.1.7 Method: close . . . . .                                | 138        |
| 71.1.8 Method: create_request_data . . . . .                  | 138        |
| 71.1.9 Method: get_svc_api_methods_dict . . . . .             | 138        |
| 71.1.10 Method: get_svc_api_methods_info_dict . . . . .       | 138        |
| 71.1.11 Method: parse_docstring . . . . .                     | 138        |
| 71.1.12 Method: svc_api_get_version . . . . .                 | 138        |
| 71.1.13 Method: svc_api_get_gui_files . . . . .               | 138        |
| 71.1.14 Method: svc_api_get_service_files . . . . .           | 138        |
| 71.1.15 Method: svc_api_get_gui_checksum . . . . .            | 139        |
| 71.1.16 Method: svc_api_shutdown . . . . .                    | 139        |
| 71.1.17 Method: is_specific_request . . . . .                 | 139        |
| 71.1.18 Method: on_specific_request . . . . .                 | 139        |
| 71.1.19 Method: dispatch_request . . . . .                    | 139        |
| 71.1.20 Method: on_request . . . . .                          | 140        |
| <b>72 service_registry.py</b>                                 | <b>141</b> |
| 72.1 Class: ServiceRegistry . . . . .                         | 141        |
| 72.1.1 Method: handle_update . . . . .                        | 141        |
| 72.1.2 Method: notify_updates . . . . .                       | 141        |
| 72.1.3 Method: svc_api_shutdown . . . . .                     | 141        |
| 72.1.4 Method: svc_api_get_services_info . . . . .            | 141        |
| 72.1.5 Method: svc_api_get_realtime_update_exchange . . . . . | 141        |
| 72.1.6 Method: svc_api_update_alias_conf . . . . .            | 142        |
| 72.1.7 Method: svc_api_get_alias_conf . . . . .               | 142        |
| 72.1.8 Method: is_specific_request . . . . .                  | 142        |
| 72.1.9 Method: on_specific_request . . . . .                  | 142        |
| 72.1.10 Method: handle_alias_request . . . . .                | 142        |
| 72.1.11 Method: run_health_check_loop . . . . .               | 142        |
| 72.1.12 Method: stop_health_check . . . . .                   | 143        |
| <b>73 factory.py</b>                                          | <b>144</b> |
| 73.1 Function: create_transport . . . . .                     | 144        |



|                                                  |            |
|--------------------------------------------------|------------|
| 73.2 Function: create_registry . . . . .         | 144        |
| 73.3 Function: create_ui_bridge . . . . .        | 145        |
| <b>74 hub_manager.py</b>                         | <b>146</b> |
| 74.1 Class: HubManagerPort . . . . .             | 146        |
| 74.1.1 Method: start_hub . . . . .               | 146        |
| 74.1.2 Method: stop_hub . . . . .                | 146        |
| 74.1.3 Method: get_status . . . . .              | 146        |
| 74.1.4 Method: start_processes . . . . .         | 146        |
| 74.1.5 Method: stop_processes . . . . .          | 147        |
| 74.1.6 Method: get_config . . . . .              | 147        |
| 74.1.7 Method: add_config . . . . .              | 147        |
| 74.1.8 Method: update_config . . . . .           | 147        |
| 74.1.9 Method: remove_config . . . . .           | 148        |
| 74.1.10 Method: get_service_log . . . . .        | 148        |
| 74.1.11 Method: import_service . . . . .         | 148        |
| 74.1.12 Method: remove_service . . . . .         | 148        |
| 74.1.13 Method: reset . . . . .                  | 148        |
| <b>75 registry.py</b>                            | <b>149</b> |
| 75.1 Class: ServiceRegistryPort . . . . .        | 149        |
| 75.1.1 Method: register . . . . .                | 149        |
| 75.1.2 Method: unregister . . . . .              | 149        |
| 75.1.3 Method: subscribe_to_events . . . . .     | 149        |
| 75.1.4 Method: notify_update . . . . .           | 150        |
| 75.1.5 Method: get_update_channel_name . . . . . | 150        |
| <b>76 transport.py</b>                           | <b>151</b> |
| 76.1 Class: TransportPort . . . . .              | 151        |
| 76.1.1 Method: connect . . . . .                 | 151        |
| 76.1.2 Method: disconnect . . . . .              | 151        |
| 76.1.3 Method: stop_consuming . . . . .          | 151        |
| 76.1.4 Method: consume . . . . .                 | 151        |
| 76.1.5 Method: rpc_call . . . . .                | 152        |
| 76.1.6 Method: publish . . . . .                 | 152        |
| <b>77 ui_bridge.py</b>                           | <b>153</b> |
| 77.1 Class: UIBridgePort . . . . .               | 153        |
| 77.1.1 Method: start . . . . .                   | 153        |
| 77.1.2 Method: stop . . . . .                    | 153        |
| 77.1.3 Method: broadcast_update . . . . .        | 153        |
| <b>78 __init__.py</b>                            | <b>154</b> |
| <b>79 consul.py</b>                              | <b>155</b> |
| 79.1 Class: ConsulRegistration . . . . .         | 155        |
| 79.1.1 Method: service_id . . . . .              | 155        |
| <b>80 reflection.py</b>                          | <b>156</b> |

|                                                         |            |
|---------------------------------------------------------|------------|
| 80.1 Function: <code>enable_reflection</code> . . . . . | 156        |
| <b>81 <code>server.py</code></b>                        | <b>157</b> |
| 81.1 Class: <code>ServicerEntry</code> . . . . .        | 157        |
| 81.2 Class: <code>ServiceRunner</code> . . . . .        | 158        |
| <b>82 <code>settings.py</code></b>                      | <b>159</b> |
| 82.1 Class: <code>BaseServiceSettings</code> . . . . .  | 159        |
| <b>83 <code>scaffold_cli.py</code></b>                  | <b>160</b> |
| 83.1 Function: <code>main</code> . . . . .              | 160        |
| <b>84 Appendix</b>                                      | <b>161</b> |
| 84.1 Appendix . . . . .                                 | 161        |
| 84.1.1 License . . . . .                                | 161        |
| 84.1.2 References . . . . .                             | 161        |
| 84.1.3 Repository . . . . .                             | 161        |
| 84.1.4 Contact . . . . .                                | 161        |
| <b>85 History</b>                                       | <b>162</b> |
| 85.1 History . . . . .                                  | 162        |
| 85.1.1 Version 2.0.0 - 09.02.2026 . . . . .             | 162        |
| 85.1.2 Version 0.1.0 - 02/2023 . . . . .                | 163        |

# Chapter 1

## Introduction

### 1.1 Introduction

**MicroserviceBase** is a Python framework for building, managing, and orchestrating microservices with RabbitMQ-based communication.

It provides:

- **Microservice Framework:** Base classes for creating services with RPC and pub/sub patterns
- **Service Registry:** Dynamic service registration and discovery via RabbitMQ exchange
- **FastAPI Bridge:** REST API and WebSocket gateway connecting GUI to microservices
- **Manager GUI:** Electron and browser-based dashboard for monitoring and controlling services
- **Local Process Hub:** Process lifecycle management with graceful shutdown
- **Hexagonal Architecture:** Clean separation via ports and adapters pattern

#### 1.1.1 Target Audience

This package is designed for developers who need to:

- Build microservices that communicate via RabbitMQ message broker
- Monitor and manage services through a graphical dashboard
- Manage local service processes with start, stop, and import capabilities
- Deploy a standalone desktop application for service management

#### 1.1.2 Prerequisites

- Python 3.10 or higher
- RabbitMQ server (message broker)
- pika package (RabbitMQ client for Python)
- FastAPI and uvicorn (for the bridge)
- Node.js and npm (for GUI development)

The sources of **MicroserviceBase** are available in [GitHub](#).

Information about how to install the **MicroserviceBase** can be found in the [README](#).

# Chapter 2

## Description

### 2.1 Description

#### 2.1.1 Overview

MicroserviceBase is a Python framework for building microservices that communicate through RabbitMQ, with a built-in GUI for monitoring, a local process hub for lifecycle management, and a service registry for dynamic discovery. The system follows hexagonal architecture principles, ensuring testability, maintainability, and extensibility.

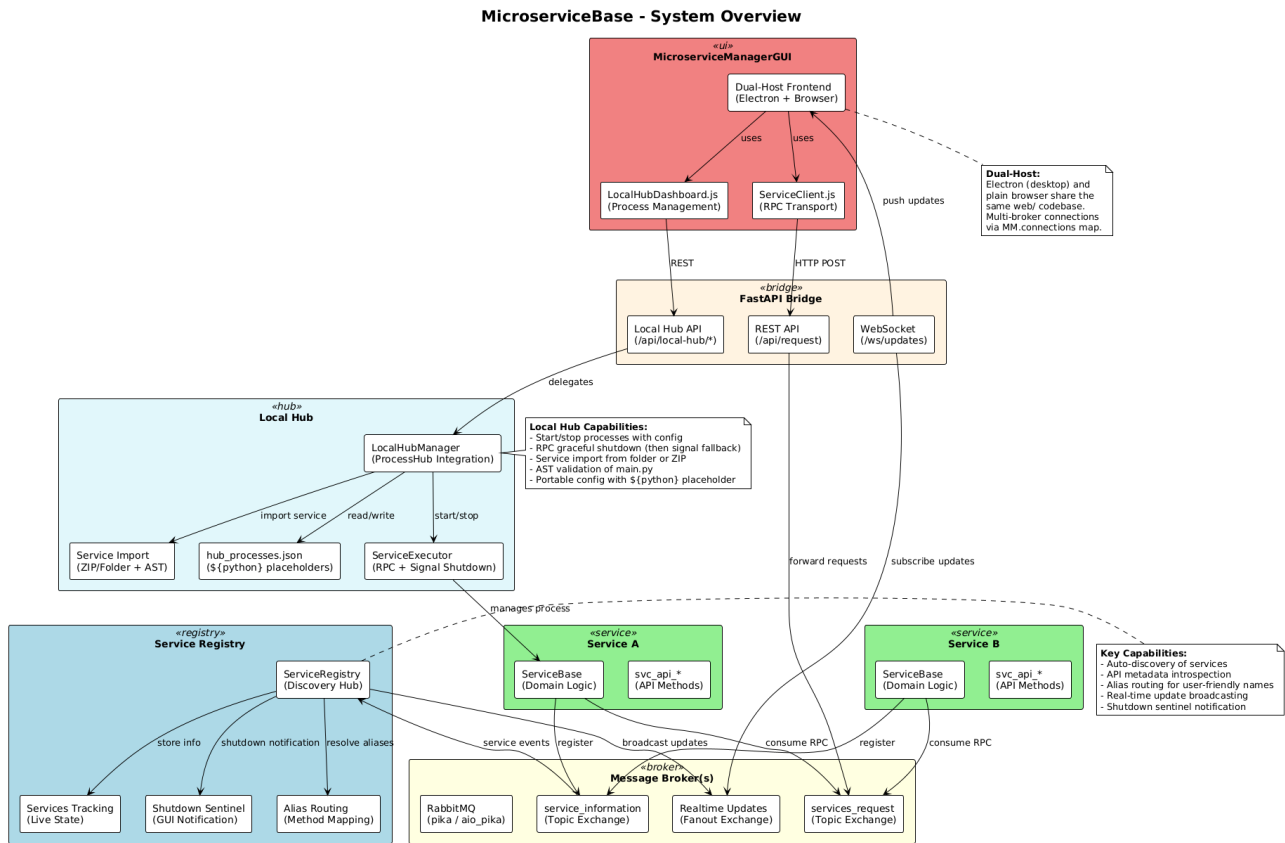


Figure 2.1: MicroserviceBase System Overview

#### Key Features

- **Microservice Framework:** Base classes for creating services with RPC and pub/sub patterns
- **Service Registry:** Dynamic service registration and discovery via RabbitMQ exchange
- **FastAPI Bridge:** REST API and WebSocket gateway connecting GUI to microservices
- **Manager GUI:** Electron + browser-based dashboard for monitoring and controlling services

- **Local Process Hub:** Start, stop, and manage service processes with graceful shutdown
- **Multi-Broker Support:** Connect to multiple RabbitMQ brokers simultaneously
- **Service Import:** Import microservice packages (folder or zip) into the local hub
- **Service Creator:** Interactive wizard for scaffolding new microservices
- **Standalone Installer:** Package as Windows desktop application (DevAtServGUI)

## Design Philosophy

MicroserviceBase follows several key design principles:

1. **Hexagonal Architecture:** Domain logic is isolated from external concerns via ports and adapters
2. **Factory Pattern:** All components are created through factories, enabling easy swapping
3. **Dual Hosting:** GUI works in both Electron (desktop) and browser modes
4. **Graceful Shutdown:** Proper process cleanup on Windows using CTRL\_BREAK\_EVENT signals
5. **Config Persistence:** Placeholder-based configuration that works across environments

### 2.1.2 Architecture

MicroserviceBase is organized into distinct layers following hexagonal architecture:

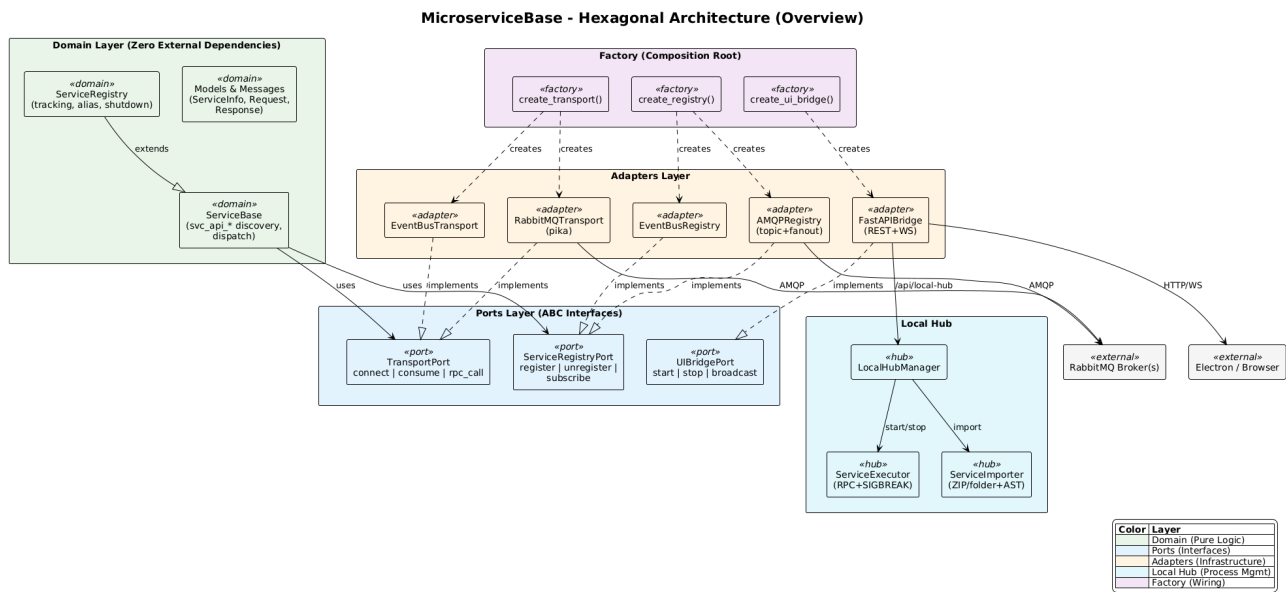


Figure 2.2: Hexagonal Architecture Overview

The detailed architecture with method signatures and implementation specifics:

#### Layer Responsibilities

**Domain Layer** Core business logic: `ServiceBase` (base class for microservices), `ServiceRegistry` (service registration and discovery), and `Factory` (component creation).

**Ports Layer** Abstract interfaces defining contracts: `TransportPort` (message passing), `RegistryPort` (service registration), `UIBridgePort` (GUI communication).

**Adapters Layer** Concrete implementations:

- **RabbitMQ Transport** – RPC calls and message consumption via pika
- **RabbitMQ Registry** – Service registration via exchange broadcasting

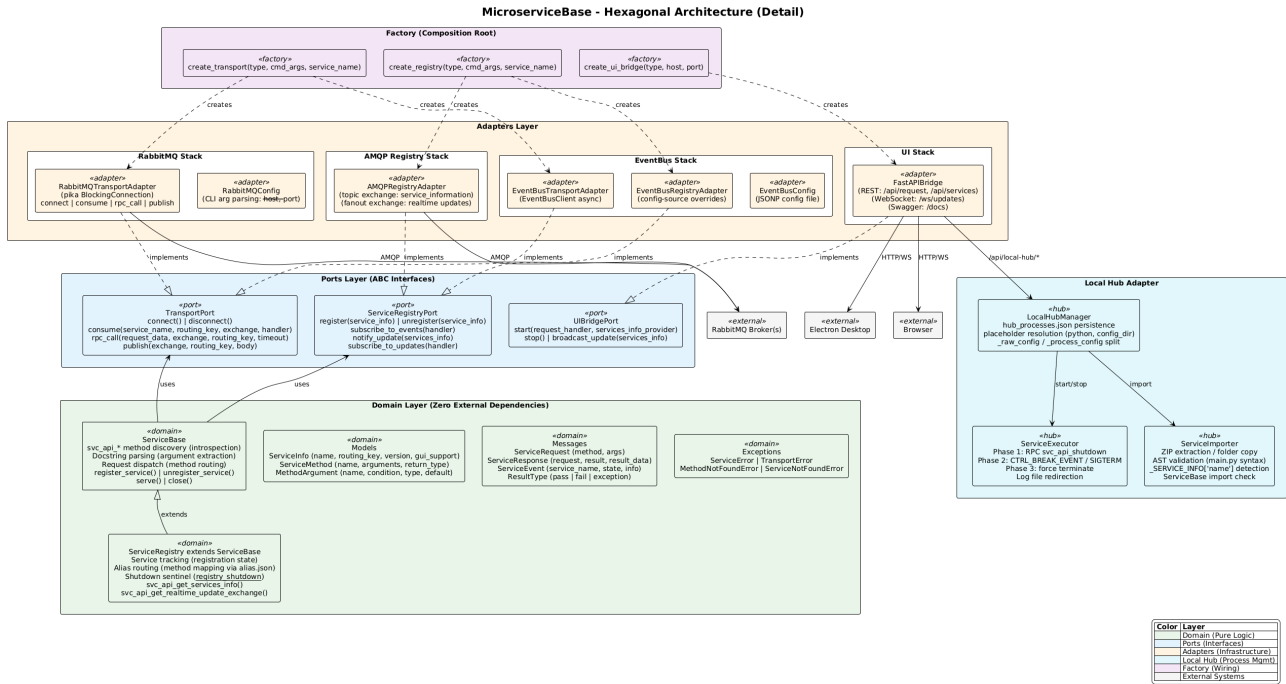


Figure 2.3: Hexagonal Architecture Detail

- FastAPI Bridge – REST API, WebSocket, and Local Hub endpoints
- Local Hub Manager – Process lifecycle management
- Service Executor – Process execution with platform-specific shutdown

GUI Layer MicroserviceManagerGUI with dual hosting:

- Electron – Desktop wrapper with contextBridge preload
- Web – Pure browser-compatible HTML/CSS/JS
- Service Plugins – Dynamically loaded per-service UI components

### 2.1.3 Components

#### ServiceBase

The base class for all microservices provides:

- API method discovery via `svc_api_` prefix convention
- Automatic service registration with the registry
- Graceful shutdown with `unregister_service()` on exit
- Service info exposure for discovery via `_SERVICE_INFO` dict
- Docstring-based API documentation generation

```
from MicroserviceBase.domain.service_base import ServiceBase
from MicroserviceBase.factory import create_transport, create_registry
```

```
class MyService(ServiceBase):
    _SERVICE_INFO = {
        'name': 'MyService',
        'routing_key': 'service.myservice',
        'version': '1.0.0',
        'gui_support': False,
        'methods': [], 'methods_info': {},
    }
```

```

def svc_api_my_method(self, arg1, arg2):
    """My API method."""
    return some_result

transport = create_transport('rabbitmq',
    cmd_args=['--host', 'localhost', '--port', '5672'],
    service_name='MyService')
registry = create_registry('rabbitmq',
    cmd_args=['--host', 'localhost', '--port', '5672'],
    service_name='MyService')

service = MyService(transport=transport, registry=registry)
service.register_service()
service.serve()

```

### Service Properties (`_SERVICE_INFO`)

Every microservice must define a `_SERVICE_INFO` class-level dictionary that describes the service to the framework. The registry, the GUI, and the service creator all rely on these properties. The following table lists every recognized key:

| Property     | Type | Required | Default | Description                                                                                    |
|--------------|------|----------|---------|------------------------------------------------------------------------------------------------|
| name         | str  | yes      | —       | Unique service name (PascalCase recommended)                                                   |
| description  | str  | no       | ''      | Full description, displayed in the Code Helper modal                                           |
| shortdesc    | str  | no       | ''      | Short description shown in the GUI sidebar                                                     |
| group        | str  | no       | ''      | Category group for sidebar organization (e.g. examples)                                        |
| tag          | str  | no       | ''      | Optional version tag                                                                           |
| version      | str  | no       | '1.0.0' | Semantic version string                                                                        |
| routing_key  | str  | no       | ''      | RabbitMQ routing key for RPC requests                                                          |
| gui.support  | bool | no       | False   | Whether the service ships a GUI plugin (GUIs/ folder)                                          |
| downloadable | bool | no       | False   | Whether the service exposes <code>svc_api.get_service_files</code> for remote download         |
| methods      | list | auto     | []      | <i>Auto-populated</i> by <code>ServiceBase.__init__</code> from <code>svc_api.*</code> methods |
| methods_info | dict | auto     | {}      | <i>Auto-populated</i> from method docstrings                                                   |

Custom properties (e.g. `sample_path`) may be added freely—the framework ignores unknown keys but they are broadcast with the service info, so the GUI or other clients can read them.

### Example

```

_SERVICE_INFO = {
    'name': 'ServiceCleware',
    'description': 'Service to control Cleware switch box devices.',
    'shortdesc': 'Cleware Controllers',
    'group': 'Switch Boxes',
    'tag': '',
    'version': '1.0.0',
    'routing_key': 'ServiceClewareKey',

```

```

'gui_support': True,
'downloadable': False,
'methods': [],          # auto-populated at __init__
'methods_info': {},     # auto-populated from docstrings
}

```

## Generated Service Template

The **Service Creator** wizard (or the `POST /api/scaffold/generate` REST endpoint) produces a ready-to-run service package with the following file structure:

```

<ServiceName>/
|-- <service_name>.py      # Service class with _SERVICE_INFO and svc_api_* methods
|-- main.py               # Entry point: creates transport, registry, registers, serves
|-- __main__.py           # Enables: python -m <ServiceName>
|-- config.jsonp          # (EventBus only) transport configuration
|-- GUIs/                 # (if gui_support is enabled)
|   |-- service.html      # Bootstrap card template (or custom HTML)
|   |-- service.js        # IIFE with load/unload functions

```

**Service class** (`<service_name>.py`) contains the `_SERVICE_INFO` dict with all metadata, and one `svc_api_*` method stub per API method defined in the wizard. Each method stub includes a full docstring with `**Arguments:**` and `**Returns:**` sections following the project convention.

**Entry point** (`main.py`) configures logging, creates the transport and registry via the factory, instantiates the service, calls `register_service()`, and enters `serve()`. Graceful shutdown is handled in a `try/except/finally` block that calls `unregister_service()` and `close()`.

**Package entry point** (`__main__.py`) allows the service to be started with `python -m <ServiceName>` from its parent directory, which is how the Local Hub launches imported services.

**GUI plugin** (`GUIs/`) is only generated when the **Generate GUI template** toggle is enabled in step 3 of the wizard. The HTML file provides a Bootstrap card layout and the JS file exposes `load<ServiceName>()` / `unload<ServiceName>()` functions using the `window.MicroserviceManager` namespace.

## Service Registry

The registry manages dynamic service discovery:

- Services register on startup with their method list and metadata
- Registry broadcasts updates to all subscribers via RabbitMQ exchange
- Shutdown notification via `__registry_shutdown__` sentinel
- GUI receives real-time updates via WebSocket

## FastAPI Bridge

The bridge connects the GUI to microservices:

**POST** `/api/request` RPC call forwarding to services

**GET** `/api/services` Registered service list

**WS** `/ws/updates` Real-time service update stream

`/api/local-hub/*` Local process hub management endpoints

**GET** `/docs` Auto-generated Swagger API documentation



## Local Hub Manager

The Local Hub provides process lifecycle management:

- Start, stop, and monitor local service processes
- Process configuration via `hub_processes.json` with placeholder support
- Service import from folders or zip archives
- Service executor with graceful shutdown using `CTRL_BREAK_EVENT` on Windows
- Log file management per process

### 2.1.4 Communication Patterns

#### RPC via RabbitMQ

Services communicate using RPC (Remote Procedure Call) pattern through RabbitMQ. The following diagram shows the complete communication flow between a service, the Service Registry, and a client, including all exchanges and routing keys used:

```
from MicroserviceBase.domain.service_base import ServiceBase

# Client sends request
request_data = ServiceBase.create_request_data('svc_api_add', [1, 2])
response = transport.rpc_call(
    request_data,
    exchange_name='services_request',
    routing_key='service.calculator'
)
# response = {"request": "svc_api_add", "result": "pass",
#            "result_data": 3}
```

#### Service Registration

Services register themselves via the registry exchange. The `ServiceBase` class handles this automatically using the `_SERVICE_INFO` dict:

```
# ServiceBase auto-discovers svc_api_* methods and builds
# the service info dict, then publishes it:
service.register_service()

# On shutdown, unregister to notify the registry:
service.unregister_service()
```

#### Multi-Broker Architecture

The GUI supports connecting to multiple RabbitMQ brokers simultaneously:

- `MM.connections` map tracks per-broker state
- `serviceToBroker` reverse lookup for request routing
- Session persistence via `sessionStorage`
- Progressive disclosure: broker sections shown only when multiple brokers connected

### 2.1.5 GUI Architecture

#### Dual Hosting

The MicroserviceManagerGUI runs in two modes:

**Electron Mode** Desktop application with native process management. Uses contextBridge preload for secure Node.js access. Can spawn and manage the FastAPI bridge process directly.

**Browser Mode** Access via `http://localhost:1112` when the bridge is running. Pure HTML/CSS/JS with no Node.js dependencies. Uses the same `window.MicroserviceManager` namespace.

#### Service Plugins

Each microservice can provide a custom GUI plugin:

- HTML file for UI layout
- JS file for logic (uses `const MM = window.MicroserviceManager;`)
- Plugins loaded dynamically into `web/services/`
- In packaged mode, plugins extracted to writable `%APPDATA%` directory

#### Standalone Installer

The GUI can be packaged as a standalone Windows application using Electron Builder:

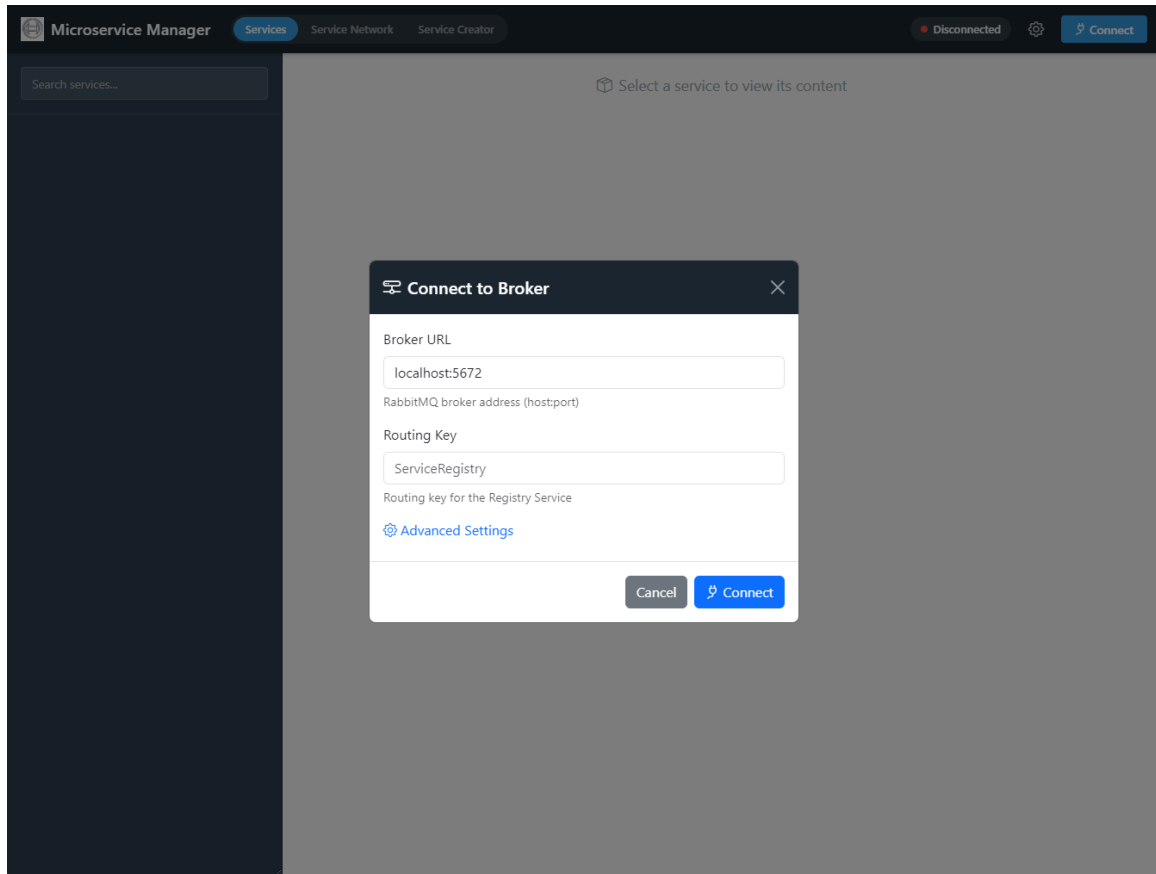
- NSIS installer for Windows (`build.bat`), Linux packages (`build.sh`)
- Read-only scripts in `resources/python/`
- Writable user data in `%APPDATA%/devatservgui/`
- First-run initialization copies templates and scripts
- Bridge process managed via PID file with detached mode

### 2.1.6 GUI User Guide

The Microservice Manager GUI provides three main tabs: **Services** for connecting to brokers and interacting with microservices, **Service Network** for managing the local process hub, and **Service Creator** for scaffolding new microservices.

#### Connecting to a Broker

To start using the GUI, click the **Connect** button in the top-right corner. The connection dialog requires two fields:



*Connect to Broker Dialog*

**Broker URL** The RabbitMQ broker address in `host:port` format (e.g., `localhost:5672`). The FastAPI bridge will connect to this broker to discover and communicate with services.

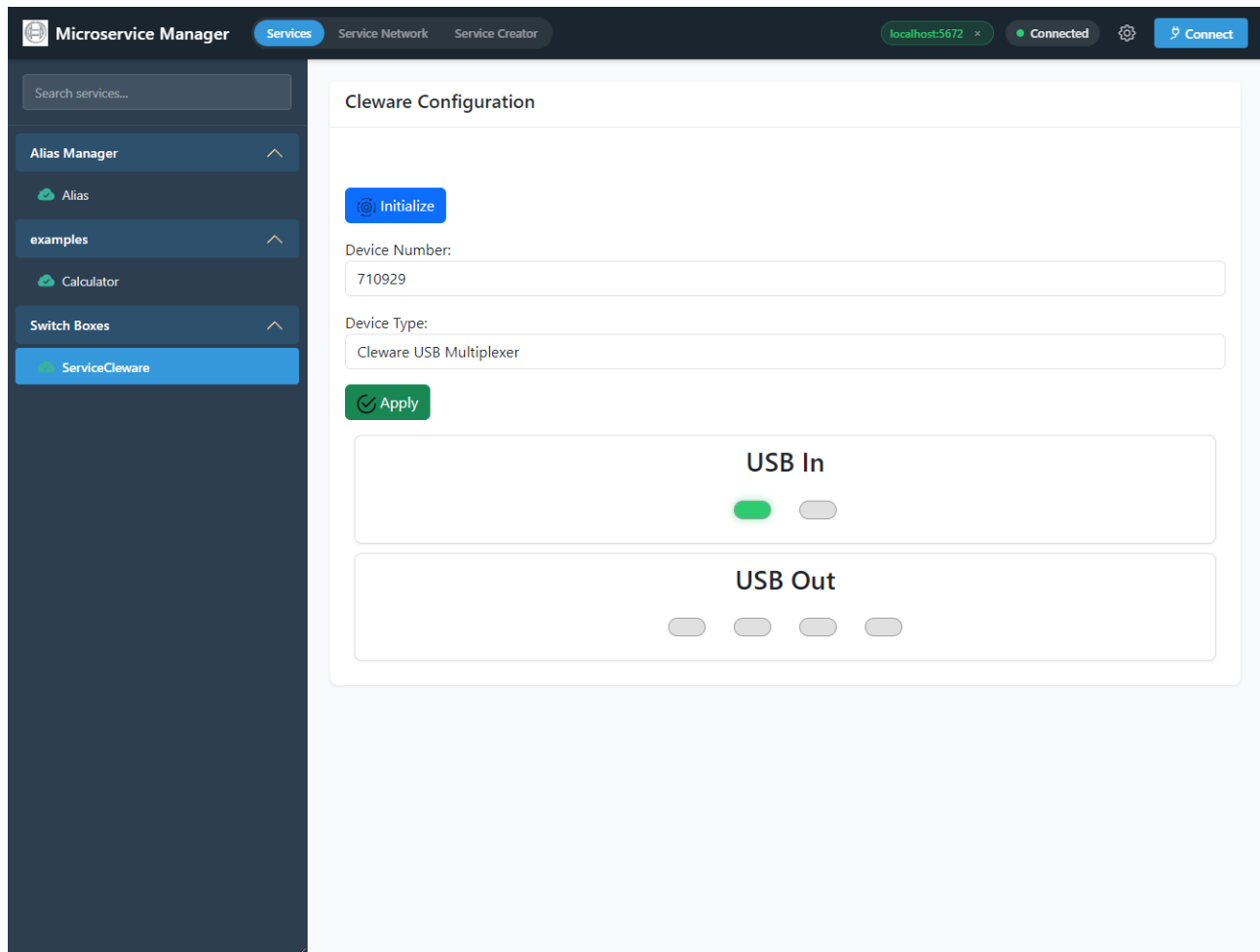
**Routing Key** The routing key of the Service Registry (default: `ServiceRegistry`). This tells the bridge where to find the registry service that tracks all registered microservices.

**Advanced Settings** Click the **Advanced Settings** link to reveal additional connection options such as exchange names and other broker-specific parameters.

After connecting, the GUI subscribes to real-time service updates via WebSocket. The connected broker address appears as a badge in the top-right corner (e.g. `localhost:5672 x`). Multiple brokers can be connected simultaneously—each broker appears as a separate section in the sidebar when more than one is active.

## Service Dashboard

Once connected, the **Services** tab displays all registered microservices in the left sidebar, grouped by their service group. Clicking a service loads its details and custom GUI plugin (if available) in the main content area.

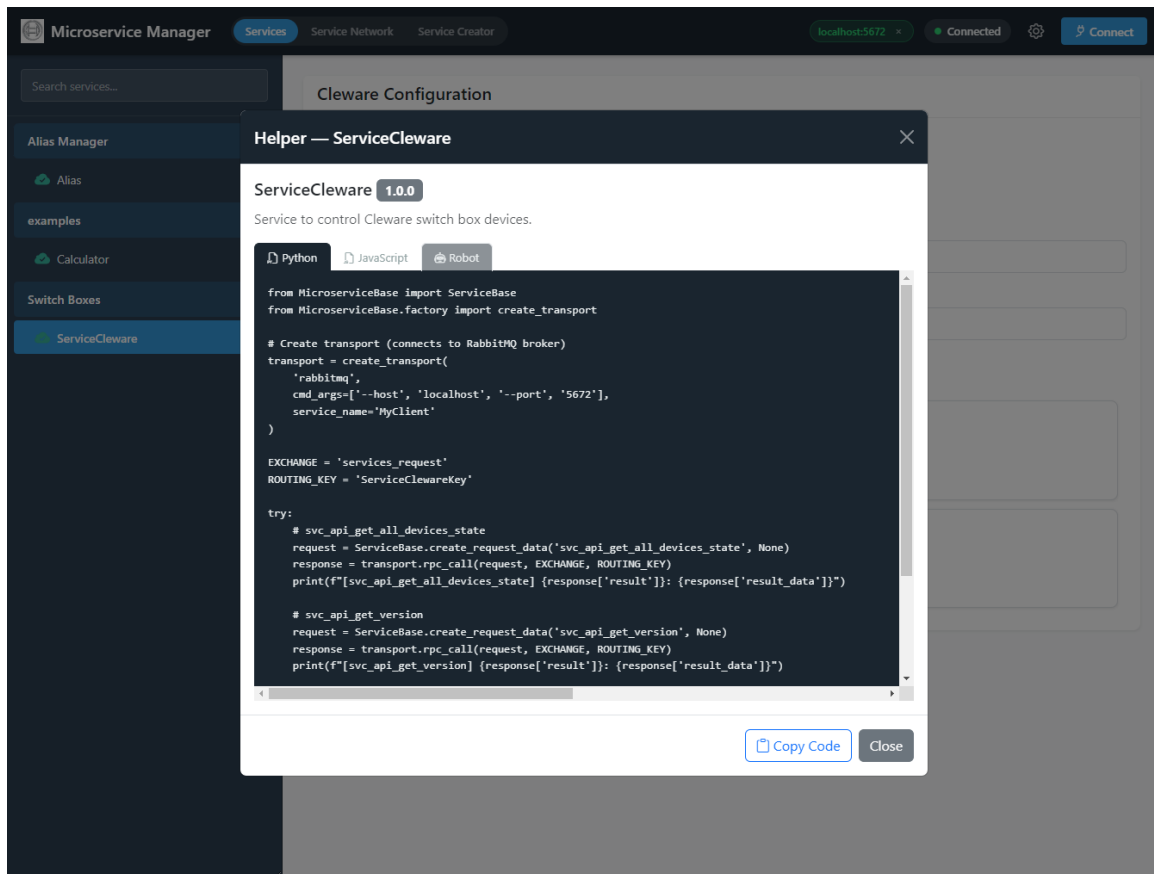


*Service Dashboard with Sidebar and Service Plugin*

The sidebar shows each service as an accordion item with the service name, version, and an expand arrow. Expanding a service item reveals its API methods as clickable list items. Services that provide a custom GUI plugin (indicated by `gui_support: True` in their `_SERVICE_INFO`) display their plugin interface in the main panel when selected.

## Code Helper

For each service, the GUI provides auto-generated code examples that demonstrate how to call the service's API methods via RPC. Click the **Helper** button (code icon) on any service to open the code example modal.



*Code Helper Modal with Python, JavaScript, and Robot Tabs*

The modal header shows the service name, version badge, and description. Three language tabs are available:

**Python** Complete boilerplate for creating a transport connection, building a request with `ServiceBase.create_request_data()` and calling the service via `rpc_call()` with the correct exchange name and routing key. Copy directly into a Python script.

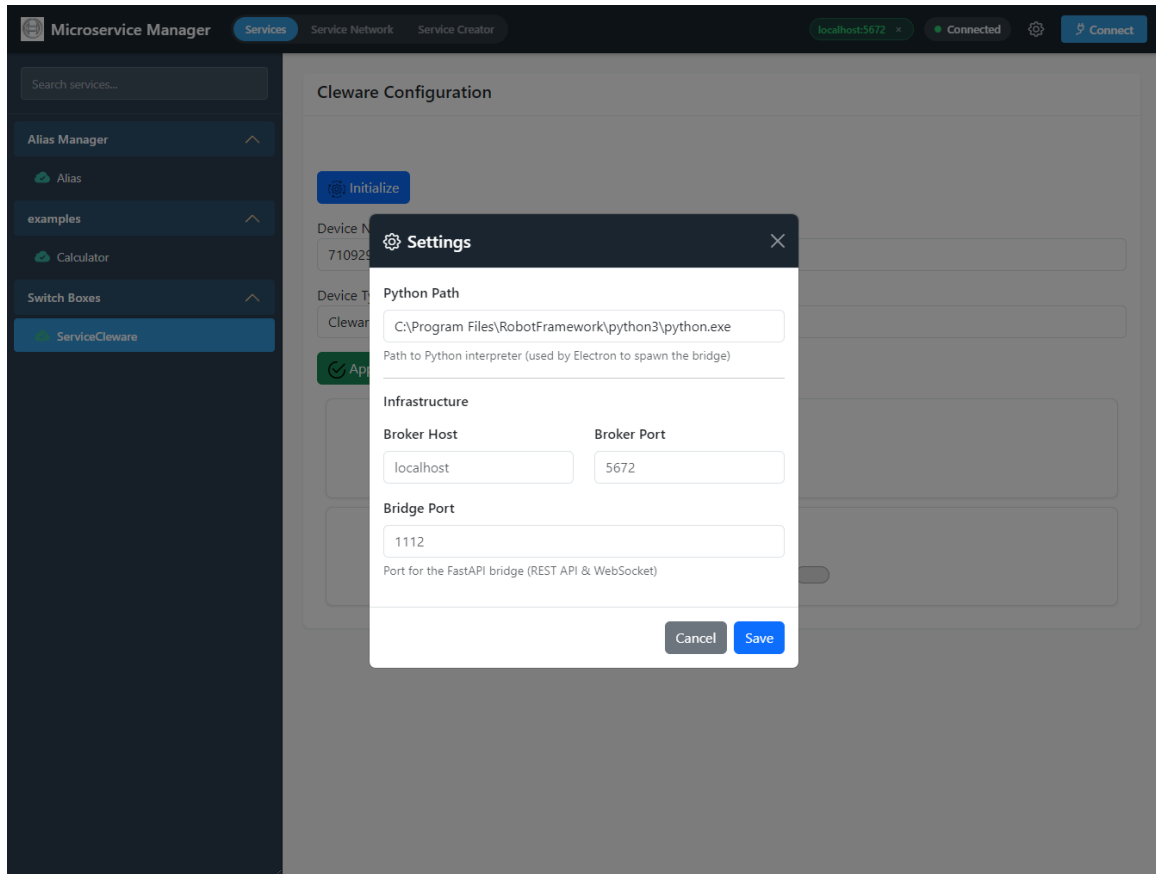
**JavaScript** Browser-side code using the `window.MicroserviceManager` namespace and the FastAPI bridge REST endpoint to call service methods from a web application.

**Robot** Robot Framework keyword examples for calling the service's API methods, ready to integrate into test automation workflows.

Click **Copy Code** to copy the currently visible tab content to the clipboard.

## Settings

Click the gear icon in the top-right corner to open the **Settings** dialog.



*Application Settings*

**Python Path** The Python executable used by the Local Hub to spawn service processes. Defaults to the system Python.

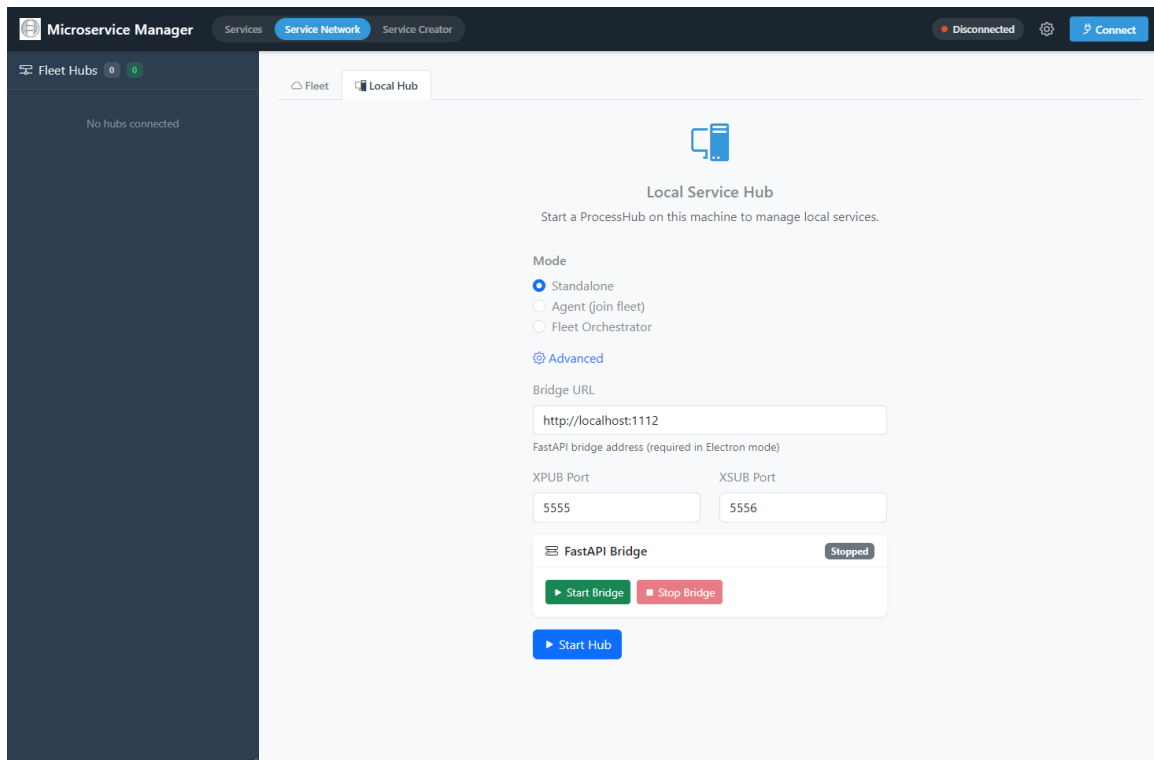
**Broker Host / Port** Default RabbitMQ broker connection parameters, pre-filled in the Connect dialog.

**Bridge Port** The port on which the FastAPI bridge listens (default: 1112). In browser mode, access the GUI at `http://localhost:<bridge-port>`.

Settings are persisted to `settings.json` and loaded on startup. In Electron mode, the file is stored alongside the application; in packaged mode, it is stored in `%APPDATA%/DevAtServGUI/`.

## Local Service Hub

The **Service Network** tab provides access to the Local Hub, which manages service processes on the local machine. Switch to the **Local Hub** sub-tab to configure and start the hub.



*Local Hub Configuration and Bridge Status*

The Local Hub configuration page provides the following options:

**Mode** Three radio buttons select the hub operating mode:

- **Standalone** — runs an independent hub on this machine (default).
- **Agent (join fleet)** — connects to an existing fleet orchestrator as a managed node.
- **Fleet Orchestrator** — starts this hub as a fleet manager that other hubs can join for centralized orchestration.

**Advanced** Click the **Advanced** link to reveal additional options.

**Bridge URL** The address of the FastAPI bridge (e.g., `http://localhost:1112`). In Electron mode, the bridge can be started directly from this page.

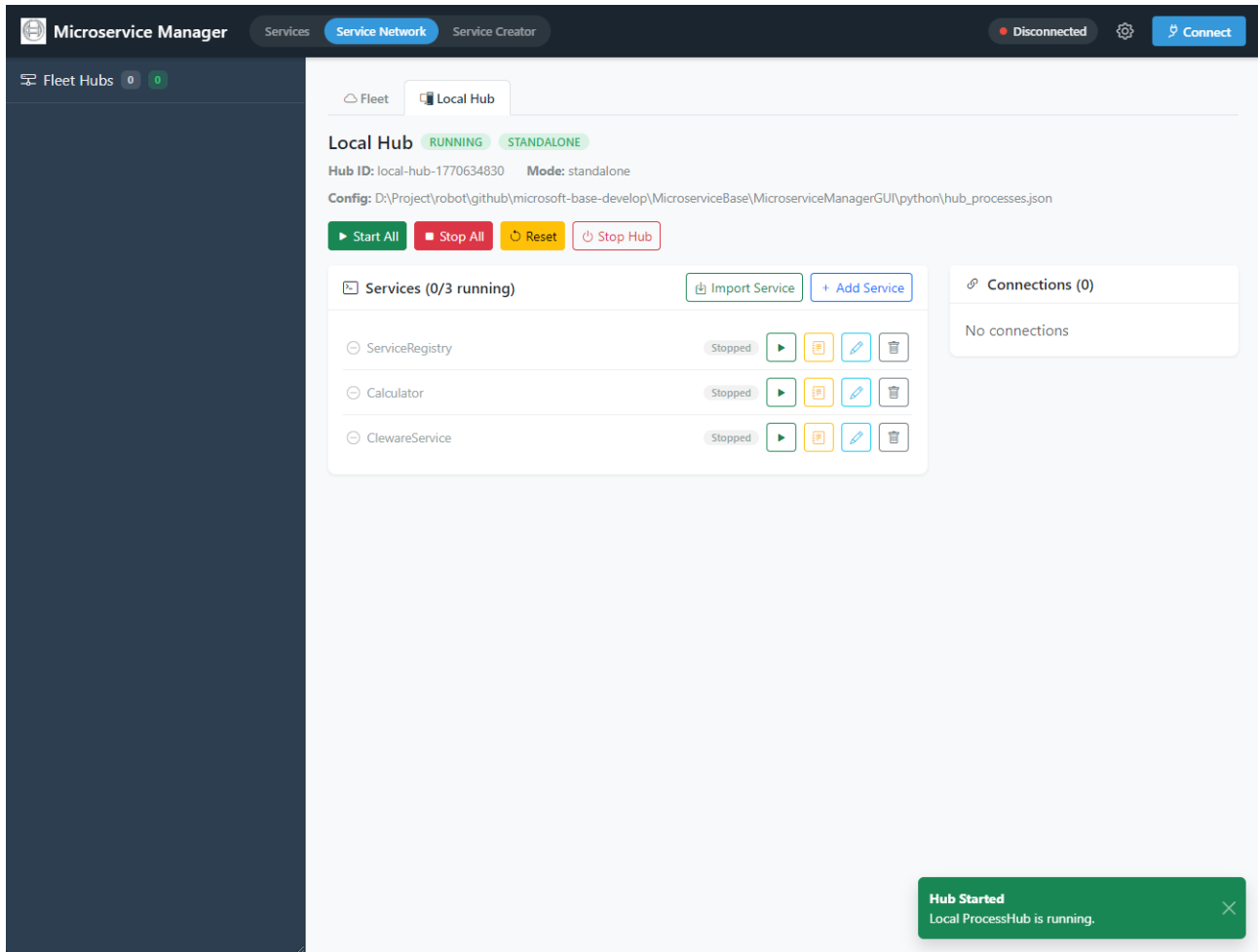
**XPUB / XSUB Port** Ports used by the EventBus proxy (defaults: 5555 / 5556). Only relevant when using EventBus transport.

**FastAPI Bridge** A status card shows whether the bridge is *Running* or *Stopped*. Use the **Start Bridge** / **Stop Bridge** buttons to control the bridge process directly (Electron mode only).

**Start Hub** Click to start the Local Hub. The hub reads `hub-processes.json` and makes all configured services available for management.

## Local Hub Dashboard

After starting the hub, the dashboard displays hub metadata, all configured services with their current status, and control buttons.



*Local Hub Dashboard with Service Process Management*

The dashboard header shows **Hub ID**, **Mode** (standalone / agent / orchestrator), and the path to the loaded **Config** file. A toast notification confirms when the hub has started successfully. The dashboard provides:

**Hub Controls** **Start All** and **Stop All** buttons to batch-control all services. **Reset** reloads the configuration. **Stop Hub** shuts down the hub and all running processes.

**Service List** A panel labelled **Services (N/M running)** shows each configured service with its name, status (*Stopped* / *Running*), and action buttons: start (▶), save config, edit configuration, and delete.

**Import Service** Click to import a microservice package from a folder or ZIP archive. The importer validates the package structure (checks for `main.py` with a `ServiceBase` subclass) and adds it to the hub configuration.

**Add Service** Manually add a new service entry to the hub configuration by specifying the script path, arguments, and process name.

**Connections** A side panel shows active broker connections with the connection count badge.

Services are started as subprocess with log file redirection. The hub uses a two-phase graceful shutdown: first an `RPC svc_api_shutdown` call, then `CTRL_BREAK_EVENT` (Windows) or `SIGTERM` (Linux), and finally a force terminate if needed.

## Service Creator Wizard

The **Service Creator** tab provides a step-by-step wizard for scaffolding new microservice projects. The wizard generates a complete, ready-to-run service package with all required files.

### Step 1: Basic Information



**Microservice Manager** Services Service Network **Service Creator** Disconnected Connect

**SERVICE CREATOR**

- 1 Basic Info
- 2 API Methods
- 3 GUI Support
- 4 Review & Generate

### ① Basic Information

Define the core metadata for your new microservice.

**Service Name \***  **Version**

PascalCase name for your service class.

**Description**

**Short Description**

**Group**  **Tag**

**Routing Key**

Auto-generated from service name. You can customize it.

**Transport Type**

**Next** →

*Service Creator – Step 1: Basic Information*

Define the core metadata for the new microservice:

**Service Name** PascalCase name for the service class (e.g., `Test`).

**Version** Semantic version string (default: `1.0.0`).

**Description / Short Description** Documentation strings included in the generated `_SERVICE_INFO` dict.

**Group** Service group for sidebar organization in the GUI (e.g., `examples`).

**Tag** Optional version tag for the service.

**Routing Key** Auto-generated from the service name in `service.<name>` format. Can be customized.

**Transport Type** Choose between `RabbitMQ` or `EventBus` transport adapter.

## Step 2: API Methods

**Microservice Manager** Services Service Network **Service Creator** Disconnected Connect

**SERVICE CREATOR**

- Basic Info
- 2 API Methods**
- 3 GUI Support
- 4 Review & Generate

### ② API Methods

Define the methods your service will expose. Each method name will be prefixed with `svc_api_`.

`svc_api_` test ✕

Return Type:  Description:

Parameters:

✕

Service Creator – Step 2: API Methods

Define the methods your service will expose. Each method name is automatically prefixed with `svc_api_` following the framework convention. For each method:

- **Return Type:** The expected return type (e.g., `int`, `str`, `dict`).
- **Description:** Docstring for the method, used in API documentation generation.
- **Parameters:** Add parameters with name, type, and condition (required or optional). Click **+ Add Parameter** to add more. Click **+ Add Method** to define additional API methods.

### Step 3: GUI Support

**Microservice Manager** Services Service Network **Service Creator** Disconnected Connect

**SERVICE CREATOR**

- Basic Info
- API Methods
- 3 GUI Support**
- 4 Review & Generate

### ③ GUI Support

Choose whether to include a GUI template for your service.

☒ Generate GUI template

**HTML Editor**

```
<div class="card">
  <div class="card-header">
    <h5>Test</h5>
    <small class="text-muted">Test service</small>
  </div>
  <div class="card-body">
    <p>Custom GUI for Test.</p>
    <!-- Add your service GUI here -->
  </div>
</div>
```

**Live Preview**

**Test**

Test service

Custom GUI for Test.

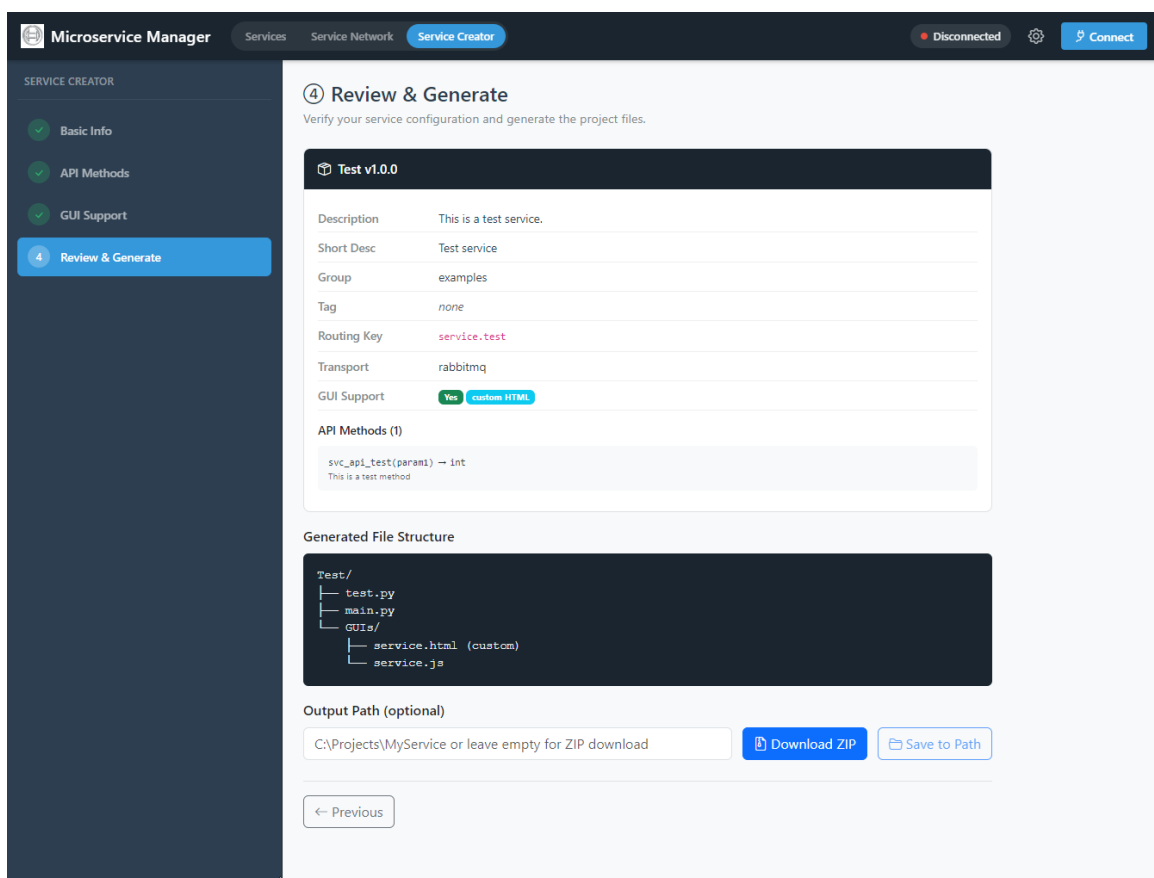
**JavaScript File (optional)**

*Service Creator – Step 3: GUI Support*

Choose whether to include a custom GUI plugin for the service. When the **Generate GUI template** toggle is enabled:

- An **HTML Editor** provides a Bootstrap card template with the service name and description. Edit the HTML directly or click **Upload HTML** to use a custom file.
- A **Live Preview** panel shows the rendered HTML in real-time.
- An optional **JavaScript File** can be uploaded via drag-and-drop for service-specific logic (using the `window.MicroserviceManager` namespace).
- Click **Reset to Template** to restore the default HTML template.

The generated GUI plugin will be placed in a `GUIs/` folder inside the service package and loaded automatically when the service is selected in the Manager GUI.

**Step 4: Review and Generate***Service Creator – Step 4: Review and Generate*

Review the complete service configuration before generating:

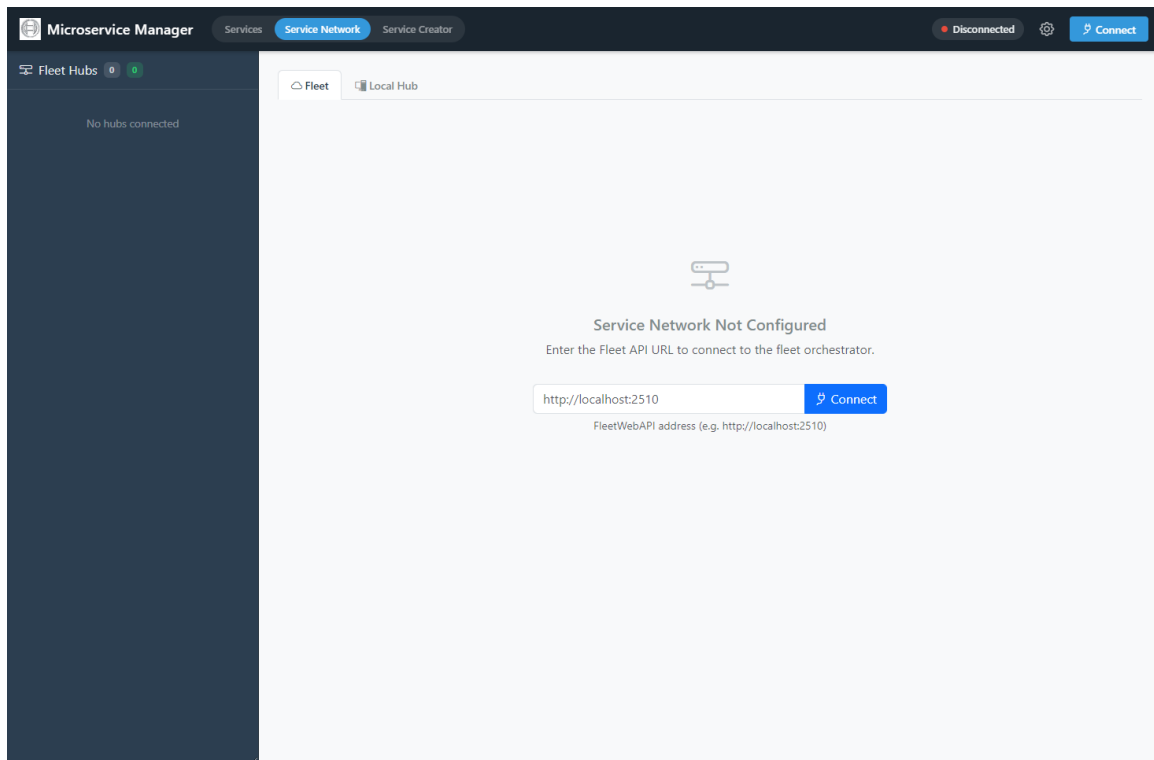
- A summary card displays all metadata, routing key, transport type, GUI support status, and API method signatures with parameter types.
- The **Generated File Structure** preview shows the directory layout that will be created (e.g., `Test/test.py`, `Test/main.py`, `Test/GUIs/service.html`).
- Specify an **Output Path** to save directly to disk, or leave empty to download as a ZIP file.
- Click **Download ZIP** to get the package as a zip archive, or **Save to Path** to write files to the specified directory.

The generated service package is ready to run. It can be imported into the Local Hub using the **Import Service** feature, or started manually via `python -m <ServiceName>` from the package directory.

## Fleet Orchestrator

The **Fleet** sub-tab in the **Service Network** page provides centralized orchestration of multiple hubs across different machines.

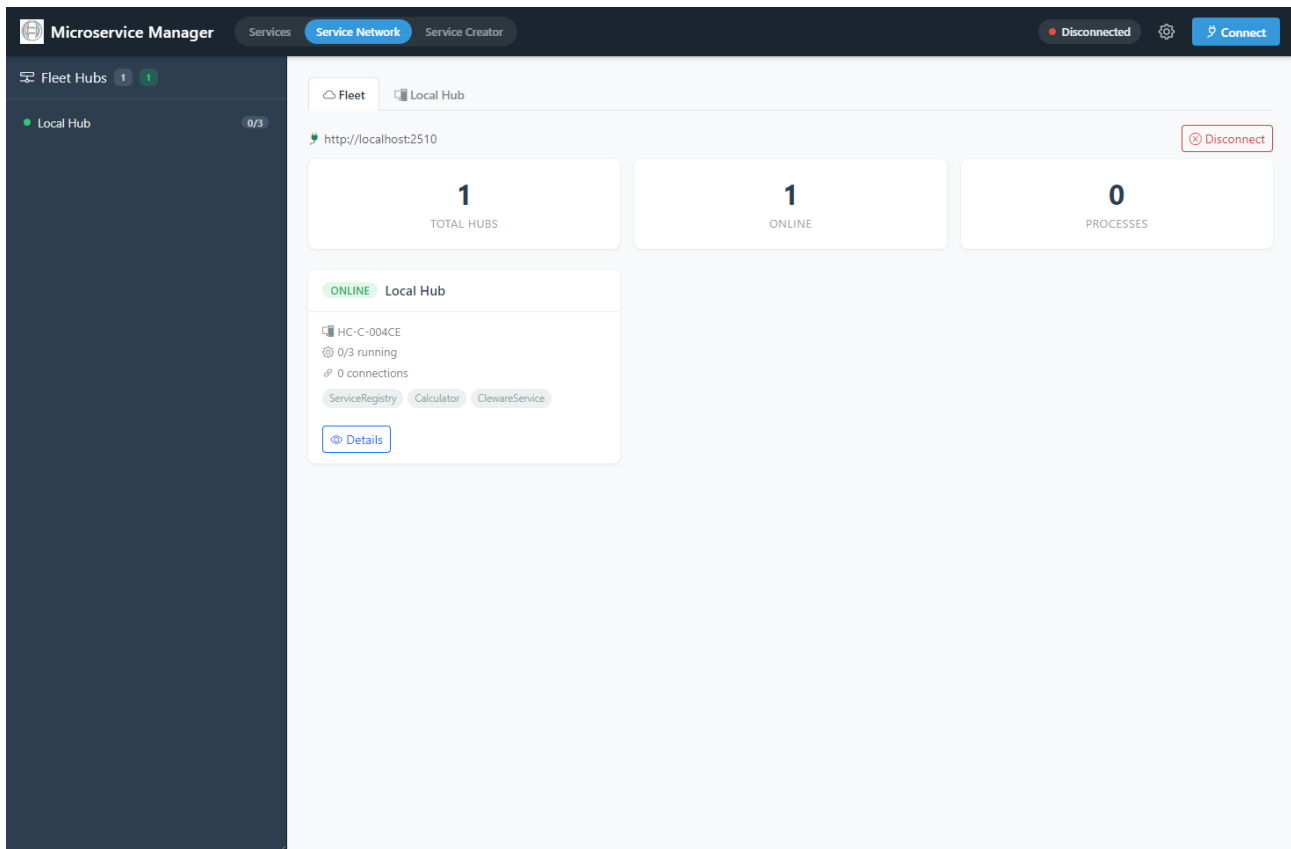
### Connecting to a Fleet



*Fleet Connection – Enter the Fleet API URL*

When no fleet connection is configured, the page displays a **Service Network Not Configured** prompt. Enter the Fleet API URL (e.g., `http://localhost:2510`) and click **Connect** to connect to an existing fleet orchestrator. The Fleet API is a WebAPI exposed by a hub running in *Fleet Orchestrator* mode.

### Fleet Dashboard



*Fleet Dashboard – Overview of All Connected Hubs*

Once connected, the Fleet Dashboard shows:

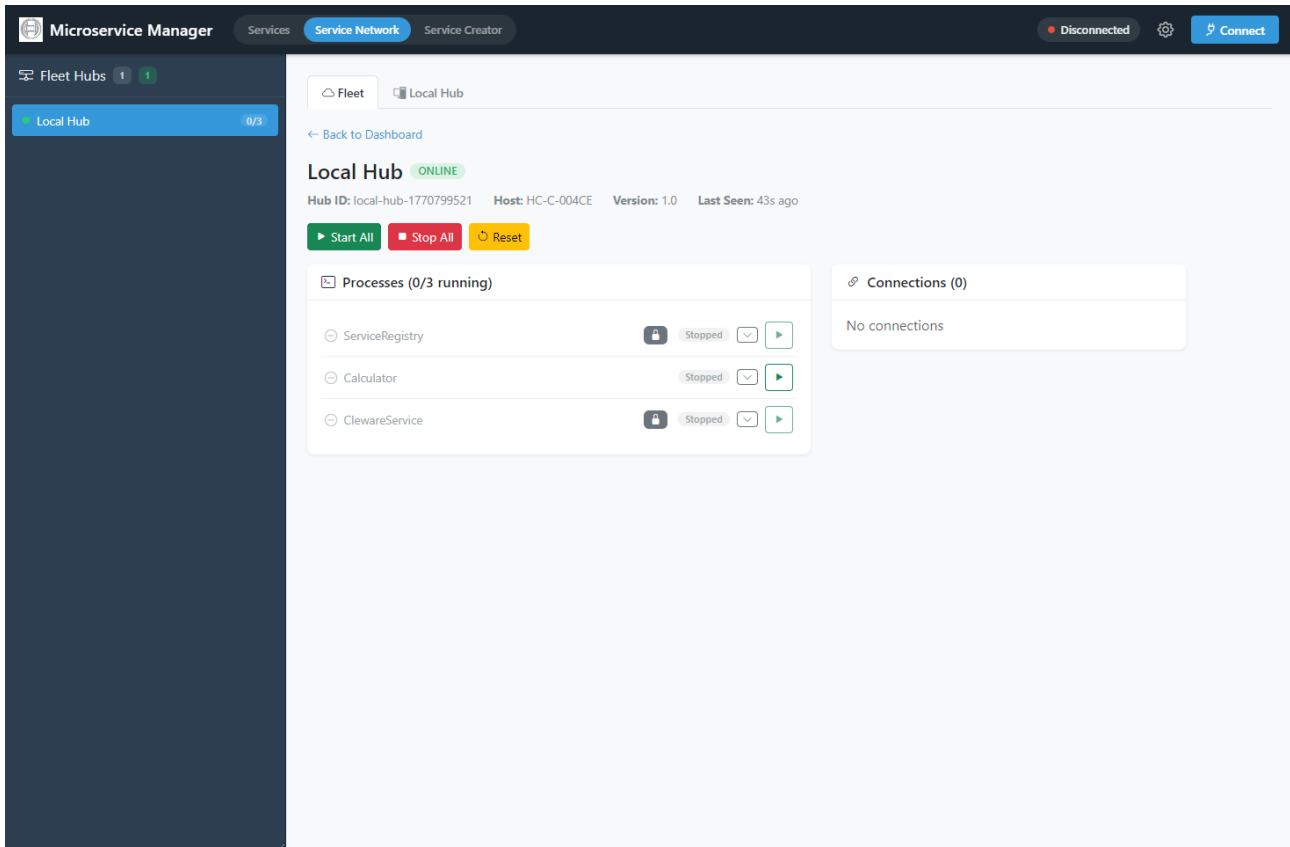
**Summary Cards** Three metric cards at the top display **Total Hubs**, **Online** hubs, and total **Processes** count across all hubs.

**Hub Cards** Each connected hub appears as a card showing its name, hostname, running process count, connection count, and the names of configured services as badges. Click **Details** to drill into a specific hub.

**Sidebar** The left sidebar lists all fleet hubs with their online status indicator and process count badge.

**Disconnect** Click **Disconnect** in the top-right corner to leave the fleet.

## Fleet Hub Details



*Fleet Hub Details – Remote Process Management*

Clicking **Details** on a hub card opens its detail view, which mirrors the Local Hub Dashboard but operates on the remote hub:

- **Hub metadata:** Hub ID, hostname, version, and last-seen timestamp.
- **Start All / Stop All / Reset:** Batch-control all processes on the remote hub.
- **Processes:** Lists each process with status, a lock icon for fleet-protected processes (`fleet_enabled: false`), an expand button, and a start/stop button.
- **Connections:** Shows broker connections active on the remote hub.
- Click **Back to Dashboard** to return to the fleet overview.

### 2.1.7 Process Configuration

Processes are configured via `hub_processes.json`:

```
{
  "ServiceRegistry": {
    "script": "${config_dir}/start_registry.py",
    "args": ["--config", "${config_dir}/config.json"],
    "process_name": "ServiceRegistry",
    "wait_time": 2.0
  },
  "Calculator": {
    "script": "${python}",
    "args": ["path/to/calculator.py"],
    "process_name": "Calculator",
    "wait_time": 1.0,
    "fleet_enabled": true
  },
  "ClewareService": {
    "script": "${python}",
```

```

    "args": ["-m", "ClewareService"],
    "cwd": "${config_dir}/services",
    "process_name": "ClewareService",
    "wait_time": 1.0,
    "service_name": "ServiceCleware"
  }
}

```

## Process Properties

Each entry in `hub_processes.json` supports the following properties:

**script** Path to the executable or Python script. Supports `${python}` and `${config_dir}` placeholders.

**args** List of command-line arguments passed to the script.

**process\_name** Display name used in the dashboard and logs.

**wait\_time** Seconds to wait after spawning before checking if the process is still alive (default: 2.0).

**cwd** (*optional*) Working directory for the process. Required when using `-m` module execution (e.g., `python -m ClewareService`).

**env** (*optional*) Dictionary of extra environment variables to set for the subprocess.

**service\_name** (*optional*) The RabbitMQ queue name the service consumes on. Used by the executor to send `svc_api_shutdown` RPC. Falls back to the entry key if omitted.

**fleet\_enabled** (*optional, default false*) When `true`, the process can be started and stopped remotely via Fleet orchestrator commands. Processes without this flag are protected from remote control (shown with a lock icon in the Fleet Hub detail view).

## Placeholders

**`${python}`** Resolves to `sys.executable` at runtime

**`${config_dir}`** Resolves to the directory containing `hub_processes.json`

Placeholders are preserved in `_raw_config` during save operations, allowing the same configuration file to work across different environments.

## 2.1.8 Windows Process Lifecycle

On Windows, process shutdown requires special handling:

- `proc.terminate()` calls `TerminateProcess` which allows no cleanup
- Service Executor uses `CTRL_BREAK_EVENT` for graceful shutdown
- `signal.SIGBREAK` handler converts to `KeyboardInterrupt` for Python cleanup
- Pika's `start_consuming()` replaced with `process_data_events(time_limit=1)` loop for interruptible consumption
- `subprocess.PIPE` blocking prevented by using `FileHandler` instead of `StreamHandler` for logging in subprocess mode

## 2.1.9 Installation

### From Source

```

git clone https://github.com/test-fullautomation/python-microservice-base.git
cd python-microservice-base
pip install .

```

```

# Development mode
pip install -e .

```

## GUI Setup

```
cd MicroserviceBase/MicroserviceManagerGUI
npm install
npm start
```

### 2.1.10 Quick Start

#### Start Service Registry

```
python MicroserviceBase/MicroserviceManagerGUI/python/start_registry.py \
    --host localhost --port 5672
```

#### Start FastAPI Bridge

```
python MicroserviceBase/MicroserviceManagerGUI/python/start_bridge.py \
    --host localhost --port 5672 --bridge-port 1112
```

#### Create a Service

```
import sys
from MicroserviceBase.domain.service_base import ServiceBase
from MicroserviceBase.factory import create_transport, create_registry

class CalculatorService(ServiceBase):
    _SERVICE_INFO = {
        'name': 'Calculator',
        'description': 'A simple calculator service.',
        'version': '1.0.0',
        'routing_key': 'service.calculator',
        'gui_support': False,
        'methods': [], 'methods_info': {},
    }

    def svc_api_add(self, a, b):
        """Add two numbers."""
        return int(a) + int(b)

transport = create_transport('rabbitmq',
    cmd_args=sys.argv[1:], service_name='Calculator')
registry = create_registry('rabbitmq',
    cmd_args=sys.argv[1:], service_name='Calculator')

service = CalculatorService(transport=transport, registry=registry)
service.register_service()
service.serve()
```

### 2.1.11 Diagrams Reference

The following PlantUML diagrams are available in the docs/diagrams/ folder:

**overview.puml** System overview showing services, broker, registry, bridge, and GUI

**architecture.puml** Hexagonal architecture with domain, ports, and adapters

**component.puml** Component diagram with package dependencies

**class\_domain.puml** Class diagram for domain layer

**class\_ports.puml** Class diagram for port interfaces

**class\_adapters.puml** Class diagram for adapter implementations



**gui\_architecture.puml** GUI dual-host architecture (Electron + browser)

**sequence\_rpc.puml** RPC call sequence from GUI to service

**sequence\_registration.puml** Service registration flow

**sequence\_alias.puml** Service alias resolution sequence

**sequence\_shutdown.puml** Two-phase graceful shutdown sequence

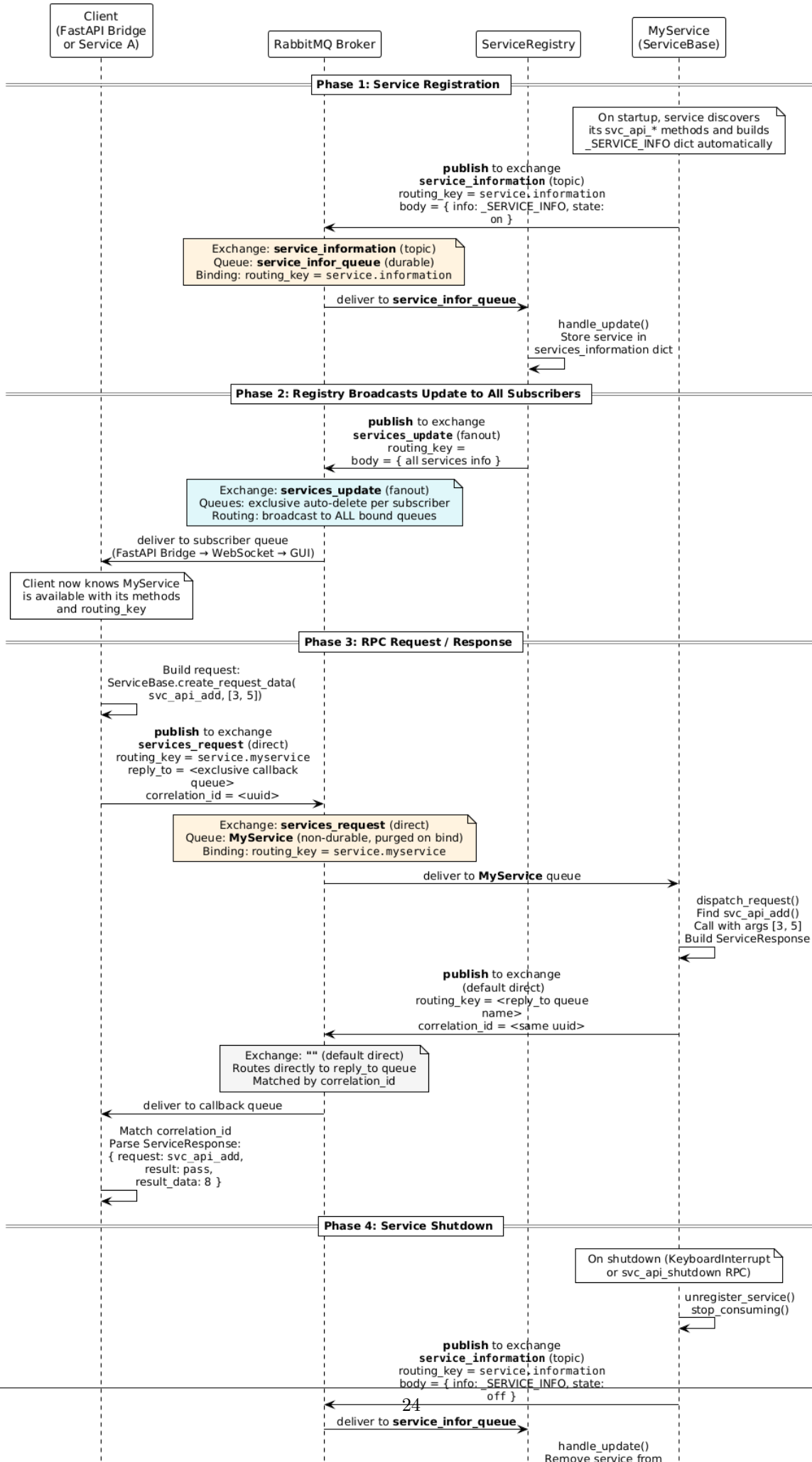
**sequence\_service\_import.puml** Service import flow (GUI to hub)

**state\_process\_lifecycle.puml** Process state machine

**component\_local\_hub.puml** Local Hub internal architecture (LocalHubManager, ServiceExecutor, ProcessHub, config management)

**component\_fleet.puml** Fleet Orchestrator multi-hub architecture (Orchestrator, Agents, FleetWebAPI, EventBus, permission guard)

## Service Communication Flow (Exchanges, Queues, and Routing Keys)



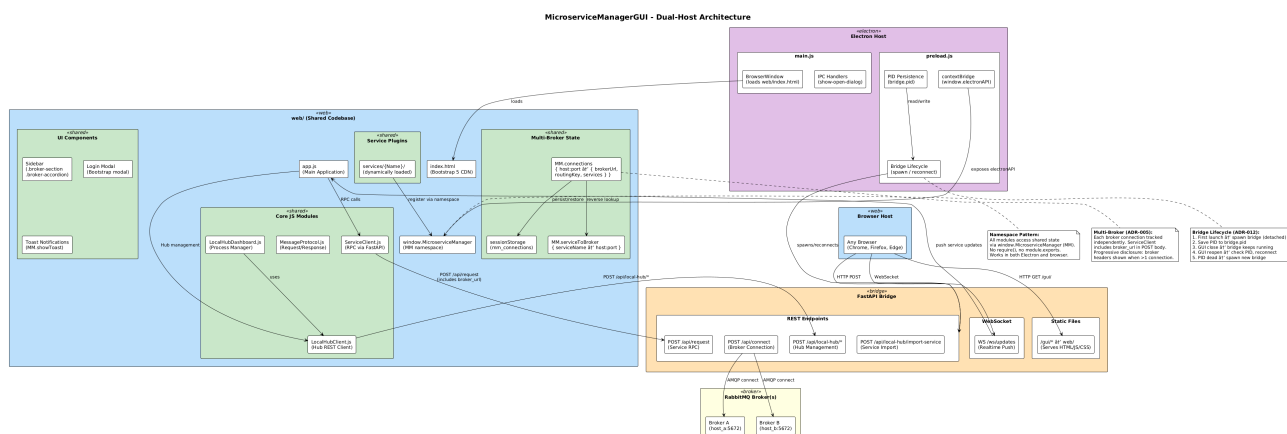


Figure 2.5: GUI Dual-Host Architecture (Electron + Browser)

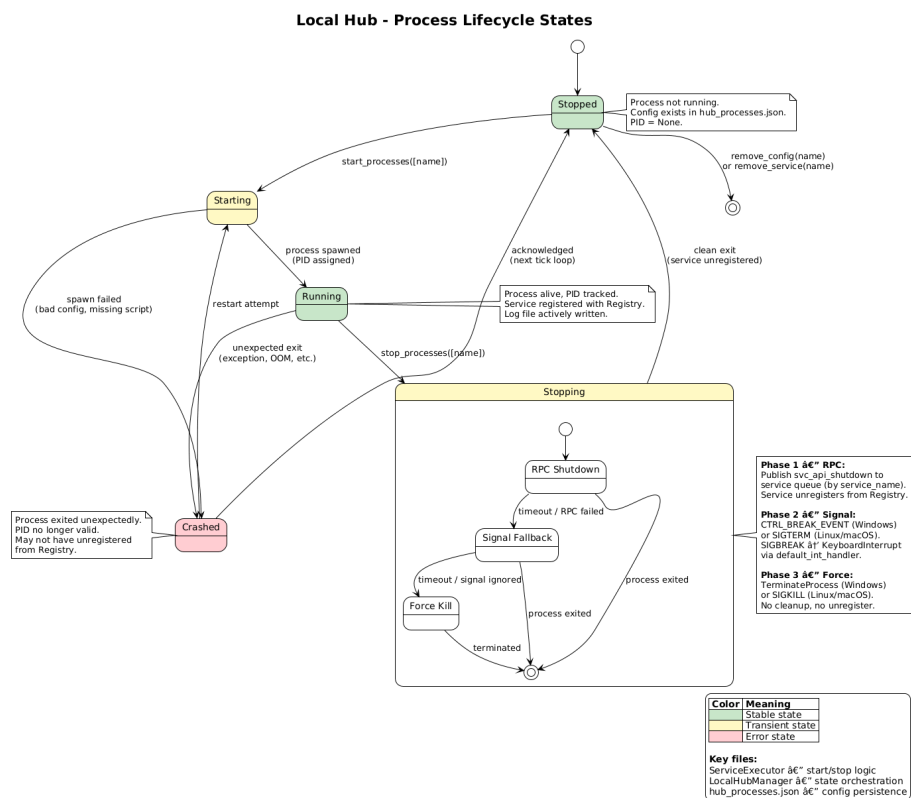


Figure 2.6: Local Hub Process Lifecycle States

# Chapter 3

## 01\_basic\_service.py

Basic Microservice Example.

This example demonstrates how to: 1. Define a service by extending ServiceBase 2. Expose API methods with the svc\_api prefix 3. Write docstrings that generate API documentation automatically 4. Create a transport and registry via the factory 5. Register the service and start serving requests

The service will: - Connect to RabbitMQ - Register itself with the Service Registry - Start consuming RPC requests on its routing key - Respond to method calls with structured ServiceResponse messages

### 3.1 Function: main

Run the Calculator service.

### 3.2 Class: CalculatorService

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.01_basic_ser
↳ import CalculatorService
```

A simple calculator service.

Demonstrates the svc\_api convention and docstring-based API discovery.

#### 3.2.1 Method: svc\_api\_add

Add two numbers.

**Arguments:**

- a  
/ Condition: required / Type: int /  
First number.
- b  
/ Condition: required / Type: int /  
Second number.

**Returns:**

/ Type: int /  
Sum of a and b.

### 3.2.2 Method: svc\_api\_subtract

Subtract b from a.

**Arguments:**

- a  
/ *Condition*: required / *Type*: int /  
First number.
- b  
/ *Condition*: required / *Type*: int /  
Number to subtract.

**Returns:**

/ *Type*: int /  
Difference of a and b.

### 3.2.3 Method: svc\_api\_multiply

Multiply two numbers.

**Arguments:**

- a  
/ *Condition*: required / *Type*: int /  
First number.
- b  
/ *Condition*: required / *Type*: int /  
Second number.

**Returns:**

/ *Type*: int /  
Product of a and b.

## Chapter 4

# 02\_service\_client.py

Service Client Example.

This example demonstrates how to: 1. Create a transport adapter for client-side RPC calls 2. Build a ServiceRequest with method name and arguments 3. Send an RPC call and receive a ServiceResponse 4. Parse the response to extract result data

### Prerequisites:

- RabbitMQ server running
- Calculator service (01\_basic\_service.py) running

## 4.1 Function: main

Run the service client example.

## Chapter 5

# 03\_service\_registry.py

Service Registry Example.

This example demonstrates how to: 1. Create and run the ServiceRegistry (central discovery hub) 2. Configure the realtime update exchange for UI clients 3. Handle service registration and unregistration events 4. Broadcast service list updates to all connected UI clients

The Service Registry: - Listens on the 'service\_information' topic exchange for register/unregister events - Maintains a dict of all currently registered services - Broadcasts the full service list when changes occur (via fanout exchange) - Exposes its own API methods for querying service information - Supports alias routing via alias.json configuration

### 5.1 Function: main

Run the Service Registry.

## Chapter 6

# 04\_eventbus\_transport.py

EventBus Transport Example.

This example demonstrates how to: 1. Use EventBus transport instead of direct RabbitMQ (pika) 2. Configure transport via a JSONP config file 3. Swap transports without changing the service code 4. Use the factory to create EventBus-based adapters

### Prerequisites:

- RabbitMQ server running
- EventBusClient package installed

The EventBus transport provides: - JSONP-based configuration (no CLI argument parsing) - Async message handling via aio\_pika under the hood - Topic and Fanout exchange handlers - Shared configuration for derivative clients (registry, updates)

**Example JSONP config file (config.jsonp):** { "host": "localhost", "port": 5672, "username": "guest", "password": "guest", "exchange\_name": "services\_request", "exchange\_handler": "TopicExchangeHandler" }

## 6.1 Function: create\_config\_file

Create a temporary JSONP config file for the EventBus transport.

In production, this file would be a persistent configuration file.

## 6.2 Function: main

Run the Greeter service with EventBus transport.

## 6.3 Class: GreeterService

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.04_eventbus_
↳ import GreeterService
```

A greeting service using EventBus transport.

Same service code as any other service -- only the transport wiring differs.



### 6.3.1 Method: svc\_api\_hello

Greet someone by name.

**Arguments:**

- name  
/ *Condition*: required / *Type*: str /  
Name of the person to greet.

**Returns:**

/ *Type*: str /  
Greeting message.

### 6.3.2 Method: svc\_api\_goodbye

Say goodbye to someone.

**Arguments:**

- name  
/ *Condition*: required / *Type*: str /  
Name of the person.

**Returns:**

/ *Type*: str /  
Farewell message.

## Chapter 7

# 05\_alias\_routing.py

Alias Routing Example.

This example demonstrates how to: 1. Configure method aliases in the Service Registry 2. Call services using user-friendly alias names 3. Use argument templates with `${input}` substitution 4. Route requests transparently through the Registry

Alias routing allows: - User-friendly names for complex service methods - Pre-configured arguments (only `${input}` needs to be provided) - Transparent forwarding (caller doesn't need to know the target service) - Centralized routing configuration

**Example alias configuration:**

```
{ "double": { "Service name": "Calculator", "Method name": "svc_api_multiply",
            "Arguments": "${input},2"
        }, "add_ten": { "Service name": "Calculator", "Method name": "svc_api_add", "Arguments":
            "${input},10" }
    }
```

**Prerequisites:**

- Calculator service (01\_basic\_service.py) running
- Service Registry (03\_service\_registry.py) running

## 7.1 Function: main

Run the alias routing example.

## Chapter 8

# 06\_fastapi\_bridge.py

FastAPI UI Bridge Example.

This example demonstrates how to: 1. Create a FastAPI bridge for browser-based access to microservices 2. Expose a REST endpoint for invoking service methods 3. Expose a REST endpoint for listing registered services 4. Push realtime service updates via WebSocket 5. Access auto-generated Swagger documentation

The FastAPI bridge provides: - REST API for any HTTP client (curl, browser, Python requests) - WebSocket for realtime push notifications - Swagger UI at /docs for interactive API exploration - Decoupled UI layer (any frontend can connect)

**Example HTTP requests:** # List all services curl <http://localhost:1112/api/services>

```
# Call a service method curl -X POST http://localhost:1112/api/request -H "Content-Type: application/json" -d '{"method": "svc_api_add", "args": [3, 5], "exchange": "services_request", "routing_key": "service.calculator"}'
```

### 8.1 Function: main

Run the FastAPI bridge example.

## Chapter 9

# 07\_mock\_fleet\_api.py

Mock FleetWebAPI Server.

Simulates a fleet orchestrator with multiple hubs for GUI development and demonstration purposes. Each hub has a set of processes that can be started and stopped via the API, and hub health status changes over time.

**Endpoints (matching the real FleetWebAPI contract):** GET /api/fleet/status - Full fleet snapshot GET /api/fleet/hubs - Hub list GET /api/fleet/hubs/{hub\_id} - Single hub detail POST /api/fleet/hubs/{hub\_id}/start - Start processes POST /api/fleet/hubs/{hub\_id}/stop - Stop processes POST /api/fleet/hubs/{hub\_id}/reset - Reset hub POST /api/fleet/command - Raw fleet command GET / - Simple status page

### 9.1 Function: dashboard

### 9.2 Function: fleet\_status

### 9.3 Function: fleet\_hubs

### 9.4 Function: fleet\_hub\_detail

### 9.5 Function: fleet\_hub\_start

### 9.6 Function: fleet\_hub\_stop

### 9.7 Function: fleet\_hub\_reset

### 9.8 Function: fleet\_command

### 9.9 Function: main

### 9.10 Class: ProcessActionRequest

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.07_mock_flee
↳ import ProcessActionRequest
```

## 9.11 Class: CommandRequest

*Imported by:*

```
from  
↳ MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.07.mock_flee  
↳ import CommandRequest
```

## Chapter 10

# 08\_fleet\_demo.py

Fleet Demo — All-in-one launcher.

Starts both the mock FleetWebAPI and a GUI server so you can demo the Service Network tab in a single command with zero infrastructure.

### 10.1 Function: `start_mock_fleet`

Start the mock FleetWebAPI in a background thread. `microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-08-fleet-demo-start-gui-server` =====

Start a minimal FastAPI server that serves the GUI + fleet proxy. `microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-08-fleet-demo-main` =====

## Chapter 11

# run\_hub\_agent.py

Hub Agent with Calculator and Cleware processes.

**Starts a ProcessHub server that manages two microservice processes:**

- Calculator: 01\_basic\_service.py (arithmetic service)
- ServiceCleware: python -m MicroserviceClewareSwitch (USB switch service)

A HubAgent sidecar connects to the FleetOrchestrator so these processes can be monitored and controlled from the Service Network tab.

### 11.1 Function: build\_process\_config

Build the process configuration for Calculator and Cleware.

Each entry tells ProcessHub how to start the process. The subprocess inherits os.environ but NOT sys.path modifications, so we must pass PYTHONPATH explicitly via the 'env' dict. microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-fleet-demo-run-hub-agent-main =====

## Chapter 12

# run\_orchestrator.py

Fleet Orchestrator.

Starts the FleetOrchestrator that receives hub agent heartbeats and routes commands, plus the FleetWebAPI on port 2510 for the GUI's Service Network tab.

Open <http://localhost:2510> for the built-in fleet dashboard, or connect from the MicroserviceManager GUI.

### 12.1 Function: main



# Chapter 13

## serve\_dev.py

serve\_dev.py — Dev server for Qt WASM service template.

Serves the WASM build output AND provides an HTTP→RabbitMQ bridge so the Qt WASM GUI can call the running MyQtWasmService.exe backend.

Uses MicroserviceBase.adapters.transport for RabbitMQ communication — the same transport layer used by the Python microservices.

**Usage:** python serve\_dev.py [port] [build\_dir] python serve\_dev.py 8090 build/wasm

**Prerequisites:**

- MicroserviceBase package installed (pip install MicroserviceBase)
- RabbitMQ running on localhost:5672
- MyQtWasmService.exe running

### 13.1 Function: load\_broker\_config

Load broker config from service\_config.json, fall back to defaults. microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-qt-wasm-service-template-serve-dev-rpc-call =====

Send an RPC request via MicroserviceBase transport and return the response. microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-qt-wasm-service-template-serve-dev-find-build-dir =====

Auto-detect WASM build output directory. microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-qt-wasm-service-template-serve-dev-main =====

### 13.2 Class: DevHandler

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.qt-wasm_serv
↳ import DevHandler
```

HTTP handler: static files + RabbitMQ bridge. microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-qt-wasm-service-template-serve-dev-devhandler-do-get -----

#### 13.2.1 Method: do\_POST

#### 13.2.2 Method: guess\_type

#### 13.2.3 Method: log\_message

Only log API calls and errors, not every static file.

## Chapter 14

### `__main__.py`

Entry point for `python -m <service_folder>` execution.

# Chapter 15

## main.py

Service Template.

Copy this folder and customize it to create your own microservice.

Steps: 1. Rename the folder to match your service name. 2. Update `_SERVICE_INFO` with your service metadata. 3. Add your own `svc_api` methods. 4. (Optional) Set `'gui_support': True` and add GUI files under `GUIs/`. 5. (Optional) Set `'downloadable': True` to let users download this service as a ZIP from the Service Manager GUI.

### 15.1 Function: main

Run the service. `microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-service-template-main-myservice` =====

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.service_template
↳ import MyService
```

A template service — replace with your own description. `microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-service-template-main-myservice-svc-api-hello` -----

Say hello.

#### Arguments:

- `name`  
/ *Condition:* required / *Type:* str /  
The name to greet.

#### Returns:

/ *Type:* str /  
A greeting message.

#### 15.1.1 Method: `svc_api_echo`

Echo a message back.

#### Arguments:

- `message`  
/ *Condition:* required / *Type:* str /  
The message to echo.

**Returns:**

*/ Type: str /*

The same message.

## Chapter 16

# launcher.py

FastAPI Bridge launcher.

Starts the FastAPI Bridge (REST/WS gateway) that connects the MicroserviceManagerGUI to microservices via RabbitMQ.

### 16.1 Function: `parse_args`

### 16.2 Function: `main`

## Chapter 17

# ClewareAccessHelper.py

### 17.1 Class: ClewareAccessHelper

*Imported by:*

```
from  
↳ MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.ClewareAccessHelper  
↳ import ClewareAccessHelper
```

ClewareAccessHelper class acts as a proxy class in Proxy pattern and forward the request to the real helper depend on platform. microservicebase-microservicemanagergui-dist-win-unpacked-resources-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-import-modules-from-paths -----

Import all modules from given paths.

**Arguments:**

- paths  
/ Condition: required / Type: list /  
List of paths to import modules from.

#### 17.1.1 Method: load\_config

Load the user config port name for USB access. Returns: None microservicebase-microservicemanagergui-dist-win-unpacked-resources-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-get-config -----

Get Cleware ports' config. Returns: Dictionary of ports' setting. microservicebase-microservicemanagergui-dist-win-unpacked-resources-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-set-config -----

Save Cleware ports' config to file. Args: config\_json: data to be saved.

**Returns:** res: saved result.

- 17.1.2 Method: `init_cleware`
- 17.1.3 Method: `open_cleware`
- 17.1.4 Method: `close_cleware`
- 17.1.5 Method: `get_handle`
- 17.1.6 Method: `set_value`
- 17.1.7 Method: `get_value`
- 17.1.8 Method: `set_switch`
- 17.1.9 Method: `set_switch_by_port_name`
- 17.1.10 Method: `get_switch`
- 17.1.11 Method: `get_version`
- 17.1.12 Method: `get_usb_type`
- 17.1.13 Method: `get_serial_number`
- 17.1.14 Method: `valid_ser_number`
- 17.1.15 Method: `get_hw_version`
- 17.1.16 Method: `is_ampel`
- 17.1.17 Method: `iox`
- 17.1.18 Method: `get_all_devices_state`

## Chapter 18

# ClewareAccessHelperAbs.py

### 18.1 Class: ClewareAccessHelperAbs

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.ClewareAccessHelperAbs
↳ import ClewareAccessHelperAbs
```

ClewareAccessHelperAbs acts as a abstract class for the real Helper in Proxy Pattern. microservicebase-microservicemanagergui-dist-win-unpacked-resources-python-services-clewareservice-clewareaccesshelperabs-clewareaccesshelperabs  
init-cleware -----  
-----

18.1.1 Method: open\_cleware

18.1.2 Method: close\_cleware

18.1.3 Method: set\_value

18.1.4 Method: get\_value

18.1.5 Method: set\_switch

18.1.6 Method: get\_switch

18.1.7 Method: get\_handle

18.1.8 Method: get\_version

18.1.9 Method: get\_usb\_type

18.1.10 Method: get\_usb\_type

18.1.11 Method: get\_serial\_number

18.1.12 Method: valid\_ser\_number

18.1.13 Method: get\_hw\_version

18.1.14 Method: is\_ampel

18.1.15 Method: iox



## Chapter 19

# ClewareAccessHelperLinux.py

### 19.1 Class: ClewareAccessHelperLinux

*Imported by:*

```
from  
↳ MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.ClewareAccessHelperLinux  
↳ import ClewareAccessHelperLinux
```

**ClewareAccessHelperLinux acts as the real object on Linux platform in Proxy Pattern**

- This is the wrapper class for the functions provided by C/C++ source for Linux.
- The C/C++ source for Linux is provided by Cleware manufacturer at : [http://www.cleware-shop.de/Downloads\\_EN](http://www.cleware-shop.de/Downloads_EN)
- The C/C++ source for Linux is modified by Cuong Nguyen (RBVH/ENG22) for support python to load dynamic library on Linux.

- 19.1.1 Method: `init_cleware`
- 19.1.2 Method: `open_cleware`
- 19.1.3 Method: `close_cleware`
- 19.1.4 Method: `get_handle`
- 19.1.5 Method: `set_value`
- 19.1.6 Method: `get_value`
- 19.1.7 Method: `set_switch`
- 19.1.8 Method: `get_switch`
- 19.1.9 Method: `get_version`
- 19.1.10 Method: `get_usb_type`
- 19.1.11 Method: `get_serial_number`
- 19.1.12 Method: `valid_ser_number`
- 19.1.13 Method: `get_hw_version`
- 19.1.14 Method: `is_ampel`
- 19.1.15 Method: `iox`
- 19.1.16 Method: `get_all_sw_state`

## Chapter 20

# ClewareAccessHelperWindows.py

### 20.1 Class: ClewareAccessHelperWindows

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.ClewareAccessHelperWindows
↳ import ClewareAccessHelperWindows
```

**ClewareAccessHelperWindows acts as the real object on Windows platform in Proxy Pattern**

- This is the wrapper class for the functions provided by USBaccess.dll.
- The USBaccess.dll is provided by Cleware manufacturer at : [http://www.cleware-shop.de/Downloads\\_EN](http://www.cleware-shop.de/Downloads_EN)

20.1.1 Method: `init_cleware`

20.1.2 Method: `open_cleware`

20.1.3 Method: `close_cleware`

20.1.4 Method: `get_handle`

20.1.5 Method: `set_value`

20.1.6 Method: `get_value`

20.1.7 Method: `set_switch`

20.1.8 Method: `get_switch`

20.1.9 Method: `get_version`

20.1.10 Method: `get_usb_type`

20.1.11 Method: `get_serial_number`

20.1.12 Method: `valid_ser_number`

20.1.13 Method: `get_hw_version`

20.1.14 Method: `is_ampel`

20.1.15 Method: `iox`

# Chapter 21

## ServiceCleware.py

### 21.1 Class: ServiceCleware

*Imported by:*

```
from  
↳ MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Cleware.py  
↳ import ServiceCleware
```

Service for controlling Cleware devices.

This class extends ServiceBase to provide functionalities specific to managing and controlling Cleware devices.

#### 21.1.1 Method: svc\_api\_get\_all\_devices\_state

Retrieve the state of all Cleware devices.

**Returns:**

*/ Type: dict /*

A dictionary containing the states of all Cleware devices.

#### 21.1.2 Method: svc\_api\_set\_switch

Set state for a Cleware device's switch.

**Arguments:**

- device\_no  
*/ Condition: required / Type: str /*  
Cleware device's number.
- switch\_id  
*/ Condition: required / Type: str /*  
Switch number to turn on or off.
- state  
*/ Condition: required / Type: str /*  
State of switch to set (on/off).

**Returns:**

*/ Type: int /*

Return ret code, 1 for succeed, 0 for failure.

### 21.1.3 Method: `notify_updates`

Notify updates to the realtime update channel for Cleware devices.

**Returns:**

*(no returns)*

## Chapter 22

# ServiceLogger.py

### 22.1 Class: ServiceLogger

*Imported by:*

```
from  
↳ MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa  
↳ import ServiceLogger
```

Logger class. microservicebase-microservicemanagergui-dist-win-unpacked-resources-python-services-clewareservice-  
servicelogger-servicelogger-set-logger -----  
-----

#### 22.1.1 Method: get\_config\_file

Get a name of a caller in the format module.class.method Args: skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

**Returns:** An empty string is returned if skipped levels exceed stack height

#### 22.1.2 Method: log

#### 22.1.3 Method: log\_info

#### 22.1.4 Method: log\_debug

#### 22.1.5 Method: log\_warning

#### 22.1.6 Method: log\_error

#### 22.1.7 Method: log\_critical

# Chapter 23

## Utils.py

### 23.1 Class: Singleton

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↳ import Singleton
```

Class to implement Singleton Design Pattern. This class is used to derive the TTFisClientReal as only a single instance of this class is allowed.

Disabled pyLint Messages: R0903: Too few public methods (%s/%s) Used when class has too few public methods, so be sure it's really worth it.

This base class implements the Singleton Design Pattern required for the TTFisClientReal. Adding further methods does not make sense.

### 23.2 Class: Utils

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↳ import Utils
```

Class to implement utilities for supporting development. microservicebase-microservicemanagergui-dist-win-unpacked-resources-python-services-cleware-service-utils-utils-get-all-descendant-classes -----

Get all descendant classes of a class Args: cls: Input class for finding descendants.

**Returns:** Array of descendant classes.

#### 23.2.1 Method: get\_all\_sub\_classes

Get all children classes of a class Args: cls: Input class for finding children.

**Returns:** Array of children classes.

#### 23.2.2 Method: caller\_name

Get a name of a caller in the format module.class.method Args: skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

**Returns:** An empty string is returned if skipped levels exceed stack height

### 23.2.3 Method: load\_library

Load native library depend on the calling convention. Args: path: library path. is\_stdcall: determine if the library's calling convention is stdcall or cdecl.

**Returns:** Loaded library object.

### 23.2.4 Method: is\_ascii\_or\_unicode

Check if the string is ascii or unicode Args: str\_check: string for checking codecs: encoding type list

**Returns:** True : if checked string is ascii or unicode False : if checked string is not ascii or unicode

### 23.2.5 Method: get\_registry\_parameters

Get service's parameters which stored in registry. Args: service\_name: Name of the service. key\_name: Name of register key.

**Returns:** Value of service's parameters

### 23.2.6 Method: create\_registry\_parameters

Create service's parameters and store it to registry. Args: service\_name: Name of the service. key\_name: Name of register key. parameters: Parameters to be stored.

**Returns:** None

## 23.3 Class: Job

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clew
↳ import Job
```

### 23.3.1 Method: stop

### 23.3.2 Method: run



## Chapter 24

### `__main__.py`

24.1 Function: `main`

24.2 Function: `signal_handler`

## Chapter 25

### main.py

25.1 Function: `main`

25.2 Function: `signal_handler`

## Chapter 26

# start\_bridge.py

Standalone FastAPI UI Bridge launcher.

Starts the FastAPI bridge that exposes microservices via REST API and WebSocket. Can be spawned by the Electron GUI or run directly from the command line.

### 26.1 Function: `parse_args`

### 26.2 Function: `main`

## Chapter 27

# start\_registry.py

Standalone Service Registry launcher.

Starts the Service Registry that tracks all services, handles registration events, and broadcasts updates to UI clients via a fanout exchange.

### 27.1 Function: `parse_args`

### 27.2 Function: `main`

## Chapter 28

### `__init__.py`

biplist -- a library for reading and writing binary property list files.

Binary Property List (plist) files provide a faster and smaller serialization format for property lists on OS X. This is a library for generating binary plists which can be read by OS X, iOS, or other clients.

The API models the plistlib API, and will call through to plistlib when XML serialization or deserialization is required.

To generate plists with UID values, wrap the values with the `Uid` object. The value must be an int.

To generate plists with `NSData`/`CFData` values, wrap the values with the `Data` object. The value must be a string.

Date values can only be `datetime.datetime` objects.

The exceptions `InvalidPlistException` and `NotBinaryPlistException` may be thrown to indicate that the data cannot be serialized or deserialized as a binary plist.

Plist generation example:

```
from biplist import * from datetime import datetime plist = {'aKey':'aValue', '0':1.322, 'now':datetime.now(),
'list':[1,2,3], 'tuple':('a','b','c')} } try: writePlist(plist, "example.plist") except (InvalidPlistException,
NotBinaryPlistException), e: print "Something bad happened:", e
```

Plist parsing example:

```
from biplist import * try: plist = readPlist("example.plist") print plist except (InvalidPlistException,
NotBinaryPlistException), e: print "Not a plist:", e
```

#### 28.1 Function: `readPlist`

Raises `NotBinaryPlistException`, `InvalidPlistException` `microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---wrapdataobject =====`

#### 28.2 Function: `writePlist`

#### 28.3 Function: `readPlistFromString`

#### 28.4 Function: `writePlistToString`

#### 28.5 Function: `is_stream_binary_plist`

#### 28.6 Class: `Uid`

*Imported by:*

```

from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↳ import Uid

```

Wrapper around integers for representing UID values. This is used in keyed archiving. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---data =====

*Imported by:*

```

from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↳ import Data

```

Wrapper around bytes to distinguish Data values. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---invalidplistexception =====

*Imported by:*

```

from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↳ import InvalidPlistException

```

Raised when the plist is incorrectly formatted. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---notbinaryplistexception =====

*Imported by:*

```

from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↳ import NotBinaryPlistException

```

Raised when a binary plist was expected but not encountered. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistreader =====

*Imported by:*

```

from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↳ import PlistReader

```

- 28.6.1 Method: parse
- 28.6.2 Method: reset
- 28.6.3 Method: readRoot
- 28.6.4 Method: setCurrentOffsetToObjectNumber
- 28.6.5 Method: beginOffsetProtection
- 28.6.6 Method: endOffsetProtection
- 28.6.7 Method: readObject
- 28.6.8 Method: readContents
- 28.6.9 Method: readInteger
- 28.6.10 Method: readReal
- 28.6.11 Method: readRefs
- 28.6.12 Method: readArray
- 28.6.13 Method: readDict
- 28.6.14 Method: readAsciiString
- 28.6.15 Method: readUnicode
- 28.6.16 Method: readDate
- 28.6.17 Method: readData
- 28.6.18 Method: readUid
- 28.6.19 Method: getSizedInteger

Numbers of 8 bytes are signed integers when they refer to numbers, but unsigned otherwise. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---hashablewrapper =====

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↳ import HashableWrapper
```

## 28.7 Class: BoolWrapper

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↳ import BoolWrapper
```

## 28.8 Class: FloatWrapper

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↳ import FloatWrapper
```

## 28.9 Class: StringWrapper

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↳ import StringWrapper
```

### 28.9.1 Method: encodingMarker

## 28.10 Class: PlistWriter

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↳ import PlistWriter
```

### 28.10.1 Method: reset

### 28.10.2 Method: positionOfObjectReference

If the given object has been written already, return its position in the offset table. Otherwise, return None.  
microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-writeroot -----

Strategy is: - write header - wrap root object so everything is hashable - compute size of objects which will be written - need to do this in order to know how large the object refs will be in the list/dict/set reference lists - write objects - keep objects in writtenReferences - keep positions of object references in referencePositions - write object references with the length computed previously - computer object reference length - write object reference positions - write trailer microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-beginrecursionprotection -----

### 28.10.3 Method: endRecursionProtection

### 28.10.4 Method: wrapRoot

### 28.10.5 Method: incrementByteCount

### 28.10.6 Method: computeOffsets

### 28.10.7 Method: writeObjectReference

Tries to write an object reference, adding it to the references table. Does not write the actual object bytes or set the reference position. Returns a tuple of whether the object was a new reference (True if it was, False if it already was in the reference table) and the new output.  
microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-writeobject -----

Serializes the given object to the output. Returns output. If setReferencePosition is True, will set the position the object was written.  
microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-writeoffsettable -----



Writes all of the object reference offsets. `microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-binaryreal` -----  
-----

### 28.10.8 Method: `binaryInt`

### 28.10.9 Method: `intSize`

Returns the number of bytes necessary to store the given integer. `microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-realsize` -----  
-----

## Chapter 29

# badge.py

### 29.1 Function: `badge_disk_icon`

# Chapter 30

## colors.py

### 30.1 Function: isAColor

### 30.2 Function: parseColor

### 30.3 Class: Color

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↳ import Color
```

#### 30.3.1 Method: to\_rgb

### 30.4 Class: RGB

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↳ import RGB
```

#### 30.4.1 Method: to\_rgb

### 30.5 Class: HSL

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↳ import HSL
```

#### 30.5.1 Method: to\_rgb

### 30.6 Class: HWB

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↳ import HWB
```

### 30.6.1 Method: to\_rgb

## 30.7 Class: CMYK

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↳ import CMYK
```

### 30.7.1 Method: to\_rgb

## 30.8 Class: Gray

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↳ import Gray
```

### 30.8.1 Method: to\_rgb

## 30.9 Class: ColorParser

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↳ import ColorParser
```

**30.9.1** Method: skipws

**30.9.2** Method: expect

**30.9.3** Method: expectEnd

**30.9.4** Method: getToken

**30.9.5** Method: parseNumber

**30.9.6** Method: parseColor

**30.9.7** Method: parseRGB

**30.9.8** Method: parseHSL

**30.9.9** Method: parseHWB

**30.9.10** Method: parseCMYK

**30.9.11** Method: parseGray

**30.9.12** Method: parseValue

**30.9.13** Method: parseAngle

# Chapter 31

## core.py

### 31.1 Function: build\_dmg

### 31.2 Class: DMGError

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.core
↳ import DMGError
```

# Chapter 32

## buddy.py

### 32.1 Class: BuddyError

*Imported by:*

```
from  
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.buddy  
↪ import BuddyError
```

### 32.2 Class: Block

*Imported by:*

```
from  
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.buddy  
↪ import Block
```

#### 32.2.1 Method: close

#### 32.2.2 Method: flush

#### 32.2.3 Method: invalidate

#### 32.2.4 Method: zero\_fill

#### 32.2.5 Method: tell

#### 32.2.6 Method: seek

#### 32.2.7 Method: read

#### 32.2.8 Method: write

#### 32.2.9 Method: insert

#### 32.2.10 Method: delete

### 32.3 Class: Allocator

*Imported by:*

```

from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds-store.buddy
↳ import Allocator

```

### 32.3.1 Method: open

### 32.3.2 Method: close

### 32.3.3 Method: flush

### 32.3.4 Method: read

Read data at 'offset', or raise an exception. 'size\_or\_format' may either be a byte count, in which case we return raw data, or a format string for 'struct.unpack', in which case we work out the size and unpack the data before returning it. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-buddy-allocator-write -----

Write data at 'offset', or raise an exception. 'data\_or\_format' may either be the data to write, or a format string for 'struct.pack', in which case we pack the additional arguments and write the resulting data. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-buddy-allocator-get-block -----

### 32.3.5 Method: allocate

Allocate or reallocate a block such that it has space for at least 'bytes' bytes. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-buddy-allocator-release -----

### 32.3.6 Method: iterkeys

### 32.3.7 Method: keys



# Chapter 33

## store.py

### 33.1 Class: ILocCodec

*Imported by:*

```
from  
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store  
↳ import ILocCodec
```

#### 33.1.1 Method: encode

#### 33.1.2 Method: decode

### 33.2 Class: PlistCodec

*Imported by:*

```
from  
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store  
↳ import PlistCodec
```

#### 33.2.1 Method: encode

#### 33.2.2 Method: decode

### 33.3 Class: BookmarkCodec

*Imported by:*

```
from  
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store  
↳ import BookmarkCodec
```

#### 33.3.1 Method: encode

#### 33.3.2 Method: decode

### 33.4 Class: DSStoreEntry

*Imported by:*

```

from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
↳ import DSStoreEntry

```

Holds the data from an entry in a `.DS_Store` file. Note that this is not meant to represent the entry itself--i.e. if you change the type or value, your changes will *not* be reflected in the underlying file.

If you want to make a change, you should either use the `DSStore` object's `DSStore.insert` method (which will replace a key if it already exists), or the mapping access mode for `DSStore` (often simpler anyway). `microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstoreentry-read` -----

Read a `.DS_Store` entry from the containing Block `microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstoreentry-byte-length` -----

Compute the length of this entry, in bytes `microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstoreentry-write` -----

Write this entry to the specified Block `microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore` =====

*Imported by:*

```

from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
↳ import DSStore

```

Python interface to a `.DS_Store` file. Works by manipulating the file on the disk--so this code will work with `.DS_Store` files for *very* large directories.

A `DSStore` object can be used as if it was a mapping, e.g.:

```
d['foobar.dat']['Iloc']
```

will fetch the "Iloc" record for "foobar.dat", or raise `KeyError` if there is no such record. If used in this manner, the `DSStore` object will return (type, value) tuples, unless the type is "blob" and the module knows how to decode it.

Currently, we know how to decode "Iloc", "bwsp", "lsvp", "lsvP" and "icvp" blobs. "Iloc" decodes to an (x, y) tuple, while the others are all decoded using `biplist`.

Assignment also works, e.g.:

```
d['foobar.dat']['note'] = ('ustr', u'Hello World!')
```

as does deletion with `del`:

```
del d['foobar.dat']['note']
```

This is usually going to be the most convenient interface, though occasionally (for instance when creating a new `.DS_Store` file) you may wish to drop down to using `DSStoreEntry` objects directly. `microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-open` -----

Open a `.DS_Store` file; pass either a Python file object, or a filename in the `file_or_name` argument and a file access mode in the `mode` argument. If you are creating a new file using the "w" or "w+" modes, you may also specify a list of entries with which to initialise the file. `microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-flush` -----

Flush any dirty data back to the file. `microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-close` -----

Flush dirty data and close the underlying file. `microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-insert` -----

Insert entry (which should be a `DSStoreEntry`) into the B-Tree. `microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-delete` -----

Delete an item, identified by `filename` and `code` from the B-Tree. `microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-find` -----  
-----

Returns a generator that will iterate over matching entries in the B-Tree.

# Chapter 34

## alias.py

### 34.1 Function: encode\_utf8

### 34.2 Function: decode\_utf8

### 34.3 Class: AppleShareInfo

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↳ import AppleShareInfo
```

### 34.4 Class: VolumeInfo

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↳ import VolumeInfo
```

#### 34.4.1 Method: filesystem\_type

### 34.5 Class: TargetInfo

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↳ import TargetInfo
```

### 34.6 Class: Alias

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↳ import Alias
```

### 34.6.1 Method: from\_bytes

Construct an `Alias` object given binary `Alias` data. `microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-alias-alias-for-file` -----  
-----

Create an `Alias` that points at the specified file. `microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-alias-alias-to-bytes` -----  
-----

Returns the binary representation for this `Alias`.

## Chapter 35

# bookmark.py

### 35.1 Function: iteritems

### 35.2 Class: Data

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.bookmar
↳ import Data
```

### 35.3 Class: URL

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.bookmar
↳ import URL
```

#### 35.3.1 Method: absolute

Return an absolute URL. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark =====

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.bookmar
↳ import Bookmark
```

#### 35.3.2 Method: from\_bytes

Create a `Bookmark` given byte data. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark-get -----

Lookup the value for a given key, returning a default if not present. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark-to-bytes -----

Convert this `Bookmark` to a byte representation. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark-for-file -----

Construct a `Bookmark` for a given file.

# Chapter 36

## osx.py

### 36.1 Function: getattrlist

### 36.2 Function: fgetattrlist

### 36.3 Function: statfs

### 36.4 Function: fstatfs

### 36.5 Class: attrlist

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↳ import attrlist
```

### 36.6 Class: attribute\_set\_t

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↳ import attribute_set_t
```

### 36.7 Class: fsobj\_id\_t

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↳ import fsobj_id_t
```

### 36.8 Class: timespec

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↳ import timespec
```

## 36.9 Class: attrreference\_t

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↳ import attrreference_t
```

## 36.10 Class: fsid\_t

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↳ import fsid_t
```

## 36.11 Class: guid\_t

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↳ import guid_t
```

## 36.12 Class: kauth\_ace

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↳ import kauth_ace
```

## 36.13 Class: kauth\_acl

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↳ import kauth_acl
```

## 36.14 Class: kauth\_filesec

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↳ import kauth_filesec
```



## 36.15 Class: diskextent

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↳ import diskextent
```

## 36.16 Class: Point

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↳ import Point
```

## 36.17 Class: Rect

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↳ import Rect
```

## 36.18 Class: FileInfo

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↳ import FileInfo
```

## 36.19 Class: FolderInfo

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↳ import FolderInfo
```

## 36.20 Class: FinderInfo

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↳ import FinderInfo
```

## 36.21 Class: vol\_capabilities\_attr\_t

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↳ import vol_capabilities_attr_t
```

## 36.22 Class: vol\_attributes\_attr\_t

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↳ import vol_attributes_attr_t
```

## 36.23 Class: struct\_statfs

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↳ import struct_statfs
```

# Chapter 37

## utils.py

### 37.1 Class: UTC

*Imported by:*

```
from  
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.utils  
↪ import UTC
```

#### 37.1.1 Method: utcoffset

#### 37.1.2 Method: dst

#### 37.1.3 Method: tzname

## Chapter 38

# launcher.py

FastAPI Bridge launcher.

Starts the FastAPI Bridge (REST/WS gateway) that connects the MicroserviceManagerGUI to microservices via RabbitMQ.

### 38.1 Function: `parse_args`

### 38.2 Function: `main`

## Chapter 39

# ClewareAccessHelper.py

### 39.1 Class: ClewareAccessHelper

*Imported by:*

```
from
↳ MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ClewareAccessHelper
↳ import ClewareAccessHelper
```

ClewareAccessHelper class acts as a proxy class in Proxy pattern and forward the request to the real helper depend on platform. microservicebase-microservicemanagergui-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-import-modules-from-paths -----

Import all modules from given paths.

**Arguments:**

- paths  
/ Condition: required / Type: list /  
List of paths to import modules from.

#### 39.1.1 Method: load\_config

Load the user config port name for USB access. Returns: None microservicebase-microservicemanagergui-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-get-config -----

Get Cleware ports' config. Returns: Dictionary of ports' setting. microservicebase-microservicemanagergui-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-set-config -----

Save Cleware ports' config to file. Args: config\_json: data to be saved.

**Returns:** res: saved result.

- 39.1.2 Method: `init_cleware`
- 39.1.3 Method: `open_cleware`
- 39.1.4 Method: `close_cleware`
- 39.1.5 Method: `get_handle`
- 39.1.6 Method: `set_value`
- 39.1.7 Method: `get_value`
- 39.1.8 Method: `set_switch`
- 39.1.9 Method: `set_switch_by_port_name`
- 39.1.10 Method: `get_switch`
- 39.1.11 Method: `get_version`
- 39.1.12 Method: `get_usb_type`
- 39.1.13 Method: `get_serial_number`
- 39.1.14 Method: `valid_ser_number`
- 39.1.15 Method: `get_hw_version`
- 39.1.16 Method: `is_ampel`
- 39.1.17 Method: `iox`
- 39.1.18 Method: `get_all_devices_state`

## Chapter 40

# ClewareAccessHelperAbs.py

### 40.1 Class: ClewareAccessHelperAbs

*Imported by:*

```
from  
↳ MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ClewareAccessHelp  
↳ import ClewareAccessHelperAbs
```

ClewareAccessHelperAbs acts as a abstract class for the real Helper in Proxy Pattern. microservicebase-microservicemanagergui-python-services-clewareservice-clewareaccesshelperabs-clewareaccesshelperabs-init-cleware

-----

40.1.1 Method: open\_cleware

40.1.2 Method: close\_cleware

40.1.3 Method: set\_value

40.1.4 Method: get\_value

40.1.5 Method: set\_switch

40.1.6 Method: get\_switch

40.1.7 Method: get\_handle

40.1.8 Method: get\_version

40.1.9 Method: get\_usb\_type

40.1.10 Method: get\_usb\_type

40.1.11 Method: get\_serial\_number

40.1.12 Method: valid\_ser\_number

40.1.13 Method: get\_hw\_version

40.1.14 Method: is\_ampel

40.1.15 Method: iox

## Chapter 41

# ClewareAccessHelperLinux.py

### 41.1 Class: ClewareAccessHelperLinux

*Imported by:*

```
from  
↪ MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ClewareAccessHelp  
↪ import ClewareAccessHelperLinux
```

**ClewareAccessHelperLinux acts as the real object on Linux platform in Proxy Pattern**

- This is the wrapper class for the functions provided by C/C++ source for Linux.
- The C/C++ source for Linux is provided by Cleware manufacturer at : [http://www.cleware-shop.de/Downloads\\_EN](http://www.cleware-shop.de/Downloads_EN)
- The C/C++ source for Linux is modified by Cuong Nguyen (RBVH/ENG22) for support python to load dynamic library on Linux.



- 41.1.1 Method: `init_cleware`
- 41.1.2 Method: `open_cleware`
- 41.1.3 Method: `close_cleware`
- 41.1.4 Method: `get_handle`
- 41.1.5 Method: `set_value`
- 41.1.6 Method: `get_value`
- 41.1.7 Method: `set_switch`
- 41.1.8 Method: `get_switch`
- 41.1.9 Method: `get_version`
- 41.1.10 Method: `get_usb_type`
- 41.1.11 Method: `get_serial_number`
- 41.1.12 Method: `valid_ser_number`
- 41.1.13 Method: `get_hw_version`
- 41.1.14 Method: `is_ampel`
- 41.1.15 Method: `iox`
- 41.1.16 Method: `get_all_sw_state`

## Chapter 42

# ClewareAccessHelperWindows.py

### 42.1 Class: ClewareAccessHelperWindows

*Imported by:*

```
from  
↳ MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ClewareAccessHelp  
↳ import ClewareAccessHelperWindows
```

**ClewareAccessHelperWindows acts as the real object on Windows platform in Proxy Pattern**

- This is the wrapper class for the functions provided by USBaccess.dll.
- The USBaccess.dll is provided by Cleware manufacturer at : [http://www.cleware-shop.de/Downloads\\_EN](http://www.cleware-shop.de/Downloads_EN)

**42.1.1 Method: init\_cleware**

**42.1.2 Method: open\_cleware**

**42.1.3 Method: close\_cleware**

**42.1.4 Method: get\_handle**

**42.1.5 Method: set\_value**

**42.1.6 Method: get\_value**

**42.1.7 Method: set\_switch**

**42.1.8 Method: get\_switch**

**42.1.9 Method: get\_version**

**42.1.10 Method: get\_usb\_type**

**42.1.11 Method: get\_serial\_number**

**42.1.12 Method: valid\_ser\_number**

**42.1.13 Method: get\_hw\_version**

**42.1.14 Method: is\_ampel**

**42.1.15 Method: iox**

# Chapter 43

## ServiceCleware.py

### 43.1 Class: ServiceCleware

*Imported by:*

```
from  
↪ MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ServiceCleware  
↪ import ServiceCleware
```

Service for controlling Cleware devices.

This class extends ServiceBase to provide functionalities specific to managing and controlling Cleware devices.

#### 43.1.1 Method: svc\_api\_get\_all\_devices\_state

Retrieve the state of all Cleware devices.

**Returns:**

*/ Type: dict /*

A dictionary containing the states of all Cleware devices.

#### 43.1.2 Method: svc\_api\_set\_switch

Set state for a Cleware device's switch.

**Arguments:**

- device\_no  
*/ Condition: required / Type: str /*  
Cleware device's number.
- switch\_id  
*/ Condition: required / Type: str /*  
Switch number to turn on or off.
- state  
*/ Condition: required / Type: str /*  
State of switch to set (on/off).

**Returns:**

*/ Type: int /*

Return ret code, 1 for succeed, 0 for failure.

### 43.1.3 Method: `notify_updates`

Notify updates to the realtime update channel for Cleware devices.

**Returns:**

*(no returns)*

## Chapter 44

# ServiceLogger.py

### 44.1 Class: ServiceLogger

*Imported by:*

```
from  
↪ MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ServiceLogger  
↪ import ServiceLogger
```

Logger class.    microservicebase-microservicemanagergui-python-services-clewareservice-servicelogger-servicelogger-set-logger -----

#### 44.1.1 Method: get\_config\_file

Get a name of a caller in the format module.class.method Args: skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

**Returns:** An empty string is returned if skipped levels exceed stack height

#### 44.1.2 Method: log

#### 44.1.3 Method: log\_info

#### 44.1.4 Method: log\_debug

#### 44.1.5 Method: log\_warning

#### 44.1.6 Method: log\_error

#### 44.1.7 Method: log\_critical

# Chapter 45

## Utils.py

### 45.1 Class: Singleton

*Imported by:*

```
from MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.Utils
↳ import Singleton
```

Class to implement Singleton Design Pattern. This class is used to derive the TTFisClientReal as only a single instance of this class is allowed.

Disabled pyLint Messages: R0903: Too few public methods (%s/%s) Used when class has too few public methods, so be sure it's really worth it.

This base class implements the Singleton Design Pattern required for the TTFisClientReal. Adding further methods does not make sense.

### 45.2 Class: Utils

*Imported by:*

```
from MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.Utils
↳ import Utils
```

Class to implement utilities for supporting development. microservicebase-microservicemanagergui-python-services-clewareservice-utils-utils-get-all-descendant-classes -----  
-----

Get all descendant classes of a class Args: cls: Input class for finding descendants.

**Returns:** Array of descendant classes.

#### 45.2.1 Method: get\_all\_sub\_classes

Get all children classes of a class Args: cls: Input class for finding children.

**Returns:** Array of children classes.

#### 45.2.2 Method: caller\_name

Get a name of a caller in the format module.class.method Args: skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

**Returns:** An empty string is returned if skipped levels exceed stack height

### 45.2.3 Method: load\_library

Load native library depend on the calling convention. Args: path: library path. is\_stdcall: determine if the library's calling convention is stdcall or cdecl.

**Returns:** Loaded library object.

### 45.2.4 Method: is\_ascii\_or\_unicode

Check if the string is ascii or unicode Args: str\_check: string for checking codecs: encoding type list

**Returns:** True : if checked string is ascii or unicode False : if checked string is not ascii or unicode

### 45.2.5 Method: get\_registry\_parameters

Get service's parameters which stored in registry. Args: service\_name: Name of the service. key\_name: Name of register key.

**Returns:** Value of service's parameters

### 45.2.6 Method: create\_registry\_parameters

Create service's parameters and store it to registry. Args: service\_name: Name of the service. key\_name: Name of register key. parameters: Parameters to be stored.

**Returns:** None

## 45.3 Class: Job

*Imported by:*

```
from MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.Utils  
↪ import Job
```

### 45.3.1 Method: stop

### 45.3.2 Method: run

## Chapter 46

### `__main__.py`

46.1 Function: `main`

46.2 Function: `signal_handler`



## Chapter 47

### main.py

47.1 Function: `main`

47.2 Function: `signal_handler`

## Chapter 48

# start\_bridge.py

Standalone FastAPI UI Bridge launcher.

Starts the FastAPI bridge that exposes microservices via REST API and WebSocket. Can be spawned by the Electron GUI or run directly from the command line.

### 48.1 Function: `parse_args`

### 48.2 Function: `main`

## Chapter 49

# start\_registry.py

Standalone Service Registry launcher.

Starts the Service Registry that tracks all services, handles registration events, and broadcasts updates to UI clients via a fanout exchange.

### 49.1 Function: `parse_args`

### 49.2 Function: `main`

# Chapter 50

## eventbus\_config.py

### 50.1 Class: EventBusConfig

*Imported by:*

```
from MicroserviceBase.adapters.config.eventbus_config import EventBusConfig
```

Configuration for EventBusClient-based adapters.

Wraps a JSONP config file path used by EventBusClient.from\_config\_sync().

#### 50.1.1 Method: load\_config

Load and return the parsed config as a dict.

Requires EventBusClient to be installed.

**Returns:**

*/ Type: dict /*

Parsed configuration dictionary.

#### 50.1.2 Method: to\_config\_source

Load config and return as a dict with optional field overrides.

Useful for creating derivative clients (e.g., registry or fanout) that share connection settings but differ in exchange configuration.

**Arguments:**

- overrides

*/ Condition: optional / Type: keyword arguments /*

Fields to override in the loaded config (e.g., exchange\_name, exchange\_handler).

**Returns:**

*/ Type: dict /*

Config dictionary with overrides applied.

# Chapter 51

## rabbitmq\_config.py

### 51.1 Class: RabbitMQConfig

*Imported by:*

```
from MicroserviceBase.adapters.config.rabbitmq.config import RabbitMQConfig
```

Configuration for connecting to RabbitMQ.

#### 51.1.1 Method: to\_connection\_params

Convert to pika ConnectionParameters kwargs.

**Returns:**

dict suitable for pika.ConnectionParameters(\*\*kwargs).

#### 51.1.2 Method: from\_cmd\_args

Parse RabbitMQ config from command-line arguments and environment variables.

**Arguments:**

- cmd\_args  
/ *Condition*: optional / *Type*: list /  
List of CLI arguments, or None for sys.argv.
- service\_name  
/ *Condition*: optional / *Type*: str / *Default*: 'Service' /  
Name of the service (used in help text).

**Returns:**

RabbitMQConfig instance.

## Chapter 52

### `__init__.py`

gRPC bridge adapter — dynamic method discovery and invocation.

Exposes `GrpcReflectClient` used by the FastAPI bridge to let the GUI enumerate and call methods on any gRPC service registered in Consul, without requiring the bridge to know the service's `.proto` ahead of time. All type information is obtained at runtime via the standard [gRPC Server Reflection Protocol](#).

## Chapter 53

# reflect\_client.py

Dynamic gRPC client using the server reflection protocol.

Services that enable the standard reflection service (which `MicroserviceBase.runtime.ServiceRunner` does automatically) can be called without their proto files being available locally — the reflection service returns their `FileDescriptorProto` bytes and we feed them into a `google.protobuf.descriptor_pool.DescriptorPool` to build dynamic message classes.

This keeps the GUI bridge completely decoupled from individual services: add a new service, and it just appears in the GUI with its methods browsable.

When the server **doesn't** ship reflection (e.g. built against `vcpkg's` `grpc` port which doesn't include `grpc++_reflection`), the bridge can fall back to `LocalProtoClient` which compiles `.proto` files from disk via `grpc_tools.protoc`. Same public API, no server help required.

Only **unary-unary** RPCs are supported at this stage. Streaming methods are detected and reported but cannot yet be invoked.

### 53.1 Class: GrpcReflectError

*Imported by:*

```
from MicroserviceBase.adapters.grpc_bridge.reflect_client import GrpcReflectError
```

Any failure from the reflection client — network, parse, or invoke.

**Attributes:** `is_unimplemented`: **True** when the failure came from a reflection RPC that the server returned `UNIMPLEMENTED` for — the canonical signal that the server wasn't built with reflection support. Bridge code uses this to decide whether to fall back to `LocalProtoClient`.

### 53.2 Class: GrpcReflectClient

*Imported by:*

```
from MicroserviceBase.adapters.grpc_bridge.reflect_client import GrpcReflectClient
```

Talk to a gRPC server using only its address + the reflection API.

**Usage:** `client = GrpcReflectClient("127.0.0.1:50051")` `services = client.list_services()` `methods = client.list_methods("hello.v1.HelloService")`  
`result = client.call_unary("hello.v1.HelloService", "Greet", '{"name": "World"}')`

The instance is **not** thread-safe. Create one per request on the bridge side. `microservicebase-adapters-grpc-bridge-reflect-client-grpcreflectclient-close` -----

### 53.2.1 Method: list\_services

Return all fully-qualified service names exposed by the target, excluding the built-in reflection / health services.  
 microservicebase-adapters-grpc-bridge-reflect-client-grpcreflectclient-list-methods -----  
 -----

Return a list of method descriptors for *full\_service\_name*.

**Each entry:** { "name": "Greet", "input\_type": "hello.v1.GreetRequest", "output\_type": "hello.v1.GreetResponse",  
               "client\_streaming": False, "server\_streaming": False, "input\_fields": [ {name, type, label, message\_type?},  
               ... ]  
               }

### 53.2.2 Method: call\_method

Invoke a unary-unary or server-streaming method.

- **Unary-unary** → returns {"streaming": False, "result": <dict>}.
- **Server-streaming** → reads up to *max\_events* events from the stream (or until *max\_seconds* elapses), then cancels the RPC and returns {"streaming": True, "events": [<dict>, ...], "truncated": bool}.
- **Client- or bidi-streaming** → raises GrpcReflectError.

Client-streaming and bidirectional streaming need a different UI pattern (incremental request sends + cancel button) and are intentionally rejected here. microservicebase-adapters-grpc-bridge-reflect-client-grpcreflectclient-call-unary  
 -----

## 53.3 Class: LocalProtoClient

Imported by:

```
from MicroserviceBase.adapters.grpc_bridge.reflect_client import LocalProtoClient
```

Drop-in replacement for GrpcReflectClient when the server has no reflection support.

Builds the descriptor pool by compiling local .proto files via `grpc_tools.protoc` instead of asking the server. Has the same public methods (`list_services`, `list_methods`, `call_method`, `call_unary`) so callers can swap one for the other.

The client owns a gRPC channel to *target* for the actual invocations — only schema discovery is local.

**Usage:** `client = LocalProtoClient( "127.0.0.1:50051", proto_files=["proto/hello.proto"], # explicit`  
       `) # or client = LocalProtoClient.from_search_paths( "127.0.0.1:50051", search_paths=["./proto", "C:/projects"],`  
       `)`

### 53.3.1 Method: from\_search\_paths

Build a client by globbing `**/*.proto` under each search path.

Empty/missing dirs are silently skipped. Each search path is also added as a `protoc -I` include path so cross-file imports resolve. microservicebase-adapters-grpc-bridge-reflect-client-localprotoclient-close -----  
 -----

### 53.3.2 Method: list\_services

Return all fully-qualified service names found in the loaded proto files, excluding built-in reflection / health services. microservicebase-adapters-grpc-bridge-reflect-client-localprotoclient-list-methods -----  
 -----



Same shape as GrpcReflectClient.list\_methods.      microservicebase-adapters-grpc-bridge-reflect-client-localprotoclient-call-method -----  
Same semantics as GrpcReflectClient.call\_method.      microservicebase-adapters-grpc-bridge-reflect-client-localprotoclient-call-unary -----

# Chapter 54

## local\_hub\_manager.py

Local Hub Manager -- manages a local ProcessHub instance lifecycle.

Wraps ProcessHub APIs (lazy import) so the MicroserviceManager GUI can start/stop a ProcessHub on the local machine, manage processes, and optionally join a fleet as an agent.

### 54.1 Class: LocalHubManager

*Imported by:*

```
from MicroserviceBase.adapters.local_hub.local_hub_manager import LocalHubManager
```

Manages a local ProcessHub instance lifecycle.

#### 54.1.1 Method: start\_hub

Start a local ProcessHub server.

**Arguments:**

- mode  
/ *Condition*: optional / *Type*: str / *Default*: 'standalone' /  
"standalone", "agent" (join fleet), or "orchestrator".
- xpub\_port  
/ *Condition*: optional / *Type*: int / *Default*: 5555 /  
ZMQ X PUB port for the broker.
- xsub\_port  
/ *Condition*: optional / *Type*: int / *Default*: 5556 /  
ZMQ X SUB port for the broker.
- orchestrator\_url  
/ *Condition*: optional / *Type*: str /  
Fleet orchestrator ZMQ address (agent mode).
- hub\_id  
/ *Condition*: optional / *Type*: str /  
Hub identifier (agent/orchestrator mode).
- hub\_name  
/ *Condition*: optional / *Type*: str /  
Human-readable hub name (agent/orchestrator mode).

- `process_config`  
/ *Condition*: optional / *Type*: dict /  
Dict of {name: config} for processes.
- `fleet_api_port`  
/ *Condition*: optional / *Type*: int / *Default*: 2510 /  
FleetWebAPI port (orchestrator mode).
- `health_timeout`  
/ *Condition*: optional / *Type*: float / *Default*: 30.0 /  
Health check timeout in seconds (orchestrator mode).

**Returns:**

/ *Type*: dict /  
Status dict.

**54.1.2 Method: stop\_hub**

Stop the local hub.

**54.1.3 Method: get\_status**

Get current hub status.

**54.1.4 Method: start\_processes**

Start named processes.

**Arguments:**

- `names`  
/ *Condition*: required / *Type*: list /  
List of process names to start.

**54.1.5 Method: stop\_processes**

Stop named processes.

**Arguments:**

- `names`  
/ *Condition*: required / *Type*: list /  
List of process names to stop.
- `force`  
/ *Condition*: optional / *Type*: bool / *Default*: False /  
If True, force-kill the processes.

**54.1.6 Method: get\_config**

Get all process configurations.

### 54.1.7 Method: add\_config

Add a new process configuration.

**Arguments:**

- name  
/ *Condition*: required / *Type*: str /  
Process name.
- config  
/ *Condition*: required / *Type*: dict /  
Process configuration dict.

### 54.1.8 Method: update\_config

Update an existing process configuration.

**Arguments:**

- name  
/ *Condition*: required / *Type*: str /  
Process name.
- config  
/ *Condition*: required / *Type*: dict /  
Updated process configuration dict.

### 54.1.9 Method: remove\_config

Remove a process configuration.

**Arguments:**

- name  
/ *Condition*: required / *Type*: str /  
Process name.

### 54.1.10 Method: remove\_service

Remove a service -- delete config and managed service folder.

Only deletes the folder if it lives inside the managed <config\_dir>/services/ directory (never touches external paths). Stops the process first if it is still running.

**Arguments:**

- name  
/ *Condition*: required / *Type*: str /  
Service name.

### 54.1.11 Method: get\_service\_log

Read the last *tail* lines of a process log file.

**Arguments:**

- name  
/ *Condition*: required / *Type*: str /  
Process name.

- `tail`  
/ *Condition*: optional / *Type*: int / *Default*: 100 /  
Number of lines to read from the tail.

#### 54.1.12 Method: `get_services_dir`

Return absolute path to `<config_dir>/services/`, creating it if needed.

#### 54.1.13 Method: `import_service`

Import a microservice from a folder or base64-encoded ZIP.

Validates structure, copies files into `<config_dir>/services/{name}/`, and creates a hub config entry using the `${python}` placeholder.

##### Arguments:

- `name`  
/ *Condition*: required / *Type*: str /  
Service name.
- `source_path`  
/ *Condition*: optional / *Type*: str /  
Path to the source directory to import from.
- `zip_data`  
/ *Condition*: optional / *Type*: str /  
Base64-encoded ZIP data to import from.
- `wait_time`  
/ *Condition*: optional / *Type*: float / *Default*: 1.0 /  
Seconds to wait after starting before checking process health.

##### Returns:

/ *Type*: dict /  
`{"success": bool, "message": str, "warnings": [...]}`.

#### 54.1.14 Method: `reset`

Reset the hub (stop processes, clear connections).

# Chapter 55

## service\_executor.py

Service-aware process executor.

Wraps a `SimpleExecutor` and attempts graceful RPC shutdown (`svc_api_shutdown`) before falling back to signal-based stop.

### 55.1 Class: ServiceExecutor

*Imported by:*

```
from MicroserviceBase.adapters.local_hub.service_executor import ServiceExecutor
```

Process executor that sends an RPC shutdown to `MicroserviceBase` services.

Delegates all operations to a wrapped `SimpleExecutor`. On `stop()`, it first tries to send `svc_api_shutdown` via RabbitMQ to the service's queue. If the service exits within `shutdown_timeout` seconds the process is considered stopped. Otherwise it falls back to the delegate's signal-based stop (`CTRL_BREAK_EVENT` on Windows, `SIGTERM` on Linux).

#### 55.1.1 Method: start

Start a named process using the given config.

**Arguments:**

- `name`  
/ *Condition*: required / *Type*: str /  
Process name.
- `config`  
/ *Condition*: required / *Type*: dict /  
Process configuration dict with 'script', 'args', 'cwd', etc.

#### 55.1.2 Method: get\_log\_path

Public accessor for log file path.

**Arguments:**

- `name`  
/ *Condition*: required / *Type*: str /  
Process name.

### 55.1.3 Method: read\_log

Public method to read process log. Returns the text content.

**Arguments:**

- `name`  
/ *Condition*: required / *Type*: str /  
Process name.
- `tail`  
/ *Condition*: optional / *Type*: int / *Default*: 100 /  
Number of lines to read from the tail.

### 55.1.4 Method: is\_running

Check if a named process is running.

**Arguments:**

- `name`  
/ *Condition*: required / *Type*: str /  
Process name.

### 55.1.5 Method: get\_pid

Get the PID of a named process.

**Arguments:**

- `name`  
/ *Condition*: required / *Type*: str /  
Process name.

### 55.1.6 Method: stop

Stop a named process, attempting RPC shutdown first.

**Arguments:**

- `name`  
/ *Condition*: required / *Type*: str /  
Process name.
- `force`  
/ *Condition*: optional / *Type*: bool / *Default*: False /  
If True, skip graceful shutdown and force-kill.

## Chapter 56

# nomad\_client.py

### 56.1 Class: NomadAPIError

*Imported by:*

```
from MicroserviceBase.adapters.nomad_hub.nomad_client import NomadAPIError
```

Raised when the Nomad API returns a non-2xx response. microservicebase-adapters-nomad-hub-nomad-client-nomadclient =====

*Imported by:*

```
from MicroserviceBase.adapters.nomad_hub.nomad_client import NomadClient
```

HTTP client for the HashiCorp Nomad v1 API.

Uses only `urllib` from the standard library — no external dependencies. Supports ACL token authentication and blocking queries (long-poll).

#### 56.1.1 Method: agent\_self

GET /v1/agent/self — verify agent connectivity. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-list-jobs -----

GET /v1/jobs — list all jobs (optional prefix filter). microservicebase-adapters-nomad-hub-nomad-client-nomadclient-get-job -----

GET /v1/job/:id — read full job specification. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-register-job -----

POST /v1/jobs — register (create or update) a job. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-parse-hcl -----

POST /v1/jobs/parse — convert an HCL job spec to JSON.

Used by the GUI "Submit Job" dialog so users can paste HCL directly instead of JSON. Nomad's own `nomad job run` does the same under the hood.

Returns the parsed job dict ready to be wrapped in `{"Job": ...}` and POSTed to `/v1/jobs`. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-stop-job -----

DELETE /v1/job/:id — stop a job. `purge=True` removes it entirely. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-get-allocations -----

GET /v1/job/:id/allocations — list allocations for a job. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-get-allocation -----

GET /v1/allocation/:id — read a single allocation. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-get-logs -----

GET /v1/client/fs/logs/:alloc\_id — read task logs.

Returns plain text when `plain=True`, otherwise JSON frames.



### 56.1.2 Method: `list_jobs_blocking`

GET `/v1/jobs` with blocking query.

Blocks until the job list changes or `wait` expires. Uses the `X-Nomad-Index` from the previous call.

## Chapter 57

# nomad\_hub\_adapter.py

### 57.1 Class: NomadHubAdapter

*Imported by:*

```
from MicroserviceBase.adapters.nomad_hub.nomad_hub_adapter import NomadHubAdapter
```

HubManagerPort implementation that manages services as Nomad jobs.

Each service config is translated into a Nomad job spec using the `raw_exec` driver (Python processes, no Docker required).

**57.1.1 Method: `start_hub`**

**57.1.2 Method: `stop_hub`**

**57.1.3 Method: `get_status`**

**57.1.4 Method: `start_processes`**

**57.1.5 Method: `stop_processes`**

**57.1.6 Method: `get_config`**

**57.1.7 Method: `add_config`**

**57.1.8 Method: `update_config`**

**57.1.9 Method: `remove_config`**

**57.1.10 Method: `get_service_log`**

**57.1.11 Method: `import_service`**

**57.1.12 Method: `remove_service`**

**57.1.13 Method: `reset`**

## Chapter 58

# amqp\_registry\_adapter.py

### 58.1 Class: AMQPRegistryAdapter

*Imported by:*

```
from MicroserviceBase.adapters.registry.amqp-registry-adapter import  
↪ AMQPRegistryAdapter
```

AMQP implementation of the ServiceRegistryPort interface.

Uses RabbitMQ topic exchange for service registration events and fanout exchange for realtime update broadcasts.

#### 58.1.1 Method: register

Register a service by publishing to the service\_information exchange.

**Arguments:**

- `service_info`  
/ *Condition*: required / *Type*: dict /  
dict containing service metadata.

#### 58.1.2 Method: unregister

Unregister a service by publishing to the service\_information exchange.

**Arguments:**

- `service_info`  
/ *Condition*: required / *Type*: dict /  
dict containing service metadata.

#### 58.1.3 Method: subscribe\_to\_events

Subscribe to service registration events.

Runs in the calling thread (blocking). Typically called from a daemon thread.

**Arguments:**

- `handler`  
/ *Condition*: required / *Type*: callable /  
Callback function(ch, method, properties, body).

#### 58.1.4 Method: `notify_update`

Broadcast services update to the realtime update fanout exchange.

**Arguments:**

- `services_info`  
/ *Condition*: required / *Type*: dict /  
dict of all current service information.

#### 58.1.5 Method: `subscribe_to_updates`

Subscribe to the realtime update fanout exchange.

Unlike `subscribe_to_events` (which listens for individual on/off events), this subscribes to the full services dict broadcast by the Registry after each change. Uses an exclusive temporary queue to avoid competing consumers.

Runs in the calling thread (blocking). Typically called from a daemon thread.

**Arguments:**

- `handler`  
/ *Condition*: required / *Type*: callable /  
Callback function(`services_info_dict`) called with the full services dict.

#### 58.1.6 Method: `get_update_channel_name`

Return the realtime update exchange name.

#### 58.1.7 Method: `cleanup_update_exchange`

Delete the realtime update exchange (for cleanup on shutdown).

#### 58.1.8 Method: `cleanup`

Close event subscription resources and clean up the update exchange.

## Chapter 59

# eventbus\_registry\_adapter.py

### 59.1 Class: EventBusRegistryAdapter

*Imported by:*

```
from MicroserviceBase.adapters.registry.eventbus_registry_adapter import
↳ EventBusRegistryAdapter
```

EventBusClient implementation of the ServiceRegistryPort interface.

Uses EventBusClient with TopicExchangeHandler for service registration events and a FanoutExchangeHandler for realtime update broadcasts.

Both clients are created from the same JSONP config with exchange overrides via `EventBusConfig.to_config_source()`.

#### 59.1.1 Method: register

Register a service by publishing to the service\_information exchange.

**Arguments:**

- `service_info`  
/ *Condition:* required / *Type:* dict /  
dict containing service metadata.

#### 59.1.2 Method: unregister

Unregister a service by publishing to the service\_information exchange.

**Arguments:**

- `service_info`  
/ *Condition:* required / *Type:* dict /  
dict containing service metadata.

#### 59.1.3 Method: subscribe\_to\_events

Subscribe to service registration events.

Blocks the calling thread. Typically called from a daemon thread. The handler receives the pika-style callback signature (ch, method, properties, body) for backward compatibility.

**Arguments:**

- `handler`  
/ *Condition:* required / *Type:* callable /  
Callback function(ch, method, properties, body).

#### 59.1.4 Method: `notify_update`

Broadcast services update to the realtime update fanout exchange.

**Arguments:**

- `services_info`  
/ *Condition*: required / *Type*: dict /  
dict of all current service information.

#### 59.1.5 Method: `get_update_channel_name`

Return the realtime update exchange name.

#### 59.1.6 Method: `cleanup`

Close all EventBusClient connections.

## Chapter 60

# `__init__.py`

Service scaffold generator.

Generates complete project directories for new microservices based on user selections (language, GUI type, infrastructure). Used by the Service Creator wizard in MicroserviceManagerGUI.

Supported combinations:

| Language | GUI type | Output                                                   | -----  | -----  | -----                                 |
|----------|----------|----------------------------------------------------------|--------|--------|---------------------------------------|
| Python   | none     | Async gRPC service (domain + adapter + proto)            | Python | html   | Same + GUIs/service.html + service.js |
| C++      | none     | CMake project (domain + adapter + proto + build scripts) | C++    | qml    | Same + QML UI + preview app + stubs   |
| C++      | wasm     | Same + Qt Widgets WASM UI + build.wasm script            | C++    | widget | Same + Qt Widgets desktop UI          |

**All combinations optionally include:**

- Nomad job spec (.nomad.hcl)
- Build/deploy scripts
- README.md

# Chapter 61

## cpp\_tmpl.py

C++ service scaffold templates.

Generates CMake-based projects for gRPC microservices using the MicroserviceBase C++ runtime (ServiceRunner + ConsulRegistration). Supports four GUI variants: none, qml, wasm, widget.

### 61.1 Function: generate

### 61.2 Function: generate\_monorepo

Emit a single-project scaffold with one executable per service.

All services share one proto (written verbatim from spec.proto\_content\_override or regenerated), one CMakeLists.txt with N add\_executable() calls, and a single build\_deploy.bat that drops every .exe into dist/.

services is a list of ServiceBlock (name + methods). v1: C++ only, no client subproject, no Qt widget GUI per service.

microservicebase-adapters-scaffold-cpp-tmpl-generate-multi-proto =====

Emit a multi-proto / single-binary scaffold (vehicle-example pattern).

**Each service in `services` becomes:**

- its own .proto file under proto/ (with its own package),
- its own domain/ + adapters/api/ source folder,

and ALL services are registered on one grpc::ServerBuilder by the project's single src/main.cpp.

v1: C++ only. Borrows monorepo CMake auto-detection + set\_env / build script infrastructure unchanged.



## Chapter 62

# generator.py

Top-level scaffold generator — dispatches to language-specific modules.

The entry point `generate_scaffold` returns a dict of `{relative_path: content_string}` that the bridge endpoint can write to disk or package into a ZIP.

### 62.1 Function: `generate_scaffold`

Return `{path: content}` for every file in the scaffold.

Paths are relative to the project root (e.g. `src/main.cpp`). `microservicebase-adapters-scaffold-generator-methodparam` =====

*Imported by:*

```
from MicroserviceBase.adapters.scaffold.generator import MethodParam
```

### 62.2 Class: `MethodSpec`

*Imported by:*

```
from MicroserviceBase.adapters.scaffold.generator import MethodSpec
```

### 62.3 Class: `ServiceBlock`

*Imported by:*

```
from MicroserviceBase.adapters.scaffold.generator import ServiceBlock
```

One gRPC service entry inside a multi-service scaffold.

Used by two layouts:

- **monorepo** - N services share ONE `.proto`, each builds its own executable. `proto_file` is ignored; the project's main `.proto` holds all services.
- **multi\_proto** - N services come from N separate `.protos` but compile into ONE executable that registers them all on the same `grpc::ServerBuilder`. Each service carries its own `proto_file` (defaults to `<snake_name>.proto` when empty).

### 62.4 Class: `ScaffoldSpec`

*Imported by:*

```
from MicroserviceBase.adapters.scaffold.generator import ScaffoldSpec
```

Everything the wizard collects across all steps. microservicebase-adapters-scaffold-generator-scaffoldspec-snake-name  
-----

CamelCase → snake\_case, keeping runs of capitals together.

MyService → my\_service, PPSService → pps\_service, XMLParser → xml\_parser. microservicebase-  
adapters-scaffold-generator-scaffoldspec-proto-package -----

### 62.4.1 Method: proto\_namespace

C++ namespace form of proto\_package (dots → ::). microservicebase-adapters-scaffold-generator-scaffoldspec-env-  
prefix -----

## Chapter 63

# python\_tmpl.py

Python service scaffold templates.

Generates the full project structure for a Python gRPC microservice using the MicroserviceBase async runtime (ServiceRunner + Consul).

### 63.1 Function: generate

### 63.2 Function: generate\_monorepo

Emit a Python monorepo scaffold with one entry-point per service.

**Layout:** **ProjectName/** pyproject.toml ← single project, lists each service README.md as a [project.scripts] entry point  
proto/ <project>.proto ← shared <project>\_pb2.py ← shared generated stubs <project>\_pb2\_grpc.py  
scripts/ generate\_protos.py deploy/ <svc1>.nomad.hcl ← one job per service <svc1>\_service\_config.json  
<svc2>.nomad.hcl ... src/ <project\_snake>/ ← Python package \_\_init\_\_.py <svc1>/ ← per-service  
module \_\_init\_\_.py main.py ← entry point — python -m <project>.<svc> config.py ← per-service env  
vars context.py ← per-service DI domain/<svc>.py adapters/api/grpc\_adapter.py <svc2>/ ...

All services share one .proto + one set of generated stubs. Each service has its own env-var prefix (<SVC1>-), its own Nomad job, and its own asyncio entry point. Mirrors the C++ monorepo layout (cpp\_tmpl.generate\_monorepo) but in Python.

## Chapter 64

# shared.py

Shared scaffold generators used by both Python and C++ templates.

Produces: - .proto file - service\_config.json - Nomad .nomad.hcl job spec - README.md

**64.1    Function: `gen_proto`**

**64.2    Function: `gen_service_config`**

**64.3    Function: `gen_nomad`**

**64.4    Function: `gen_readme`**

# Chapter 65

## eventbus\_adapter.py

### 65.1 Class: `_ChannelProxy`

*Imported by:*

```
from MicroserviceBase.adapters.transport.eventbus_adapter import _ChannelProxy
```

Lightweight proxy that mimics a pika channel for the domain handler.

Translates `basic_publish` and `basic_ack` calls into `EventBusClient` operations.

#### 65.1.1 Method: `basic_publish`

Publish a reply message via the `EventBusClient`.

**Arguments:**

- `exchange`  
/ *Condition*: required / *Type*: str /  
Exchange name (ignored, `EventBusClient` uses its configured exchange).
- `routing_key`  
/ *Condition*: required / *Type*: str /  
Routing key for the reply (the `reply_to` value).
- `properties`  
/ *Condition*: optional / *Type*: object /  
Properties object with `correlation_id` attribute.
- `body`  
/ *Condition*: optional / *Type*: str or bytes /  
The message body (JSON string or bytes).

#### 65.1.2 Method: `basic_ack`

Acknowledge a message (no-op for `EventBusClient` which auto-acks).

### 65.2 Class: `_MethodProxy`

*Imported by:*

```
from MicroserviceBase.adapters.transport.eventbus_adapter import _MethodProxy
```

Lightweight proxy that mimics a pika method frame.

## 65.3 Class: \_PropertiesProxy

Imported by:

```
from MicroserviceBase.adapters.transport.eventbus_adapter import _PropertiesProxy
```

Lightweight proxy that mimics pika BasicProperties.

## 65.4 Class: EventBusTransportAdapter

Imported by:

```
from MicroserviceBase.adapters.transport.eventbus_adapter import
↳ EventBusTransportAdapter
```

EventBusClient implementation of the TransportPort interface.

Creates the underlying EventBusClient from a JSONP configuration file via `EventBusClient.from_config_sync()`.

### 65.4.1 Method: connect

Create the EventBusClient from the JSONP config and establish connection.

### 65.4.2 Method: disconnect

Close the EventBusClient connection.

### 65.4.3 Method: consume

Start consuming RPC requests for a service.

Subscribes to the routing key and blocks the calling thread. Incoming messages are adapted to the pika-style handler signature.

**Arguments:**

- `service_name`  
/ *Condition*: required / *Type*: str /  
Name used as the service identifier.
- `routing_key`  
/ *Condition*: required / *Type*: str /  
Routing key for message binding.
- `exchange`  
/ *Condition*: required / *Type*: str /  
Exchange name (must match the configured exchange).
- `handler`  
/ *Condition*: required / *Type*: callable /  
Callback function(ch, method, props, body).

### 65.4.4 Method: rpc\_call

Send an RPC request and wait for the response.

Creates a temporary subscription for the reply, sends the request with `reply_to` and `correlation_id` headers, and waits for the response.

**Arguments:**

- `request_data`  
/ *Condition*: required / *Type*: dict /  
dict to send as the request body.
- `exchange_name`  
/ *Condition*: required / *Type*: str /  
Exchange to publish to (must match configured exchange).
- `routing_key`  
/ *Condition*: required / *Type*: str /  
Routing key for the target service.

**Returns:**

Parsed response dict.

**65.4.5 Method: publish**

Publish a message to the exchange.

**Arguments:**

- `exchange`  
/ *Condition*: required / *Type*: str /  
Exchange name (must match configured exchange).
- `routing_key`  
/ *Condition*: required / *Type*: str /  
Routing key.
- `body`  
/ *Condition*: required / *Type*: str, bytes, or dict /  
Message body.
- `properties`  
/ *Condition*: optional / *Type*: dict /  
Optional message properties (used for headers).

## Chapter 66

# rabbitmq\_adapter.py

### 66.1 Class: RabbitMQTransportAdapter

*Imported by:*

```
from MicroserviceBase.adapters.transport.rabbitmq_adapter import  
↳ RabbitMQTransportAdapter
```

RabbitMQ implementation of the TransportPort interface.

#### 66.1.1 Method: connect

Establish connection to RabbitMQ.

#### 66.1.2 Method: disconnect

Close connection to RabbitMQ.

#### 66.1.3 Method: connection

Access the underlying pika connection (for backward compat).

#### 66.1.4 Method: stop\_consuming

Signal the consume loop to exit.

#### 66.1.5 Method: consume

Start consuming RPC requests for a service.

Sets up the exchange, queue, binding, and starts blocking consumption.

**Arguments:**

- `service_name`  
/ *Condition*: required / *Type*: str /  
Name used as the queue name.
- `routing_key`  
/ *Condition*: required / *Type*: str /  
Routing key for queue binding.



- `exchange`  
/ *Condition*: required / *Type*: str /  
Exchange name to bind to.
- `handler`  
/ *Condition*: required / *Type*: callable /  
Callback function(ch, method, props, body).

### 66.1.6 Method: `rpc_call`

Send an RPC request and wait for the response.

Creates a new connection for the RPC call (matching original behavior).

#### Arguments:

- `request_data`  
/ *Condition*: required / *Type*: dict /  
dict to serialize as the request body.
- `exchange_name`  
/ *Condition*: required / *Type*: str /  
Exchange to publish to.
- `routing_key`  
/ *Condition*: required / *Type*: str /  
Routing key for the target service.
- `timeout`  
/ *Condition*: optional / *Type*: int / *Default*: 30 /  
Timeout in seconds for waiting for the response.

#### Returns:

Parsed response dict.

### 66.1.7 Method: `publish`

Publish a message to an exchange.

#### Arguments:

- `exchange`  
/ *Condition*: required / *Type*: str /  
Exchange name.
- `routing_key`  
/ *Condition*: required / *Type*: str /  
Routing key.
- `body`  
/ *Condition*: required / *Type*: str or bytes /  
Message body (str or bytes).
- `properties`  
/ *Condition*: optional / *Type*: pika.BasicProperties /  
Optional pika.BasicProperties.

# Chapter 67

## fastapi\_bridge.py

### 67.1 Class: FastAPIBridge

*Imported by:*

```
from MicroserviceBase.adapters.ui_bridge.fastapi_bridge import FastAPIBridge
```

FastAPI implementation of UIBridgePort.

Exposes a REST API that any UI client (Electron, browser, mobile) can call. Also provides a WebSocket endpoint for push-based service updates.

**Endpoints:** POST /api/request - Route a service request through the transport GET /api/services - Get current services info WS /ws/updates - Real-time service update stream

Requires `fastapi` and `uvicorn` (optional dependencies).

#### 67.1.1 Method: start

Start the FastAPI server in a background thread.

**Arguments:**

- `request_handler`  
/ *Condition:* required / *Type:* callable /  
Callable(request\_data, exchange, routing\_key) -> response dict.
- `services_info_provider`  
/ *Condition:* required / *Type:* callable /  
Callable() -> services info dict.

#### 67.1.2 Method: wait

Block the calling thread until the server thread exits.

Uses a loop with timeout so that KeyboardInterrupt can be caught on Windows.

#### 67.1.3 Method: stop

Stop the FastAPI server.

#### 67.1.4 Method: broadcast\_update

Push a services update to all connected WebSocket clients.

**Arguments:**

- `services_info`  
/ *Condition*: required / *Type*: dict /  
dict of all current service information.

# Chapter 68

## exceptions.py

### 68.1 Class: ServiceError

*Imported by:*

```
from MicroserviceBase.domain.exceptions import ServiceError
```

Base exception for service-related errors.

### 68.2 Class: TransportError

*Imported by:*

```
from MicroserviceBase.domain.exceptions import TransportError
```

Raised when a transport-level operation fails.

### 68.3 Class: ServiceNotFoundError

*Imported by:*

```
from MicroserviceBase.domain.exceptions import ServiceNotFoundError
```

Raised when a requested service is not found in the registry.

### 68.4 Class: MethodNotFoundError

*Imported by:*

```
from MicroserviceBase.domain.exceptions import MethodNotFoundError
```

Raised when a requested method is not found on a service.

# Chapter 69

## messages.py

### 69.1 Class: ResultType

*Imported by:*

```
from MicroserviceBase.domain.messages import ResultType
```

Result types for service responses.

### 69.2 Class: ServiceRequest

*Imported by:*

```
from MicroserviceBase.domain.messages import ServiceRequest
```

Structured request to a service method.

#### 69.2.1 Method: to\_dict

Converts this `ServiceRequest` instance to a dictionary.

**Returns:**

*/ Type: dict /*

Dictionary representation of this request.

#### 69.2.2 Method: to\_json

Converts this `ServiceRequest` instance to a JSON string.

**Returns:**

*/ Type: str /*

JSON string representation of this request.

#### 69.2.3 Method: from\_dict

Creates a `ServiceRequest` instance from a dictionary.

**Arguments:**

- data

*/ Condition: required / Type: dict /*

Dictionary containing request fields.

**Returns:**

*/ Type: ServiceRequest /*  
New instance populated from the dictionary.

**69.2.4 Method: from\_json**

Creates a `ServiceRequest` instance from a JSON string.

**Arguments:**

- `json_str`  
*/ Condition: required / Type: str /*  
JSON string containing request fields.

**Returns:**

*/ Type: ServiceRequest /*  
New instance populated from the JSON string.

**69.3 Class: ServiceResponse**

*Imported by:*

```
from MicroserviceBase.domain.messages import ServiceResponse
```

Structured response from a service method.

**69.3.1 Method: to\_dict**

Converts this `ServiceResponse` instance to an ordered dictionary.

**Returns:**

*/ Type: OrderedDict /*  
Sorted ordered dictionary representation of this response.

**69.3.2 Method: to\_json**

Converts this `ServiceResponse` instance to a JSON string.

**Returns:**

*/ Type: str /*  
JSON string representation of this response.

**69.3.3 Method: get\_json**

Backward-compatible alias for `to_json()`.

**Returns:**

*/ Type: str /*  
JSON string representation of this response.

### 69.3.4 Method: `get_data`

Backward-compatible alias for `result_data`.

**Returns:**

*/ Type: Any /*

The result data of this response.

### 69.3.5 Method: `from_dict`

Creates a `ServiceResponse` instance from a dictionary.

**Arguments:**

- `data`

*/ Condition: required / Type: dict /*

Dictionary containing response fields.

**Returns:**

*/ Type: ServiceResponse /*

New instance populated from the dictionary.

### 69.3.6 Method: `from_json`

Creates a `ServiceResponse` instance from a JSON string.

**Arguments:**

- `json_str`

*/ Condition: required / Type: str /*

JSON string containing response fields.

**Returns:**

*/ Type: ServiceResponse /*

New instance populated from the JSON string.

## 69.4 Class: `ServiceEvent`

*Imported by:*

```
from MicroserviceBase.domain.messages import ServiceEvent
```

Event emitted when service state changes.

### 69.4.1 Method: `to_dict`

Converts this `ServiceEvent` instance to a dictionary.

**Returns:**

*/ Type: dict /*

Dictionary representation of this event.

### 69.4.2 Method: to\_json

Converts this `ServiceEvent` instance to a JSON string.

**Returns:**

*/ Type: str /*  
JSON string representation of this event.

### 69.4.3 Method: from\_dict

Creates a `ServiceEvent` instance from a dictionary.

**Arguments:**

- `data`  
*/ Condition: required / Type: dict /*  
Dictionary containing event fields.

**Returns:**

*/ Type: ServiceEvent /*  
New instance populated from the dictionary.

### 69.4.4 Method: from\_json

Creates a `ServiceEvent` instance from a JSON string.

**Arguments:**

- `json_str`  
*/ Condition: required / Type: str /*  
JSON string containing event fields.

**Returns:**

*/ Type: ServiceEvent /*  
New instance populated from the JSON string.



# Chapter 70

## models.py

### 70.1 Class: MethodArgument

*Imported by:*

```
from MicroserviceBase.domain.models import MethodArgument
```

Describes a single argument of a service API method.

#### 70.1.1 Method: to\_dict

Converts this MethodArgument instance to a dictionary.

**Returns:**

*/ Type: dict /*

Dictionary representation of this argument.

#### 70.1.2 Method: from\_dict

Creates a MethodArgument instance from a dictionary.

**Arguments:**

- data

*/ Condition: required / Type: dict /*

Dictionary containing argument fields.

**Returns:**

*/ Type: MethodArgument /*

New instance populated from the dictionary.

### 70.2 Class: ServiceMethod

*Imported by:*

```
from MicroserviceBase.domain.models import ServiceMethod
```

Describes a single API method exposed by a service.

### 70.2.1 Method: to\_dict

Converts this `ServiceMethod` instance to a dictionary.

**Returns:**

*/ Type: dict /*

Dictionary representation of this method.

### 70.2.2 Method: from\_dict

Creates a `ServiceMethod` instance from a name and dictionary.

**Arguments:**

- name

*/ Condition: required / Type: str /*

The method name.

- data

*/ Condition: required / Type: dict /*

Dictionary containing method fields.

**Returns:**

*/ Type: ServiceMethod /*

New instance populated from the dictionary.

## 70.3 Class: ServiceInfo

*Imported by:*

```
from MicroserviceBase.domain.models import ServiceInfo
```

Describes a service and its metadata.

### 70.3.1 Method: to\_dict

Converts this `ServiceInfo` instance to a dictionary.

**Returns:**

*/ Type: dict /*

Dictionary representation of this service info.

### 70.3.2 Method: from\_dict

Creates a `ServiceInfo` instance from a dictionary.

**Arguments:**

- data

*/ Condition: required / Type: dict /*

Dictionary containing service info fields.

**Returns:**

*/ Type: ServiceInfo /*

New instance populated from the dictionary.

# Chapter 71

## service\_base.py

### 71.1 Class: ServiceBase

*Imported by:*

```
from MicroserviceBase.domain.service_base import ServiceBase
```

Pure domain base class for services.

All methods used to export APIs of a service must begin with the prefix 'svc\_api'. This class handles API discovery, docstring parsing, and request dispatch. Transport and registry interactions are delegated to injected ports.

#### 71.1.1 Method: get\_service\_info

Return the ServiceInfo dataclass for this service.

#### 71.1.2 Method: get\_service\_info\_dict

Return the service info as a plain dict (backward compat).

#### 71.1.3 Method: serve

Start serving requests via the transport port.

#### 71.1.4 Method: register\_service

Register this service via the registry port.

#### 71.1.5 Method: unregister\_service

Unregister this service via the registry port.

#### 71.1.6 Method: request\_service

Send an RPC request to another service via the transport port.

**Arguments:**

- request\_data  
/ *Condition*: required / *Type*: dict /  
Dict with 'method' and 'args' keys (backward compat format).

- `exchange_name`  
/ *Condition*: required / *Type*: str /  
The exchange to publish to.
- `routing_key`  
/ *Condition*: required / *Type*: str /  
The routing key for the target service.

**Returns:**

/ *Type*: dict /  
The response dict from the target service.

**71.1.7 Method: close**

Close the service.

**71.1.8 Method: create\_request\_data**

Create request data for a given method name and arguments.

**71.1.9 Method: get\_svc\_api\_methods\_dict**

Retrieve all service API methods (methods starting with 'svc\_api').

**71.1.10 Method: get\_svc\_api\_methods\_info\_dict**

Retrieve information for all service APIs from their docstrings.

**71.1.11 Method: parse\_docstring**

Parse function's docstring to get arguments and return information.

**71.1.12 Method: svc\_api\_get\_version**

Get the service version.

**Returns:**

/ *Type*: str /  
Version of the service.

**71.1.13 Method: svc\_api\_get\_gui\_files**

Compress and return GUI files as bytes.

**71.1.14 Method: svc\_api\_get\_service\_files**

Compress and return the entire service directory as bytes.

**Returns:**

/ *Type*: bytes /  
ZIP archive of the service directory, or None if not downloadable.

**71.1.15 Method: `svc_api_get_gui_checksum`**

Get an MD5 checksum of all GUI files.

Returns None when gui\_support is disabled or the GUIs directory is missing.

**Returns:**

*/ Type: str /*

MD5 hex-digest of the GUI files, or None.

**71.1.16 Method: `svc_api_shutdown`**

Gracefully shut down this service.

**71.1.17 Method: `is_specific_request`**

Check if the request is a specific request. Override in subclasses.

**71.1.18 Method: `on_specific_request`**

Handle a specific request. Override in subclasses.

**Arguments:**

- `request`

*/ Condition: required / Type: str /*

The request method name.

- `body`

*/ Condition: required / Type: dict /*

The parsed request body dict.

**Returns:**

*/ Type: ServiceResponse /*

A ServiceResponse, or None if not handled.

**71.1.19 Method: `dispatch_request`**

Dispatch a parsed request body and return a ServiceResponse.

This is the core domain logic: given a request dict with 'method' and 'args', find the matching API method, call it, and return a structured response.

**Arguments:**

- `request_body`

*/ Condition: required / Type: dict /*

Dict with 'method' and 'args' keys.

**Returns:**

*/ Type: ServiceResponse /*

ServiceResponse with result type and data.

### 71.1.20 Method: on\_request

Handle an incoming request (transport callback signature).

This method is called by the transport adapter. It delegates to `dispatch_request` for the actual domain logic, then uses the transport to send the response back.

#### Arguments:

- `ch`  
/ *Condition*: required / *Type*: object /  
Channel object from transport.
- `method`  
/ *Condition*: required / *Type*: object /  
Delivery method from transport.
- `props`  
/ *Condition*: required / *Type*: object /  
Message properties from transport.
- `body`  
/ *Condition*: required / *Type*: bytes /  
Raw message body (bytes or dict).

# Chapter 72

## service\_registry.py

### 72.1 Class: ServiceRegistry

*Imported by:*

```
from MicroserviceBase.domain.service_registry import ServiceRegistry
```

Pure domain registry that tracks services, handles aliases, and routes requests.

#### 72.1.1 Method: handle\_update

Handle a service registration/unregistration event.

**Arguments:**

- `service_information`  
/ *Condition*: required / *Type*: dict /  
Dict with 'info' and 'state' keys.

#### 72.1.2 Method: notify\_updates

Notify connected clients about service updates via the registry port.

#### 72.1.3 Method: svc\_api\_shutdown

Gracefully shut down the Service Registry.

#### 72.1.4 Method: svc\_api\_get\_services\_info

Retrieve information of all services connected to the broker.

**Returns:**

/ *Type*: dict /  
A dictionary containing information of all connected services.

#### 72.1.5 Method: svc\_api\_get\_realtime\_update\_exchange

Retrieve the exchange name of the realtime update exchange.

**Returns:**

/ *Type*: str /  
The name of the realtime update exchange.

### 72.1.6 Method: `svc_api_update_alias_conf`

Update the alias configuration information.

**Arguments:**

- `alias_string`  
/ *Condition*: required / *Type*: str /  
The alias configuration string to be updated.

**Returns:**

(no returns)

### 72.1.7 Method: `svc_api_get_alias_conf`

Retrieve the alias configuration string in JSON format.

**Returns:**

/ *Type*: str /  
The alias configuration string in JSON format.

### 72.1.8 Method: `is_specific_request`

Check if the request is an alias-routed request.

### 72.1.9 Method: `on_specific_request`

Handle an alias-routed request by forwarding to the target service.

### 72.1.10 Method: `handle_alias_request`

Handle an alias request by routing to the actual service.

**Arguments:**

- `body`  
/ *Condition*: required / *Type*: dict /  
Dict with 'method' and 'args' keys.

**Returns:**

/ *Type*: ServiceResponse /  
ServiceResponse or raw response dict from the target service.

### 72.1.11 Method: `run_health_check_loop`

Blocking loop for a daemon thread. Periodically checks whether each registered service still has active RabbitMQ consumers.

**Arguments:**

- `interval`  
/ *Condition*: optional / *Type*: int / *Default*: 30 /  
Seconds between health-check cycles.
- `max_failures`  
/ *Condition*: optional / *Type*: int / *Default*: 2 /  
Consecutive zero-consumer checks before auto-unregister.



- `broker_host`  
/ *Condition*: optional / *Type*: str / *Default*: 'localhost' /  
RabbitMQ broker host for the temporary connection.
- `broker_port`  
/ *Condition*: optional / *Type*: int / *Default*: 5672 /  
RabbitMQ broker port for the temporary connection.

#### **72.1.12 Method: `stop_health_check`**

Signal the health-check loop to stop.

## Chapter 73

# factory.py

### 73.1 Function: create\_transport

Create a TransportPort implementation.

**Arguments:**

- `transport_type`  
/ *Condition*: optional / *Type*: str / *Default*: 'rabbitmq' /  
Transport backend identifier. Supports 'rabbitmq' and 'eventbus'.
- `cmd_args`  
/ *Condition*: optional / *Type*: list /  
Command-line arguments for config parsing (used by 'rabbitmq').
- `service_name`  
/ *Condition*: optional / *Type*: str / *Default*: 'Service' /  
Service name for help text (used by 'rabbitmq').
- `config_path`  
/ *Condition*: optional / *Type*: str /  
Path to JSONP config file (used by 'eventbus').

**Returns:**

/ *Type*: TransportPort /  
A connected TransportPort instance.

### 73.2 Function: create\_registry

Create a ServiceRegistryPort implementation.

**Arguments:**

- `transport_type`  
/ *Condition*: optional / *Type*: str / *Default*: 'rabbitmq' /  
Transport backend identifier. Supports 'rabbitmq' and 'eventbus'.
- `cmd_args`  
/ *Condition*: optional / *Type*: list /  
Command-line arguments for config parsing (used by 'rabbitmq').

- `service_name`  
/ *Condition*: optional / *Type*: str / *Default*: 'Service' /  
Service name for help text (used by 'rabbitmq').
- `update_exchange_name`  
/ *Condition*: optional / *Type*: str /  
Name for the realtime update fanout exchange.
- `config_path`  
/ *Condition*: optional / *Type*: str /  
Path to JSONP config file (used by 'eventbus').

**Returns:**

/ *Type*: ServiceRegistryPort /  
A ServiceRegistryPort instance.

## 73.3 Function: create\_ui\_bridge

Create a UIBridgePort implementation.

**Arguments:**

- `bridge_type`  
/ *Condition*: optional / *Type*: str / *Default*: 'fastapi' /  
Bridge backend identifier. Currently supports 'fastapi'.
- `host`  
/ *Condition*: optional / *Type*: str / *Default*: 'localhost' /  
Host to bind to.
- `port`  
/ *Condition*: optional / *Type*: int / *Default*: 8000 /  
Port to bind to.

**Returns:**

/ *Type*: UIBridgePort /  
A UIBridgePort instance (not yet started).

# Chapter 74

## hub\_manager.py

### 74.1 Class: HubManagerPort

*Imported by:*

```
from MicroserviceBase.ports.hub_manager import HubManagerPort
```

Abstract interface for process/job orchestration backends.

Implementations manage service lifecycles through different backends: - LocalHubManager: local OS processes via ProcessHub - NomadHubAdapter: Nomad jobs via HTTP API

#### 74.1.1 Method: start\_hub

Start the orchestration backend.

**Arguments:**

- mode  
/ *Condition*: optional / *Type*: str / *Default*: 'standalone' /  
Operating mode. Backend-specific values.

**Returns:**

- dict with at least {"status": str}

#### 74.1.2 Method: stop\_hub

Stop the orchestration backend and release resources.

**Returns:**

- dict with at least {"status": str}

#### 74.1.3 Method: get\_status

Get a status snapshot of all managed services/jobs.

**Returns:**

- dict containing running, mode, processes list, etc.

#### 74.1.4 Method: start\_processes

Start named services/jobs.

**Arguments:**

- `names`  
/ *Condition*: required / *Type*: list[str] /  
List of service/process names to start.

**Returns:**

- dict mapping name to {"success": bool, "message": str}

**74.1.5 Method: stop\_processes**

Stop named services/jobs.

**Arguments:**

- `names`  
/ *Condition*: required / *Type*: list[str] /  
List of service/process names to stop.
- `force`  
/ *Condition*: optional / *Type*: bool / *Default*: False /  
Skip graceful shutdown and force-kill immediately.

**Returns:**

- dict mapping name to {"success": bool, "message": str}

**74.1.6 Method: get\_config**

Get all service/job configurations.

**Returns:**

- dict of process/job configs keyed by name.

**74.1.7 Method: add\_config**

Add a new service/job configuration.

**Arguments:**

- `name`  
/ *Condition*: required / *Type*: str /
- `config`  
/ *Condition*: required / *Type*: dict /

**Returns:**

- dict with {"success": bool, "message": str}

**74.1.8 Method: update\_config**

Update an existing service/job configuration.

**Returns:**

- dict with {"success": bool, "message": str}

#### 74.1.9 Method: `remove_config`

Remove a service/job configuration.

**Returns:**

- dict with {"success": bool, "message": str}

#### 74.1.10 Method: `get_service_log`

Read service/job log output.

**Returns:**

- dict with {"name": str, "log": str}

#### 74.1.11 Method: `import_service`

Import a service package.

**Returns:**

- dict with {"success": bool, "message": str}

#### 74.1.12 Method: `remove_service`

Remove a service entirely (stop + delete config + files).

**Returns:**

- dict with {"success": bool, "message": str}

#### 74.1.13 Method: `reset`

Reset the hub — stop all services and clear state.

**Returns:**

- dict with {"success": bool, "message": str}

# Chapter 75

## registry.py

### 75.1 Class: ServiceRegistryPort

*Imported by:*

```
from MicroserviceBase.ports.registry import ServiceRegistryPort
```

Abstract interface for service registration and discovery.

Implementations handle how services announce themselves and how the registry discovers/tracks them.

#### 75.1.1 Method: register

Register a service with the registry.

**Arguments:**

- `service_info`  
/ *Condition*: required / *Type*: dict /  
Dict containing service metadata.

#### 75.1.2 Method: unregister

Unregister a service from the registry.

**Arguments:**

- `service_info`  
/ *Condition*: required / *Type*: dict /  
Dict containing service metadata.

#### 75.1.3 Method: subscribe\_to\_events

Subscribe to service registration/unregistration events.

**Arguments:**

- `handler`  
/ *Condition*: required / *Type*: callable /  
Callback function(ch, method, properties, body) for events.

#### 75.1.4 Method: `notify_update`

Broadcast a services update to all subscribers.

**Arguments:**

- `services_info`  
/ *Condition*: required / *Type*: dict /  
Dict of all current service information.

#### 75.1.5 Method: `get_update_channel_name`

Return the name of the realtime update channel/exchange.

**Returns:**

Channel/exchange name as a string.



# Chapter 76

## transport.py

### 76.1 Class: TransportPort

*Imported by:*

```
from MicroserviceBase.ports.transport import TransportPort
```

Abstract interface for message transport.

Implementations provide the actual connectivity (e.g., RabbitMQ, Redis, MQTT). The domain layer depends only on this interface.

#### 76.1.1 Method: connect

Establish connection to the message broker.

#### 76.1.2 Method: disconnect

Close connection to the message broker.

#### 76.1.3 Method: stop\_consuming

Signal the consume loop to stop.

#### 76.1.4 Method: consume

Start consuming messages for a service.

**Arguments:**

- `service_name`  
/ *Condition*: required / *Type*: str /  
Name of the service (used as queue name).
- `routing_key`  
/ *Condition*: required / *Type*: str /  
Routing key to bind the queue.
- `exchange`  
/ *Condition*: required / *Type*: str /  
Exchange name to bind to.
- `handler`  
/ *Condition*: required / *Type*: callable /  
Callback function(ch, method, props, body) for incoming messages.

### 76.1.5 Method: `rpc_call`

Send an RPC request and wait for the response.

**Arguments:**

- `request_data`  
/ *Condition*: required / *Type*: dict /  
Dict to serialize and send as the request body.
- `exchange_name`  
/ *Condition*: required / *Type*: str /  
Exchange to publish to.
- `routing_key`  
/ *Condition*: required / *Type*: str /  
Routing key for the target service.

**Returns:**

Parsed response dict from the target service.

### 76.1.6 Method: `publish`

Publish a message to an exchange.

**Arguments:**

- `exchange`  
/ *Condition*: required / *Type*: str /  
Exchange name.
- `routing_key`  
/ *Condition*: required / *Type*: str /  
Routing key.
- `body`  
/ *Condition*: required / *Type*: str or bytes /  
Message body.
- `properties`  
/ *Condition*: optional / *Type*: dict /  
Optional message properties dict.

# Chapter 77

## ui\_bridge.py

### 77.1 Class: UIBridgePort

*Imported by:*

```
from MicroserviceBase.ports.ui_bridge import UIBridgePort
```

Abstract interface for UI communication bridge.

Implementations provide the actual UI connectivity (e.g., FastAPI REST, gRPC). Any frontend (Electron, browser, mobile) can connect through this port.

#### 77.1.1 Method: start

Start the UI bridge server.

**Arguments:**

- `request_handler`  
/ *Condition*: required / *Type*: callable /  
Callable that accepts a `ServiceRequest` dict and returns a `ServiceResponse` dict. Used to route UI requests to services.
- `services_info_provider`  
/ *Condition*: required / *Type*: callable /  
Callable that returns the current services info dict.

#### 77.1.2 Method: stop

Stop the UI bridge server.

#### 77.1.3 Method: broadcast\_update

Push a services update to all connected UI clients.

**Arguments:**

- `services_info`  
/ *Condition*: required / *Type*: dict /  
Dict of all current service information.

## Chapter 78

### `__init__.py`

MicroserviceBase runtime library.

Public API used by service authors:

- `ServiceRunner` — start a gRPC service with Consul registration, reflection, and graceful shutdown.
- `ServiceEntry` — one service to bind to the gRPC server.
- `ConsulRegistration` — low-level Consul register/deregister.
- `BaseServiceSettings` — Pydantic parent class for service settings.

# Chapter 79

## consul.py

Consul service registration.

Direct HTTP client (no python-consul dependency). Consul's HTTP API is small and stable; adding a third-party wrapper adds a dependency without value.

The ConsulRegistration class is used as an async context manager:

```
async with ConsulRegistration(  
    name="hello",  
    address="127.0.0.1",  
    port=50051,  
    consul_addr="http://localhost:8500",  
):  
    await server.wait_for_termination()
```

The service is registered when the context is entered and deregistered on exit. A gRPC health check is registered against the same port so Consul can mark the service healthy/unhealthy automatically.

### 79.1 Class: ConsulRegistration

*Imported by:*

```
from MicroserviceBase.runtime.consul import ConsulRegistration
```

Async context manager that registers a service with Consul on entry and deregisters it on exit.

The service ID is generated from the service name and a short random suffix so multiple instances of the same service can coexist on the same node.

**Args:** name: Logical service name, e.g. "hello". address: Address other services should use to reach this one. port: gRPC port. consul\_addr: Consul HTTP API endpoint. tags: Optional Consul tags (e.g. ["v1", "demo"]). meta: Optional key/value metadata attached to the service. token: Optional Consul ACL token. health\_interval: How often Consul probes the gRPC health endpoint. health\_timeout: Health probe timeout. deregister\_after: Deregister the service if critical for this long.

#### 79.1.1 Method: service\_id

## Chapter 80

# reflection.py

gRPC reflection helper.

Enables the standard gRPC server reflection protocol so clients can list services and methods at runtime — which is how `MicroserviceManagerGUI` discovers the method surface of each registered service without needing to know each `.proto` ahead of time.

### 80.1 Function: `enable_reflection`

Register the reflection service on *server*.

**Args:** `server`: The running async gRPC server. `service_names`: Fully-qualified service names to advertise, e.g. `["hello.v1.HelloService", ...]`. The reflection service itself is added automatically.

# Chapter 81

## server.py

Service runtime.

`ServiceRunner` is the single entry point a service author uses to:

- Start an async gRPC server
- Bind a configurable list of servicers
- Enable gRPC reflection so clients can discover methods
- Enable the standard gRPC health check protocol
- Register the service with Consul using the (possibly OS-assigned) port
- Deregister + shut down gracefully on `SIGINT/SIGTERM/SIGBREAK`

Usage (simplified):

```
runner = ServiceRunner(
    service_name="hello",
    servicers=[(hello_pb2_grpc.add>HelloServiceServicer>to>server,
                hello_pb2.DESRIPTOR.services_by_name["HelloService"].full_name,
                HelloGrpcAdapter(domain))],
    settings=settings,
)
await runner.serve_forever()
```

The `servicers` argument is a list of tuples: `(register_fn, full_service_name, servicer_instance)`.

### 81.1 Class: `ServicerEntry`

*Imported by:*

```
from MicroserviceBase.runtime.server import ServicerEntry
```

One gRPC servicer to bind to the server.

**Attributes:** `register_fn`: The `add_<Name>Servicer_to_server` function generated by `grpcio.tools`.

`full_service_name`: Fully-qualified proto service name (e.g. `"hello.v1>HelloService">HelloService`). Used for reflection + optional Consul metadata.

`servicer`: The servicer instance implementing the RPCs.

## 81.2 Class: ServiceRunner

*Imported by:*

```
from MicroserviceBase.runtime.server import ServiceRunner
```

Run an async gRPC service with Consul registration and graceful shutdown.

**Args:** `servicers`: List of `ServiceEntry` describing what to expose. `settings`: The service's settings instance (a `BaseServiceSettings` subclass). `tags`: Optional Consul tags. `shutdown_grace_sec`: Grace period for in-flight RPCs on shutdown.

`settings.service_name` must be non-empty; it's the logical name other services and the GUI use to find this one.  
microservicebase-runtime-server-servicerunner-request-shutdown -----

Trigger graceful shutdown. Safe to call from a signal handler.



## Chapter 82

# settings.py

Base Pydantic settings for microservices.

Every service subclasses `BaseServiceSettings` and adds its own fields. Environment variables are loaded with a service-specific prefix.

**Example:** `class HelloSettings(BaseServiceSettings):` `model.config = SettingsConfigDict(env_prefix="HELLO", env_file=".env")`  
`greeting: str = "Hello"`

The base provides the fields that every service needs to talk to Consul and expose a gRPC server.

### 82.1 Class: BaseServiceSettings

*Imported by:*

```
from MicroserviceBase.runtime.settings import BaseServiceSettings
```

Common settings shared by all microservices.

Subclasses add service-specific fields and set `env_prefix` to the service's name (e.g. `HELLO_`).

**Attributes:** `service_name`: Logical service name used for Consul registration. `service_host`: Address the gRPC server binds to. `0.0.0.0` is the default so the service is reachable on any interface; Consul registers the advertised address from `advertise_addr` if set, otherwise falls back to the local hostname. `grpc_port`: TCP port for the gRPC server. `0` means "let the operating system pick a free port". The chosen port is then announced to Consul. `advertise_addr`: Address announced to Consul. Leave empty to use the local hostname. `consul_addr`: Consul HTTP API endpoint (e.g. `http://localhost:8500`). `consul_token`: Optional ACL token for Consul. `log_level`: Python logging level name (`DEBUG`, `INFO`, ...).

## Chapter 83

# scaffold\_cli.py

Generate microservice scaffolds via the bridge's HTTP API.

Does the same job as the Service Creator wizard in the Manager GUI, but from a terminal — so you can script it, commit reproducible specs, or run from CI.

Two ways to supply input:

1. **Config file** (YAML or JSON) with any subset of the `/api/scaffold/generate-v2` fields:

```
python -m MicroserviceBase.tools.scaffold_cli --config my_service.yaml
```

2. **Inline flags** for the common cases:

```
python -m MicroserviceBase.tools.scaffold_cli \  
    --name Hello --language python --output ./out
```

Config + flags compose: flags override fields loaded from the config.

Importing an existing .proto is supported via `--proto path/to/file.proto`. Pass `--monorepo` to emit one project folder with N executables (one per service declared in the .proto).

Requires the bridge to be running (default `http://127.0.0.1:1112`; set via `--bridge-url` or the `MB_BRIDGE_URL` env var).

### 83.1 Function: main

# Chapter 84

## Appendix

### 84.1 Appendix

#### 84.1.1 License

MicroserviceBase is licensed under the Apache License, Version 2.0.

You may obtain a copy of the License at: <http://www.apache.org/licenses/LICENSE-2.0>

#### 84.1.2 References

- RabbitMQ: <https://www.rabbitmq.com>
- pika (RabbitMQ client for Python): <https://pika.readthedocs.io>
- FastAPI: <https://fastapi.tiangolo.com>
- Electron: <https://www.electronjs.org>
- Python: <https://www.python.org>

#### 84.1.3 Repository

Source code: <https://github.com/test-fullautomation/python-microservice-base>

Issue tracker: <https://github.com/test-fullautomation/python-microservice-base/issues>

#### 84.1.4 Contact

Author: Nguyen Huynh Tri Cuong

Email: [Cuong.NguyenHuynhTri@vn.bosch.com](mailto:Cuong.NguyenHuynhTri@vn.bosch.com)

# Chapter 85

## History

### 85.1 History

#### 85.1.1 Version 2.0.0 - 09.02.2026

Major restructure: hexagonal architecture, dual-host GUI, multi-broker support, local process hub, standalone installer.

##### Architecture

- **Hexagonal Architecture:** Reorganized codebase into domain, ports, and adapters layers
- **Factory Pattern:** All components created through `factory.py` for easy swapping
- **Port Interfaces:** `TransportPort`, `RegistryPort`, `UIBridgePort`
- **Adapter Implementations:** RabbitMQ transport, RabbitMQ registry, FastAPI bridge, Local Hub Manager, Service Executor

##### GUI v2.0

- **Dual Hosting:** Electron desktop app and browser mode via FastAPI bridge
- **Pure Web Frontend:** HTML/CSS/JS with Bootstrap 5 CDN (no Node.js dependencies)
- **Namespace:** `window.MicroserviceManager` (alias MM) replaces Node.js global and `require()` patterns
- **Service Plugins:** Dynamically loaded per-service UI components in `web/services/`
- **Dark Sidebar Theme:** Custom Bootstrap dark sidebar with accordion navigation
- **Login Modal:** Bootstrap modal replaces separate Electron popup window
- **Toast Notifications:** `MM.showToast()` replaces `notifier.notify()` and `dialog.showMessageBox()`

##### Multi-Broker Support

- **Connection Map:** `MM.connections` tracks per-broker state
- **Reverse Lookup:** `serviceToBroker` maps service names to broker endpoints
- **Progressive Disclosure:** Broker headers shown only with multiple connections
- **Session Persistence:** Connection state saved via `sessionStorage`

## Local Process Hub

- **Hub Manager:** Start, stop, and monitor local service processes
- **Service Executor:** Process execution with `CTRL_BREAK_EVENT` for graceful shutdown on Windows
- **Config Persistence:** `hub_processes.json` with `${python}` and `${config_dir}` placeholders preserved across save operations
- **Hub Dashboard:** Real-time process status display in the GUI
- **REST API:** Full hub management via `/api/local-hub/*` endpoints

## Service Management

- **Service Import:** Import microservice packages from folders or zip archives
- **Service Creator:** Interactive wizard for scaffolding new microservices
- **Registry Shutdown:** `__registry_shutdown__` sentinel notifies subscribers when a service unregisters

## Electron Packaging

- **Standalone Installer:** NSIS installer for Windows (`build.bat`), Linux packages (`build.sh`)
- **Read-only Resources:** Scripts in `resources/python/`, app in `app.asar`
- **Writable User Data:** Settings, config, services, logs in `%APPDATA%/devatservgui/`
- **First-Run Init:** Template files copied from resources on first launch
- **Bridge Lifecycle:** Detached bridge process managed via PID file

## Windows Process Fixes

- `SIGBREAK` handler converted to `KeyboardInterrupt` for graceful Python cleanup
- `Pika.start_consuming()` replaced with `process_data.events(time_limit=1)` loop for interruptible consumption
- `subprocess.PIPE` blocking prevented by using `FileHandler` instead of `StreamHandler` for logging in subprocess mode

## Documentation

- 12 Architecture Decision Records (ADR-001 through ADR-012)
- 12 PlantUML diagrams (overview, architecture, component, class, sequence, state)
- Updated README with architecture diagrams, quick start guide, and API reference
- Package documentation with Introduction, Description, and History

### 85.1.2 Version 0.1.0 - 02/2023

Initial version.

- Basic microservice framework with RabbitMQ communication
- Service registration and discovery
- Electron-based GUI for service monitoring

---

**MicroserviceBase.pdf**

*Created at 08.05.2026 - 15:24:01*

*by GenPackageDoc v. 0.16.0*

---